

TRABAJO FIN DE GRADO
INGENIERÍA EN INFORMÁTICA

**Entrenamiento eficiente de un agente de
aprendizaje por refuerzo mediante la
generación automática de currículos con
IA neuro-simbólica**

—
Septiembre de 2026

Entrenamiento eficiente de un agente de aprendizaje por refuerzo mediante la generación automática de currículos con IA neuro-simbólica

Palabras clave: aprendizaje por refuerzo, aprendizaje profundo, inteligencia artificial neuro-simbólica, *curriculum learning*, *automatic curriculum generation*, planificación automática

Resumen

Los métodos de generación automática de currículos tienen como objetivo la creación de datos sintéticos para entrenar a un agente inteligente a resolver una serie de tareas de la forma más eficiente posible. Estos métodos generan tareas cuya dificultad depende del rendimiento del agente resolutor de manera que, la complejidad progresa junto con el aprendizaje del agente. Este aumento gradual de la dificultad permite al agente aprender de manera eficiente, es decir, utilizando pocos datos.

La mayoría de métodos de generación de currículo asumen la existencia a priori de un generador de problemas/tareas. Esto tiene dos desventajas principales: 1) limita la aplicación a tareas para las que existe un generador parametrizado y 2) a veces no permite controlar de manera precisa las características de los problemas resultantes.

Este TFG propone explorar una nueva estrategia en generación de currículos mediante la generación precisa de problemas haciendo uso de métodos de IA neuro-simbólica. Más específicamente, se propone utilizar NeSIG como generador, un método neuro-simbólico para la generación de problemas de planificación automática. Como resolutor, se utiliza un agente de aprendizaje por refuerzo que aprende a resolver una serie de tareas relacionadas. De esta forma, NeSIG se entrena para generar problemas de dificultad adaptada al rendimiento actual del agente, el cual aprende progresivamente a resolver problemas de dificultad cada vez mayor.

Los resultados obtenidos muestran que, a pesar de que el ajuste iterativo de NeSIG mejora el rendimiento en *success rate* más de 30 puntos porcentuales respecto a versiones sin dicho ajuste, el sistema propuesto no logra igualar a dos métodos de currículo más simples: obtiene un *success rate* medio un 30 % inferior y un área bajo la curva de 0.43 frente a 0.79 y 0.84. Además, se ha detectado que NeSIG genera una cantidad de problemas triviales elevada, lo que limita la calidad de la señal de entrenamiento recibida por el agente. En conjunto, estos resultados muestran que, aunque el enfoque presenta una motivación teórica sólida, su complejidad introduce una inestabilidad y un coste adicionales que no llegan a compensarse con una mejora de rendimiento.

Efficient training of a reinforcement learning agent through automatic curriculum generation using neuro-symbolic AI

Keywords: machine learning, deep learning, neuro-symbolic artificial intelligence, curriculum learning, automatic curriculum generation, automated planning

Abstract

Automatic curriculum generation methods aim to create synthetic data to train an intelligent agent to solve a set of tasks as efficiently as possible. These methods generate problems whose difficulty depends on the performance of the solving agent, such that complexity increases alongside the agent’s development. This gradual increase in difficulty enables the agent to learn efficiently, i.e., using fewer data samples.

Most curriculum generation methods assume the prior existence of a problem/task generator. This has two main drawbacks: (1) it limits applicability to tasks for which a parameterized generator already exists, and (2) it does not always allow precise control over the characteristics of the resulting problems.

This Bachelor’s Thesis proposes exploring a new strategy in curriculum generation through the precise generation of problems using neuro-symbolic AI methods. More specifically, we propose using NeSIG as the generator, a neuro-symbolic method for generating automated planning problems. As the solver, a reinforcement learning agent is used, which learns to solve a set of related tasks. In this way, NeSIG is trained to generate problems with difficulty adapted to the agent—neither too easy nor too difficult—while the agent progressively learns to solve increasingly complex problems.

The results obtained show that, although the iterative fine-tuning of NeSIG improves the success rate by more than 30 percentage points compared to versions without such fine-tuning, the proposed system fails to match two simpler curriculum methods: it obtains a mean success rate 30 % lower and a considerably lower AUC. Furthermore, NeSIG was found to generate a large quantity of trivial problems, which limits the quality of the training signal received by the agent. Overall, these results show that, although the neuro-symbolic approach offers a sound theoretical motivation, its added complexity introduces instability and additional computational cost that are not offset by a corresponding performance improvement.

Índice general

1. Introducción	1
1.1. Definición del problema	1
1.2. Motivación	3
1.3. Objetivos	4
1.4. Planificación económica y temporal	5
2. Fundamentos Teóricos	9
2.1. Planificación automática	9
2.1.1. <i>Planning domain definition language</i> (PDDL)	11
2.2. Aprendizaje por refuerzo	14
2.2.1. Elementos básicos del aprendizaje por refuerzo	15
2.2.2. Interacción agente-entorno y procesos de decisión de Markov	17
2.2.3. Aprendizaje por refuerzo profundo	21
2.2.4. Desafíos del aprendizaje por refuerzo profundo	25
2.3. <i>Neural logic machines</i>	27
2.4. NeSIG: <i>Neuro-Symbolic Instance Generator</i>	32
2.5. <i>Curriculum learning</i> : conceptos básicos	35
2.5.1. <i>Automated curriculum learning</i>	35
2.5.2. <i>Automated curriculum generation</i>	37
3. Estado del Arte	39
3.1. Orígenes del <i>curriculum learning</i>	40
3.2. ACL en aprendizaje por refuerzo	41
3.2.1. Estrategias <i>harder-first</i> y <i>easier-first</i>	44
3.2.2. <i>Benchmarks</i> y entornos de evaluación	44
3.3. ACG en la literatura reciente	45
4. Método Propuesto	51
4.1. Descripción general del sistema	51
4.1.1. Agente base	51
4.1.2. Generador neuro-simbólico de problemas	53
4.2. Entrenamiento de la arquitectura	55

4.3. Detalles técnicos de la implementación	58
5. Experimentación	63
5.1. Diseño experimental	63
5.1.1. Entorno de ejecución	63
5.1.2. Dominio experimental	64
5.1.3. Datos	65
5.1.4. Métricas	66
5.1.5. Hiperparámetros del método propuesto	68
5.1.6. Métodos del estudio de ablación	69
5.1.7. Métodos de referencia (<i>baselines</i>)	70
5.1.8. Experimentos realizados	71
5.2. Resultados experimentales	71
5.2.1. Experimento 1: ajuste de hiperparámetros	71
5.2.2. Experimento 2: estudio de ablación	74
5.2.3. Experimento 3: comparativa de <i>automated curriculum generation</i> basado en NeSIG con métodos <i>baseline</i> . .	78
5.3. Discusión	83
5.3.1. Resumen de resultados	84
5.3.2. El problema de la generación de instancias triviales . .	86
5.3.3. Por qué el método propuesto no supera a los <i>baselines</i> . .	88
6. Conclusiones	91
Bibliografía	93
Apéndice A. Blocksworld	101
Glosario de acrónimos	103

Índice de figuras

1.1. Ilustración visual sobre el funcionamiento de CL para aprendizaje supervisado.	2
1.2. Esquema simple del sistema de ACG propuesto en este TFG.	5
2.1. Representación visual de un problema del dominio <i>Blocksworld</i>	10
2.2. Ejemplo de dominio en <i>Blocksworld</i> junto con una instancia de problema.	12
2.3. Esquema del proceso de RL.	15
2.4. Ejemplo de RL en un problema de <i>Gridworld</i>	17
2.5. Esquema que muestra la diferencia entre políticas determinadas y estocásticas.	18
2.6. Ilustración visual del ciclo entre el agente y el entorno.	19
2.7. Esquema de RL profundo.	22
2.8. Funcionamiento simple de una NLM.	28
2.9. Representación interna de una NLM.	31
2.10. Esquema de entradas y salidas de NeSIG.	32
2.11. Esquema de funcionamiento interno de NeSIG.	34
2.12. Ilustración visual sobre el funcionamiento de CL.	36
2.13. Ilustración visual sobre el funcionamiento de ACG.	37
3.1. Representación gráfica del número de publicaciones en CL a lo largo de los años.	40
3.2. Representación gráfica del número de publicaciones en ACG a lo largo de los años.	40
3.3. Comparativas de un modelo de lenguaje entrenado con y sin currículo.	41
3.4. Clasificación de tipos de CL en RL.	42
3.5. Esquema de interacción del método profesor-estudiante en ACL.	43
3.6. Esquema de funcionamiento del método ACL <i>Self-Play</i>	47
3.7. Esquema de funcionamiento del método ACL <i>setter-solver</i>	48
3.8. Esquema de funcionamiento del método ACL CARML	49
4.1. Esquema visual con la estructura completa del sistema ACG propuesto.	52

4.2.	Esquema del proceso temporal de co-entrenamiento entre Ne-SIG y el agente de RL.	57
4.3.	Diagrama de clases de la implementación del sistema ACG. . .	60
4.4.	Diagrama de secuencia del proceso de entrenamiento del sistema ACG.	61
5.1.	Distribución del tamaño de los problemas del conjunto de evaluación.	66
5.2.	Ejemplo del problema de generación de instancias triviales durante el entrenamiento.	76
5.3.	Gráfico de cajas de los resultados del <i>success rate</i> del experimento 2.	78
5.4.	Gráfico de cajas de los resultados del <i>success rate</i> del experimento 3.	83
5.5.	Gráfica comparativa del mejor <i>success rate</i> obtenido por cada método.	84
5.6.	Tasa de éxito media según el nivel de dificultad.	85

Índice de tablas

1.1. Planificación inicial del proyecto.	6
1.2. Planificación modificada del proyecto en febrero.	7
1.3. Planificación final del proyecto.	7
1.4. Desglose de costes asociados al proyecto.	7
5.1. Configuración de la arquitectura NLM del agente y NeSIG. . .	68
5.2. Hiperparámetros del agente de RL básico.	68
5.3. Hiperparámetros del generador de instancias NeSIG.	69
5.4. Extensión de hiperparámetros de la Tabla 5.2 para el método <i>baseline</i> de ACG con generador parametrizado.	71
5.5. Resumen de los experimentos realizados.	72
5.6. Resultados del experimento 1 para el hiperparámetro de la diversidad.	73
5.7. Resultados del experimento 1 para el <i>learning rate</i>	74
5.8. Resultados del experimento 1 para el número de épocas de PPO.	74
5.9. Resumen de sensibilidad del método a los hiperparámetros estudiados.	75
5.10. Resultados comparativos del experimento 2 sobre el <i>success</i> <i>rate</i>	77
5.11. Resultados comparativos del experimento 2 sobre la estabilidad. . .	77
5.12. Resultados comparativos del experimento 2 sobre el promedio de problemas triviales.	79
5.13. Resultados comparativos del experimento 3 sobre el <i>success</i> <i>rate</i>	80
5.14. Resultados comparativos del experimento 3 sobre el AUC. . .	80
5.15. Resultados comparativos del experimento 3 sobre el coste computacional.	81
5.16. Resultados comparativos del experimento 3 sobre la estabilidad. . .	82
5.17. Recopilación del <i>success rate</i> y las métricas de estabilidad obtenidas.	86
5.18. Recopilación del AUC y el tiempo de ejecución.	86

5.19. Recopilación del promedio de problemas triviales del experimento 2.	87
---	----

Capítulo 1

Introducción

1.1. Definición del problema

Actualmente, existe una inversión y un interés en investigación crecientes dentro del campo de la inteligencia artificial (IA) [1] [2], y en particular, en el aprendizaje automático (ML) [3] [4] [5] [6], disciplina que engloba ramas como el aprendizaje por refuerzo [7] y el aprendizaje profundo (DL) [8] [9] [10] [11]. El entrenamiento de modelos punteros en estos campos supone una necesidad cada vez mayor de recursos que sean capaces de satisfacer la enorme demanda. Entre las principales limitaciones se encuentran la elevada cantidad de datos necesarios y la potencia de cálculo requerida para ser competitivo en el panorama global actual. En muchos entornos, la obtención de datos suficientemente representativos es difícil. Esto motiva la búsqueda de estrategias que optimicen el proceso de entrenamiento.

En este contexto, *curriculum learning* (CL) es un paradigma que propone estructurar el entrenamiento con una sucesión progresiva de problemas llamada currículum. Partiendo de ejemplos sencillos, se introduce dificultad de forma gradual hasta llegar a los ejemplos más complejos. De forma análoga, una persona que aprende a resolver rompecabezas no comienza enfrentándose a configuraciones altamente complejas. Inicialmente practica con instancias sencillas, con pocas piezas y estructuras claras, y progresivamente afronta configuraciones más grandes y desordenadas a medida que adquiere habilidad. La aplicación de este enfoque a los modelos de ML persigue mejorar la eficiencia del aprendizaje, reducir el coste computacional y mitigar la inestabilidad durante la optimización [12] [13] [14].

En las primeras propuestas, la elaboración de los currículos se realizaba de forma manual. El proceso requería conocimiento experto sobre el dominio para estimar la dificultad de cada tarea y establecer una ordenación adecuada [12]. Dicho proceso resultaba laborioso y difícilmente escalable, es-

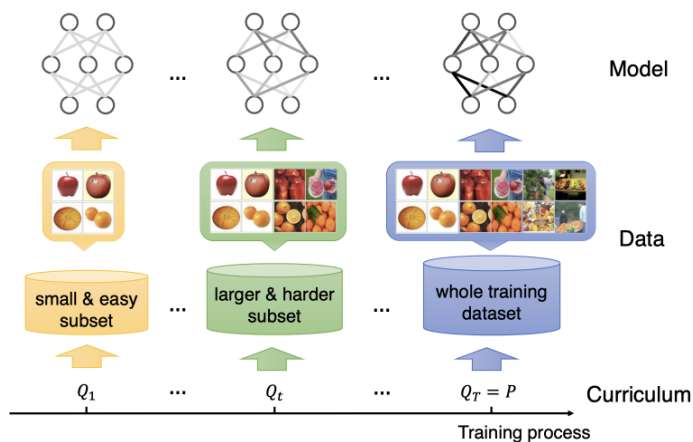


Figura 1.1: Ilustración visual sobre el funcionamiento de CL para aprendizaje supervisado. El proceso de entrenamiento transcurre de forma progresiva. Las primeras muestras de entrenamiento resultan sencillas de identificar, piezas de frutas fácilmente distinguibles. En el siguiente paso, se agregan imágenes más complicadas con fondos que dificultan distinguir las frutas. Finalmente, se entrena el modelo sobre todo el conjunto de muestras, incluyendo las imágenes más complicadas. Figura extraída de [15].

pecialmente en dominios complejos donde inferir la dificultad no es evidente. Para abordar esta limitación, se planteó la automatización de la selección de currículos, conocida como *automated curriculum learning* (ACL) [13]. Estos métodos facilitan la creación de un currículo mediante mecanismos automáticos que seleccionan de forma inteligente los datos de entrenamiento, lo que permite el desarrollo de sistemas más generales y flexibles.

Si ACL se centra en automatizar la selección y organización de los problemas de entrenamiento, *automated curriculum generation* (ACG) amplía este enfoque al incorporar la generación automática de los datos que componen el currículo [13]. Esto permite que el proceso de selección quede integrado de manera implícita dentro del propio sistema, de modo que el currículo se construye en función del estado de aprendizaje del modelo en cada momento. Cuando un conjunto de muestras deja de aportar aprendizaje significativo, el sistema genera nuevas instancias que suponen un mayor desafío. Por ejemplo, en un entorno de entrenamiento como el de los videojuegos, puede considerarse un sistema compuesto por un agente entrenado para resolver laberintos y un segundo modelo encargado de generar instancias de problemas en función de su rendimiento. Si el agente resuelve con facilidad los laberintos actuales, se crearán ejemplos más complejos; mientras que si resultan demasiado complicados, se reducirá la dificultad. De este modo, el currículo se adapta de forma dinámica al nivel de competencia del agente, ajustando

la dificultad de los problemas durante el propio proceso de entrenamiento.

Este paradigma ha sido aplicado tradicionalmente tanto en aprendizaje supervisado como en RL. No obstante, en los últimos años el RL ha ganado un protagonismo especial en este ámbito [13] [16]. Sus entornos suelen presentar dificultades añadidas, como recompensas escasas y espacios de estados muy amplios, que complican el proceso de exploración. En este contexto, ACG permite guiar dicha exploración mediante la generación progresiva de problemas que definan objetivos personalizados para el agente. De esta forma, problemas complejos pueden ser fragmentados en subproblemas más sencillos de aprender a resolver.

Sin embargo, trasladar estas ventajas a la práctica no resulta sencillo. Existe un número limitado de sistemas de ACG capaces de generar entornos de forma dinámica. Además, la mayoría de las propuestas disponibles dependen de la existencia previa de un generador de problemas parametrizado. Esta dependencia no solo limita la aplicación de ACG a problemas para los que exista dicho generador, sino que también dificulta el control preciso de las características de los problemas resultantes. Esto representa un gran problema en RL, donde existe una gran diversidad de entornos y muchos de ellos carecen de infraestructura que permita la generación explícita de problemas o datos. Asimismo, los dominios con menor disponibilidad de datos suelen coincidir con aquellos menos explorados y con menor volumen de investigación, lo que agrava aún más la dificultad de aplicar técnicas de aprendizaje en estos contextos.

1.2. Motivación

ACG permite generar datos de entrenamiento y adaptarlos formando un currículo. Esto favorece la exploración eficiente de los problemas y contribuye a reducir el tiempo y el coste del aprendizaje de los modelos, acelerar la convergencia y aumentar la capacidad de generalización [12] [14] [13]. En conjunto, este enfoque facilita el desarrollo de agentes más robustos y contribuye a mitigar algunas limitaciones actuales del ML.

Estas ventajas resultan especialmente relevantes en RL, donde muchos entornos presentan recompensas escasas o poco informativas. En estas circunstancias, los agentes pueden requerir un número muy elevado de interacciones con el entorno antes de recibir señales útiles para el aprendizaje. En problemas donde únicamente se obtiene una recompensa por tener éxito, el agente puede acumular múltiples episodios fallidos recibiendo una señal de recompensa constante, lo que dificulta identificar qué acciones son positivas. En lugar de entrenar directamente con problemas completos, ACG permite generar instancias más sencillas en las que el agente puede alcanzar la

solución con mayor facilidad y obtener recompensas informativas.

Por ello, ACG se presenta como un enfoque relevante para abordar algunas de las dificultades inherentes al entrenamiento de agentes de RL. No obstante, su desarrollo aún se encuentra en una fase temprana, y los métodos existentes presentan diversas limitaciones. La literatura existente en ACG no es muy amplia (véase la Figura 3.1) y no existen marcos formales bien establecidos que unifiquen las distintas propuestas. En este contexto, la superación de dichas limitaciones podría convertir estas técnicas en un elemento central en el desarrollo de agentes. Permitiría su entrenamiento en multitud de problemas, independientemente de la cantidad de datos disponibles o de la densidad de las recompensas. Este potencial, junto con la falta de investigación en el área, motiva el desarrollo de nuevos métodos orientados a propuestas más generales y robustas.

Por esta razón, en este trabajo se propone explorar el campo de ACG mediante el estudio de métodos capaces de generar datos de entrenamiento. En la literatura actual existen diversas propuestas centradas en la generación de problemas de forma independiente del dominio. Entre enfoques recientes, destaca NeSIG [17], un método neuro-simbólico [18] que combina redes neuronales y razonamiento lógico para la generación de problemas en cualquier entorno. NeSIG genera problemas a partir de descripciones PDDL, un lenguaje formal, basado en la lógica de primer orden, para codificar problemas y dominios de planificación.

Así, el uso de NeSIG como herramienta para la generación de problemas de entrenamiento en sistemas de ACG representa una línea aún no explorada en la literatura. La novedad y potencial de la propuesta constituyen la motivación principal de este trabajo.

1.3. Objetivos

El objetivo principal de este TFG es **desarrollar un sistema ACG haciendo uso de métodos de IA neuro-simbólica, potencialmente aplicable a cualquier dominio expresado en PDDL, y realizar una primera validación de su viabilidad en el dominio *Blocksworld***. Para cumplir este objetivo general, se han definido los siguientes objetivos:

1. Realizar una revisión exhaustiva del estado del arte en ACL y ACG con especial atención a métodos aplicados al RL.
2. Analizar en profundidad el método de generación neuro-simbólica NeSIG [17], comprendiendo su formulación, capacidades y limitaciones, con el objetivo de explorar su potencial como generador adaptativo.

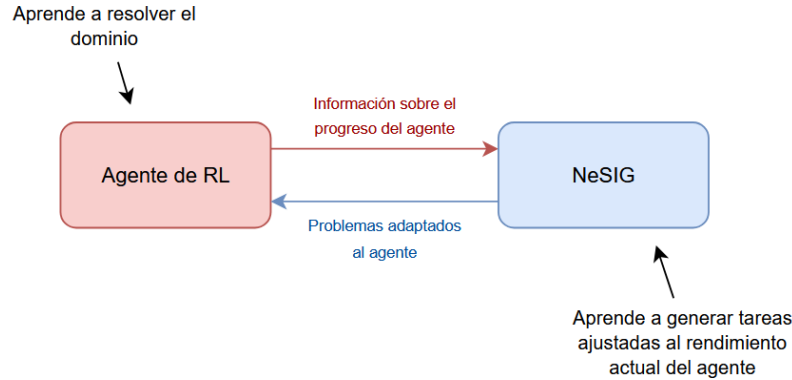


Figura 1.2: Esquema simple sobre el sistema de ACG que se propone en este TFG. Un agente de RL aprenderá a resolver problemas de un dominio concreto. Este dominio junto con sus problemas vendrán especificados en PDDL. El modelo neuro-simbólico, NeSIG, aprenderá a generar problemas en esta especificación adecuados a las capacidades del agente en cada momento. Cada iteración de aprendizaje, proporcionará problemas al agente, recibiendo retroalimentación de la calidad de dichos problemas que usará para ajustar los futuros problemas generados.

3. Diseñar e implementar un sistema de ACG basado en NeSIG que, combinado con un agente de RL, constituya un marco general aplicable a una amplia variedad de dominios expresados en PDDL.
4. Desarrollar un método *baseline* que permita comparar el sistema propuesto con un enfoque clásico, basado en generadores parametrizados.
5. Realizar una evaluación experimental profunda de los dos métodos (NeSIG y *baseline*), analizando comparativamente la eficacia y la eficiencia de cada uno.

1.4. Planificación económica y temporal

Un TFG es una asignatura evaluada en 12 créditos, donde a cada crédito le correspondería una carga en tiempo de 25 horas. Esto se traduce en un total de 300 horas de trabajo. Distribuido a lo largo de un curso entero, lo que es aproximadamente 40 semanas, se requeriría dedicar 7 horas y media a la semana. Esto equivale a 1 hora y media durante 5 días.

Para gestionar el desarrollo del proyecto se adoptó una metodología secuencial con una estructura bien definida como se describe en modelos clásicos de desarrollo de software [19] [20]. No obstante, dado que los requisitos

pueden evolucionar durante el desarrollo y en ML es habitual la necesidad de iterar sobre los experimentos, se considera adecuado complementar este enfoque con principios de metodologías ágiles, tal y como se recoge en el Manifiesto Ágil [21] y en el marco de trabajo Scrum [22]. Por ello, se han incorporado elementos como revisiones periódicas de los resultados obtenidos, priorización dinámica de tareas mediante una lista de trabajo y evaluación continua del progreso. De este modo, se busca combinar la solidez de los modelos tradicionales con la flexibilidad de los enfoques iterativos. Siguiendo esta metodología, el proyecto se ha dividido en las siguientes fases:

- **Investigación y revisión bibliográfica:** estudio del estado del arte en ACL y ACG y revisión de métodos neuro-simbólicos con especial atención a NeSIG.
- **Diseño del sistema propuesto:** definición del alcance y las capacidades del modelo a desarrollar, estableciendo las directrices y la línea de actuación que guiarán su implementación.
- **Implementación:** desarrollo del sistema ACG propuesto, incluyendo el agente de RL, el método *baseline* y el sistema completo basado en NeSIG.
- **Experimentación y análisis de resultados:** obtención de resultados de los métodos propuestos sobre el dominio de *Blocksworld* [23].
- **Redacción y conclusiones:** elaboración de la memoria y extracción de conclusiones.

Fase	Duración sem · h	Noviembre 2025	Diciembre 2025	Enero 2026	Febrero 2026	Marzo 2026	Abril 2026	Mayo 2026
Revisión bibliográfica	5 · 75	03 10 17 24	01 08 15 22	05 12 19 26	02 09 16 23	02 09 16 23	06 13 20 27	04 11 18 25
Diseño del sistema propuesto	1 · 15							
Implementación	5 · 75							
Análisis de datos	6 · 90							
Conclusión y redacción	5 · 75							

Tabla 1.1: Planificación inicial del proyecto.

La planificación temporal del proyecto se puede consultar en la Tabla 1.1. Sin embargo, durante el primer cuatrimestre no se avanzó al ritmo esperado. Esto llevó a replanificar el tiempo para poder entregar el trabajo en junio, lo que obligó a concentrar una parte importante del trabajo durante los meses de marzo y abril. La planificación resultante queda reflejada en la Tabla 1.2. Finalmente, como los resultados no terminaban de ser satisfactorios llegado junio, se decidió posponer la entrega para septiembre con vistas a obtener mejoras. La planificación final se puede observar en la Tabla 1.3. El cómputo aproximado de trabajo realizado ha resultado en un total de 660 horas, un poco más del doble de lo inicialmente esperado.

Fase	Duración sem · h	Noviembre 2025	Diciembre 2025	Enero 2026	Febrero 2026	Marzo 2026	Abril 2026	Mayo 2026	Junio 2026
Revisión bibliográfica	5 · 75								
Diseño del sistema propuesto	1 · 15								
Implementación	6 · 90								
Análisis de datos	10 · 150								
Conclusión y redacción	13 · 195								

Tabla 1.2: Planificación modificada del proyecto en febrero.

Fase	Duración sem · h	Noviembre 2025	Diciembre 2025	Enero 2026	Febrero 2026	Marzo 2026	Abril 2026	Mayo 2026	Junio 2026	Julio 2026	Agosto 2026
Revisión bibliográfica	5 · 75										
Diseño del sistema propuesto	1 · 15										
Implementación	6 · 90										
Análisis de datos	14 · 210										
Conclusión y redacción	18 · 270										

Tabla 1.3: Planificación final del proyecto.

Para establecer una estimación del presupuesto del proyecto se ha tenido en cuenta tanto los gastos de personal como los recursos materiales y herramientas empleados. El coste por hora se estima en 30 €/h. Este es el valor de referencia para un investigador o experto en IA en una empresa tecnológica en España [24] [25] [26].

Ítem	Coste
Gastos de personal ($660h \times 30 \text{ €/h}$)	19.800,00 €
Ordenador personal (amortización* 9 meses)	255,00 €
Overleaf (3 meses de suscripción)	33,00 €
Claude (3 meses de suscripción)	60,00 €
Total	20.148,00 €

Tabla 1.4: Desglose de costes asociados al proyecto. *Amortización calculada como $\frac{9}{60} \times 1700 = 255$, con un periodo de amortización de 60 meses y un coste del equipo de 1700 €.

Capítulo 2

Fundamentos Teóricos

Esta sección describe los fundamentos teóricos necesarios para comprender este TFG. En primer lugar, se introduce la planificación automática junto con PDDL, el lenguaje utilizado para representar formalmente las tareas. A continuación, se presentan los fundamentos del RL y las técnicas de RL profundo, en particular *proximal policy optimization* (PPO), que es el algoritmo empleado para entrenar al agente. Posteriormente, se describe el modelo de *neural logic machine* (NLM), que constituye la arquitectura neuronal utilizada por dicho agente. Por último, se explica en detalle CL, con el objetivo de entender los fundamentos de un sistema de ACG, como el propuesto en este trabajo.

2.1. Planificación automática

En este trabajo, se emplea la planificación automática [27] como base para la formulación de las tareas. Esta rama de la inteligencia artificial se centra en resolver problemas mediante la búsqueda automática de una secuencia de acciones que permita a un agente alcanzar un objetivo específico a partir de un estado inicial. Mediante la planificación automática se dota a máquinas y software de la capacidad para tomar decisiones y resolver tareas complejas sin intervención humana directa. Las aplicaciones van desde mover robots simples por una trayectoria ordenada hasta organizar procesos logísticos.

En el contexto de la planificación automática, el problema a resolver se formaliza mediante lo que se denomina una tarea de planificación. Se define mediante los siguientes componentes:

- **Espacio de estados:** conjunto de todas las situaciones posibles en las que puede encontrarse el entorno.

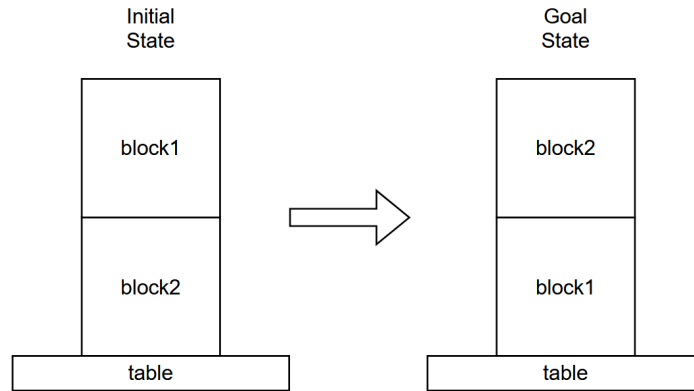


Figura 2.1: Se observa un problema de planificación automática en el dominio *Blocksworld*. A la izquierda se presenta el estado inicial y a la derecha el objetivo. La tarea consiste en encontrar una secuencia de acciones válida cuya aplicación transforme el estado inicial en el objetivo. En el proceso también se pueden optimizar objetivos secundarios como el número de pasos.

- **Estado inicial:** el estado concreto desde el que parte el agente antes de ejecutar ninguna acción.
- **Estado objetivo:** el estado, o conjunto de estados, que se desea alcanzar como solución al problema.
- **Espacio de acciones:** conjunto de acciones que el agente puede ejecutar, cada una definida por sus precondiciones y sus efectos.
- **Función de coste** (opcional): criterio que permite comparar planes alternativos, como el número de acciones ejecutadas o un coste acumulado asociado a cada una de ellas.

La solución de la tarea se denomina plan y consiste en una secuencia ordenada de acciones que, aplicadas desde el estado inicial, transforma el entorno hasta satisfacer el estado objetivo. La finalidad de la planificación automática es encontrar el plan que resuelva el problema, optimizando, si existe, la función de coste. Esta búsqueda es un problema computacionalmente costoso [28]. En el caso general, se considera perteneciente a la clase de complejidad PSPACE-completa, y bajo restricciones a la clase NP-completa, lo que implica que encontrar un plan óptimo puede requerir recursos exponenciales en función del tamaño del espacio de estados. Esto implica que si la estrategia de búsqueda no es eficiente o el espacio de estados es muy amplio puede dar lugar a tiempos de ejecución muy largos o inalcanzables.

El planificador es el algoritmo que procesa el problema para elaborar el plan. Existen diversas estrategias de búsqueda para encontrar una secuen-

cia válida de acciones. Actualmente, es común el uso de planificadores de búsqueda sistemática en el espacio de estados como *Fast Downward* [29]. El proceso consiste en la ejecución de acciones de forma iterativa comprobando los diferentes resultados intermedios obtenidos.

Funcionamiento de un planificador simple

A continuación, se presenta un ejemplo simplificado del proceso que siguen planificadores como *Fast Downward* para elaborar planes:

1. Analizar el estado inicial.
2. Evaluar las acciones posibles que se pueden aplicar.
3. Obtener nuevos estados a partir de esas acciones.
4. Evaluar si alguno de esos estados cumple los predicados del estado objetivo.
5. Repetir el proceso hasta encontrar una secuencia de acciones válida.

2.1.1. *Planning domain definition language* (PDDL)

La representación del entorno es un elemento clave en planificación automática. La mayoría de sistemas actuales describen cada estado mediante un conjunto de predicados basados en una versión restringida de la lógica de primer orden, donde todos los predicados no especificados explícitamente se consideran falsos bajo la hipótesis del mundo cerrado. Esta representación simbólica permite simplificar el entorno y hacer la búsqueda tratable, ya que en el mundo real el espacio de estados y acciones puede alcanzar un tamaño muy elevado [28] [30]. Para codificar estos problemas en este TFG se usa PDDL [31], un lenguaje ampliamente adoptado en la comunidad de planificación automática. En PDDL, un problema se divide en dos archivos complementarios:

- **Dominios:** definen los tipos de objetos permitidos, los predicados existentes, las constantes y las acciones disponibles.
- **Problemas:** describen el estado inicial y las metas que se desean alcanzar mediante objetos y predicados.

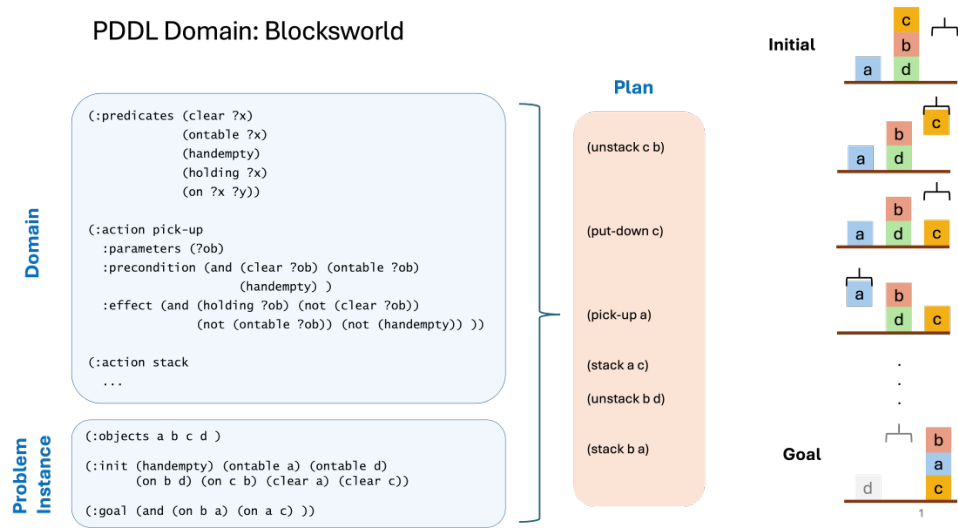


Figura 2.2: Ejemplo detallado de un dominio y un problema de *Blocksworld*. A la izquierda se encuentra la especificación de ambos archivos, y a la derecha el plan que se debe encontrar para resolver el problema. Aplicando de forma secuencial las acciones del plan, se logra transformar el estado inicial hasta cumplir los predicados del objetivo. Figura extraída de [32].

Archivo de dominio

Un dominio PDDL describe las acciones disponibles y la estructura del entorno. Son las reglas básicas que siguen todos los problemas que derivan de él. A continuación, se detallan todos sus elementos:

- **Cabecera:** se define mediante la palabra clave **domain**. Define el nombre del dominio.
- **Requisitos:** se definen mediante **:requirements**. Este campo especifica los requisitos que debe tener el planificador para resolver el dominio. Es similar a importar paquetes en un lenguaje de programación como *Python*. Se puede revisar la referencia [33] para una lista completa de requisitos.
- **Tipos:** se definen mediante **:types**. Permite indicar qué tipo de objetos pueden aparecer en el problema. Existe la herencia, por lo que se pueden establecer jerarquías.
- **Predicados:** se definen mediante **:predicates**. Son enunciados que establecen las posibles relaciones entre objetos. Estas relaciones pueden implicar a varios objetos, a un solo objeto o a ninguno. En este

último caso, funciona como una propiedad global que puede ser verdadera o falsa en cada momento.

- **Acciones:** se definen mediante `:actions`. De un modo similar a los predicados, sirven para especificar todas las acciones realizables en los problemas. Se definen especificando tres campos: 1) parámetros mediante `:parameters` que son los objetos implicados en la acción. 2) precondiciones mediante `:precondition` que son el conjunto de predicados que deben cumplirse para poder ejecutar la acción. 3) efectos mediante `:effect` que definen el impacto que tiene realizar la acción sobre el estado actual del problema.
- **Constantes:** se definen mediante `:constants`. Representa objetos esenciales que se manifiestan en cualquier problema del dominio.

Ejemplo de un dominio especificado en PDDL

```
(define (domain light)
  (:requirements :strips :typing)
  (:types room)
  (:predicates
    (light-on ?r - room)
    (in-room ?r - room)
  )

  (:action move
    :parameters (?from - room ?to - room)
    :precondition (in-room ?from)
    :effect (and (in-room ?to) (not (in-room ?from)))
  )

  (:action switch-on
    :parameters (?r - room)
    :precondition (and (in-room ?r) (not (light-on ?r)))
    :effect (light-on ?r)
  )
)
```

Archivo de problema

Un problema PDDL especifica el estado inicial y la meta a alcanzar usando los objetos, constantes y predicados del dominio. Para que un pro-

blema sea resoluble, se debe poder alcanzar el estado final a partir del inicial haciendo uso de las acciones del dominio. Los elementos del problema son:

- **Cabecera:** para especificar la cabecera se usa `problem`. En este apartado se debe especificar el nombre del problema y del dominio al que pertenece. El dominio se especifica mediante la etiqueta `:domain`.
- **Objetos:** se definen mediante `:objects`. Es la lista de objetos que existen en el problema. Los objetos pueden pertenecer a un tipo del dominio.
- **Estado inicial:** se define mediante `:init`. Especifica el estado inicial del problema mediante predicados. Bajo la hipótesis del mundo cerrado, únicamente es necesario declarar aquellos hechos que son verdaderos en el estado inicial, ya que el resto se asume falso por defecto.
- **Objetivo:** se define mediante `:goal`. Es la meta del problema. En esta sección se especifican los predicados que deben cumplirse para alcanzar el estado final.

Ejemplo de un problema especificado en PDDL

```
(define (problem light-problem)
  (:domain light)

  (:objects kitchen bedroom - room)

  (:init
    (in-room kitchen)
  )

  (:goal
    (and (light-on kitchen) (light-on bedroom))
  )
)
```

2.2. Aprendizaje por refuerzo

El ML es el campo de la IA centrado en el desarrollo de algoritmos capaces de aprender patrones a partir de los datos y mejorar su desempeño con la experiencia. Dentro de este campo se pueden distinguir tres paradigmas principales. El aprendizaje supervisado utiliza datos etiquetados para

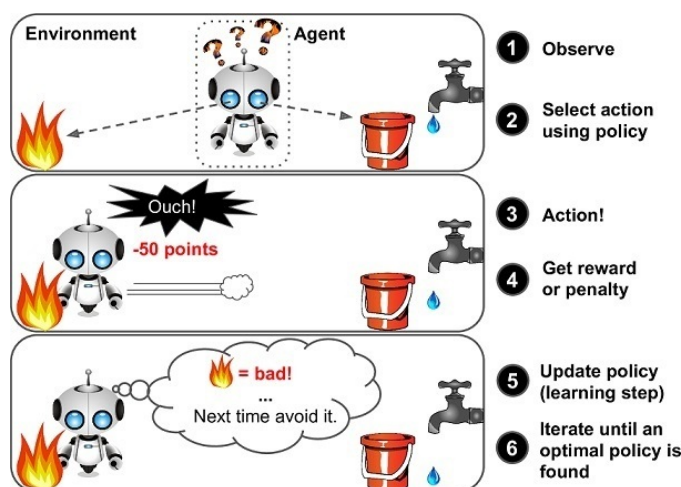


Figura 2.3: Esquema del proceso de RL. El agente observa el estado del entorno, selecciona una acción según su política actual, recibe una recompensa o penalización, y actualiza su política en consecuencia. Este proceso se repite iterativamente hasta converger en una política óptima. Figura extraída de [34].

entrenar modelos que puedan predecir salidas conocidas. El aprendizaje no supervisado trabaja con datos sin etiquetas, buscando identificar estructuras, agrupamientos o relaciones ocultas en la información. Por último, el RL entrena agentes a partir de la información obtenida al interactuar con un entorno. A diferencia del aprendizaje supervisado, este paradigma no requiere datos etiquetados, en su lugar, el agente aprende mediante señales de recompensa. En el RL existe un conjunto de acciones que se pueden ejecutar sobre un entorno para interactuar con él. El objetivo del agente es seleccionar las acciones para cada estado de forma que se maximice la recompensa acumulada. Esto implica considerar no solo el efecto inmediato, sino también su impacto en los estados futuros y las recompensas que se deriven de ellos.

2.2.1. Elementos básicos del aprendizaje por refuerzo

Los dos elementos principales del RL son el agente y el entorno. El agente es el sistema que aprende y toma decisiones con el objetivo de maximizar una recompensa determinada, mientras que el entorno es el sistema con el que interactúa el agente.

Se puede imaginar un laberinto de dos dimensiones. El objetivo del agente será encontrar la salida. Para lograrlo, puede desplazarse en cuatro direcciones: arriba, abajo, izquierda y derecha, que serán sus acciones disponibles. El estado del sistema corresponde a la posición del agente dentro del laberinto.

to en un momento determinado, y la recompensa se otorga cuando el agente se acerca a la salida o logra abandonar el laberinto, mientras que se pueden aplicar penalizaciones cuando el agente choque con un muro o malgaste movimientos. Mediante repetidos intentos, el agente explorará el entorno adaptando su comportamiento para aumentar la recompensa acumulada. De esta forma, aprenderá cuáles son las acciones más convenientes en cada punto del laberinto y mejorará progresivamente su capacidad de resolver el problema. Como se observa, el RL se basa en la interacción continua entre el agente y el entorno.

Los elementos fundamentales que constituyen el RL son los siguientes:

- **Estado y espacio de estados:** el estado, $S_t \in \mathcal{S}$, es la representación del mundo que recibe el agente del propio entorno en un instante t . El espacio de estados $\mathcal{S}$ es el conjunto de todos los estados que existen en el entorno.
- **Acción y espacio de acciones:** el espacio de acciones $\mathcal{A}$ es el conjunto de todas las acciones que tiene disponible el agente. Entonces, para cada estado, $\mathcal{A}(s)$ correspondería a las acciones de $\mathcal{A}$ que son aplicables en el estado s . Una acción $A_t \in \mathcal{A}(s)$ es una decisión que puede tomar el agente en un estado y en un instante dado. No todas las acciones se pueden tomar en todos los estados, aquellas cuya aplicación genera un estado inválido quedarían excluidas. En el laberinto, por ejemplo, una acción inválida sería caminar en una dirección bloqueada por un muro.
- **Recompensa:** el agente dispone de un objetivo que debe aprender a completar en el entorno. En RL, este objetivo no está especificado de forma explícita. Se codifica como una señal proporcionada al agente por el entorno llamada recompensa. El agente fijará como objetivo maximizar la cantidad de recompensas que recibe. De esta forma, se asocian recompensas a acciones o comportamientos que acerquen al agente al objetivo final. Es por ello que se busca, antes que la recompensa inmediata, maximizar la recompensa a largo plazo.
- **Política:** la política define el comportamiento del agente. Este componente es el que se busca entrenar hasta obtener una política con capacidad suficiente para resolver la tarea objetivo. De forma más rigurosa, una política es una función que mapea estados al espacio de probabilidades sobre acciones. Se define como una función de la siguiente forma:

$$\pi : \mathcal{S} \rightarrow P(\mathcal{A})$$

donde $\pi(a|s)$ es la probabilidad de hacer la acción a dado el estado s .

- **Transición:** una transición es una tupla que describe la dinámica de interacción entre el agente y el entorno. La tupla contiene el estado actual s_t , la acción ejecutada a_t , la recompensa obtenida r_{t+1} y el estado resultante s_{t+1} . Formalmente, se representa como $(s_t, a_t, r_{t+1}, s_{t+1})$.
- **Episodio y trayectoria:** un episodio es una secuencia finita de transiciones comprendida entre un estado inicial y un estado terminal. Cada episodio comienza de forma independiente del anterior. Por su parte, una trayectoria describe una secuencia de transiciones que no necesariamente cubre un episodio completo ni finaliza en un estado terminal.

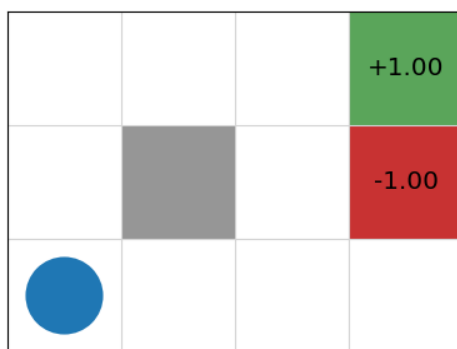


Figura 2.4: Ejemplo de entorno *Gridworld* utilizado en RL. El agente (círculo azul) interactúa con una cuadrícula discreta (entorno) donde cada celda representa un estado (espacio de estados). El objetivo del agente es llegar a la casilla objetivo (verde) y evitar las trampas (rojo). Las acciones disponibles para el agente son moverse en cuatro direcciones: arriba, izquierda, derecha y abajo (espacio de acciones). Para lograrlo, el agente debe desarrollar una estrategia para escoger las acciones (política). Esta estrategia se desarrolla ejecutando acciones sobre el entorno y recibiendo retroalimentación de cómo de positivas o negativas han sido las elecciones (recompensas). Un episodio sería la secuencia de estados de principio a fin del agente en el entorno, es decir, todos los movimientos hasta alcanzar un estado terminal. Una trayectoria sería una subsecuencia de dicho episodio. Figura extraída de [35].

2.2.2. Interacción agente-entorno y procesos de decisión de Markov

El ciclo de interacción entre el agente y el entorno se suele definir mediante un proceso de decisión de Markov o MDP. El MDP es una forma

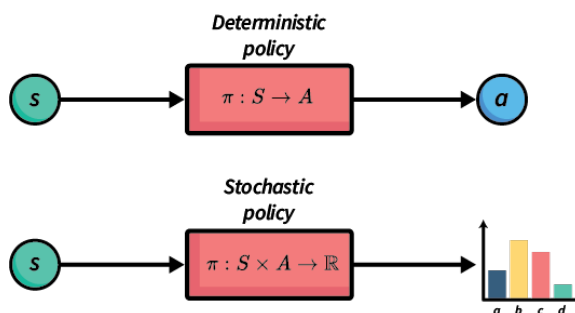


Figura 2.5: Comparación entre políticas deterministas y estocásticas. Una política determinista asigna una única acción a cada estado. Por el contrario, una política estocástica asigna a cada estado una distribución de probabilidad sobre el conjunto de acciones, lo que permite la exploración de diferentes decisiones ante un mismo estado. Figura extraída de [35].

matemática de formular el proceso de RL. Proporciona un marco formal para estructurar el problema de un agente aprendiendo a alcanzar un objetivo mediante la interacción con el entorno. En este esquema, el agente toma acciones y el entorno responde actualizando su estado. Asimismo, el entorno devuelve recompensas numéricas que reflejan si las acciones del agente son favorables o desfavorables. El aprendizaje del agente está guiado por estas recompensas, con el objetivo de maximizar su valor acumulado a largo plazo. A continuación, se repasará una definición matemática formal de MDP aplicada al ciclo de RL tomando como referencia principal el libro de Sutton y Barto [7].

La interacción entre el agente y el entorno se desarrolla a lo largo del tiempo discreto, representado mediante la variable:

$$t = 0, 1, 2, 3, \dots$$

En cada instante t , el agente recibe una representación del entorno llamada estado, $S_t \in \mathcal{S}$. Luego, el agente selecciona una acción, $A_t \in \mathcal{A}(S_t)$ que ejecutar. Como consecuencia de la acción, el agente recibe del entorno una recompensa $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ y el siguiente estado S_{t+1} . En la Figura 2.6 se visualiza una representación del ciclo de interacción entre el agente y el entorno.

Este ciclo puede estructurarse en MDPs episódicos o no episódicos. En los primeros, la interacción se organiza en episodios finitos que terminan al alcanzar un estado terminal, mientras que en los segundos el proceso continúa indefinidamente ($t = 0, 1, 2, \dots, \infty$), sin una segmentación natural en episodios.

Las variables R_{t+1} y S_{t+1} se determinan de manera probabilística en

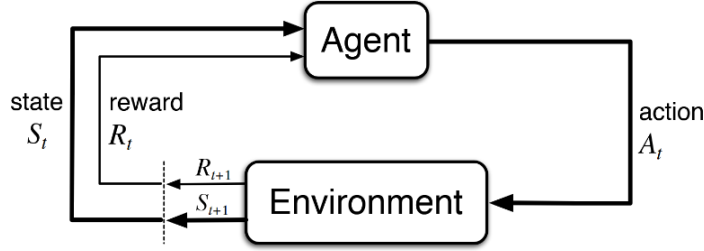


Figura 2.6: Ciclo de interacción básico entre el agente y el entorno. El entorno en cada instante proporciona al agente una representación del estado actual del problema indicado en la figura como S_t . El agente toma una decisión en forma de acción (A_t) que transmite al entorno. El entorno, en respuesta, le proporciona al agente una nueva representación del problema (S_{t+1}) junto con una recompensa (R_{t+1}). Este proceso constituye el mecanismo de interacción mediante el cual el agente aprende la política. Figura extraída de [7].

función del estado y la acción en el instante anterior. En particular, dado un estado $S_t = s$ y una acción $A_t = a$, existe una distribución de probabilidad sobre los posibles valores $s' \in \mathcal{S}$ y $r \in \mathcal{R}$:

$$p(s', r | s, a) := Pr\{S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a\} \quad (2.1)$$

donde $a \in \mathcal{A}(s)$.

Esta función p se define como la función de transición de probabilidad del MDP, y describe la dinámica del entorno. Formalmente, asigna a cada tupla (s, a, s', r) una probabilidad en $[0, 1]$, que representa la probabilidad de que, dado un estado s y una acción a , el sistema transite al estado s' y reciba una recompensa r :

$$p : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \times \mathcal{R} \rightarrow [0, 1] \quad (2.2)$$

En un proceso de decisión de Markov, las probabilidades dadas por p caracterizan por completo la dinámica del entorno. La probabilidad de cada S_{t+1} y R_{t+1} depende exclusivamente de S_t y A_t . Esto significa que el estado debe contener toda la información necesaria para predecir qué puede pasar a continuación; es decir, con el estado actual no es necesaria ninguna otra información del pasado. Cuando un estado cumple esta propiedad se denomina que cumple la propiedad de Markov.

En un problema se busca maximizar la recompensa acumulada conocida como *return* G_t . En el caso más simple se calcula como la suma acumulada de las recompensas de cada instante t :

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T \quad (2.3)$$

donde T es el instante final. Esto tiene sentido cuando hay un horizonte finito de episodios. Sin embargo, hay problemas en los que no hay un instante final claro, sino que T podría ser teóricamente infinito. En estos casos la recompensa podría llegar a ser infinita, por lo que se usa una definición del *return* más compleja. Aquí se introduce el concepto de factor de descuento o γ . El factor de descuento se usa para controlar la importancia de las recompensas futuras frente a las inmediatas. Con valores bajos se disminuye la importancia de recompensas futuras y se maximizan las recompensas tempranas. Con un valor de 0 se maximiza la recompensa inmediata. En cambio, valores altos de γ incrementan la importancia de las recompensas futuras, favoreciendo decisiones a largo plazo. Con $\gamma = 1$, las recompensas futuras no se descuentan y todas tienen el mismo peso. Pero, si el horizonte es infinito, un valor de 1 puede no converger, es por eso que en MDPs infinitos se exige $\gamma < 1$. La fórmula del *return* con descuento se denota de la siguiente forma:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \quad (2.4)$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.5)$$

donde $\gamma \in [0, 1]$.

Ligado al concepto de política también es importante mencionar el valor de un estado $V(s)$. Se denota $v_\pi(s)$ como el *return* esperado a partir del estado s si el agente sigue la política π . La definición formal es:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]. \quad (2.6)$$

donde $\mathbb{E}_\pi$ denota la esperanza matemática sobre todas las trayectorias posibles a partir del estado s , cuando las acciones se seleccionan según la política π y las transiciones siguen la dinámica del entorno.

Un desafío importante en la búsqueda de la mejor política es la capacidad de determinar cuándo una política es mejor que otra. Para MDPs finitos se declara que una política π es mejor que otra π' si la recompensa acumulada de π es mayor o igual que la de π' para todo estado $s \in \mathcal{S}$. De manera formal:

$$\pi \geq \pi' \iff v_\pi(s) \geq v_{\pi'}(s), \quad \forall s \in \mathcal{S} \quad (2.7)$$

En MDPs finitos con $\gamma < 1$, la política óptima π^* es aquella cuya función de valor satisface $v_{\pi^*}(s) \geq v_{\pi}(s)$ para todo estado $s \in \mathcal{S}$ y para cualquier otra política π . En RL, el objetivo es aprender una política cuyo comportamiento se aproxime lo máximo posible al de la política óptima. Dado que v^* y π^* son desconocidas, los algoritmos de aprendizaje estiman progresivamente funciones de valor o políticas que inducen decisiones cada vez más cercanas a las que tomaría π^* , reduciendo así la diferencia en términos de recompensa acumulada esperada.

Los algoritmos de RL se clasifican en función de si aprenden solo de su política (*on-policy*) o si aprenden también de políticas diferentes (*off-policy*). En el caso de *on-policy*, el algoritmo solo aprende de las experiencias generadas por la política que está actualmente optimizando. Este enfoque es más estable pero puede quedar atrapado en políticas subóptimas. Por otro lado, en *off-policy* se aprende de experiencias generadas por políticas distintas a la que se está optimizando. Este acercamiento es más eficiente y permite converger más fácilmente a una política óptima pero al precio de una mayor complejidad e inestabilidad del aprendizaje.

2.2.3. Aprendizaje por refuerzo profundo

Tradicionalmente, el RL se ha basado principalmente en técnicas tabulares y algoritmos que aprenden directamente de la interacción con el entorno. Estos métodos usaban estructuras simples para almacenar explícitamente el conocimiento aprendido como tablas de valores.

Sin embargo, estos métodos no escalan bien en espacios de estados de gran dimensión. Su aplicación en entornos complejos, como videojuegos o robótica, no resulta práctica. A medida que el espacio de estados crece, resulta inviable representar toda la información mediante tablas debido al elevado número de combinaciones posibles.

El RL profundo surgió para abordar estas limitaciones. En lugar de almacenar explícitamente valores para cada estado, las redes neuronales posibilitan aproximar la función de valor, la función acción-valor o la propia política del agente. También, permiten relacionar experiencias similares sin necesidad de almacenar en una tabla cada estado individualmente.

Mediante el DL, los nuevos modelos de ML son capaces de generalizar estados similares sin necesidad de almacenar experiencias de forma individual. Esto permite abordar problemas con espacios continuos de estados como entornos del mundo real.

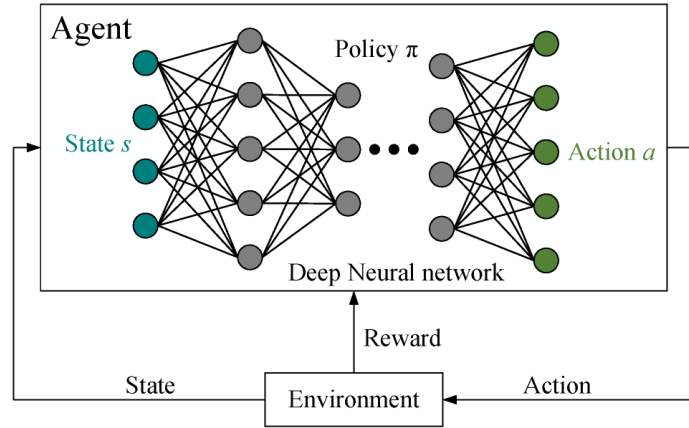


Figura 2.7: Esquema de RL profundo. El agente se compone de una red neuronal que se entrena con la recompensa del entorno. La red aprenderá sobre el impacto de sus acciones y desarrollará una política capaz de generalizar a problemas parecidos. Figura extraída de [36].

Proximal policy optimization

Para entrenar al agente, en este trabajo se emplea PPO, un algoritmo de RL profundo basado en el paradigma actor-crítico, que permite reducir la varianza en las actualizaciones del aprendizaje, mejorar la estabilidad del entrenamiento y acelerar la convergencia del algoritmo [37].

El actor y el crítico son dos redes neuronales independientes. El actor se encarga de modelar la política del agente, es decir, la distribución de probabilidad sobre acciones dado un estado, $\pi(a|s)$. El crítico, por su parte, estima la función de valor $V(s)$, que aproxima la recompensa esperada a largo plazo desde un estado dado.

A continuación, se detallan los principales mecanismos y características de este algoritmo:

- **Red neuronal para la función de la política $\pi(a|s)$:** red neuronal que representa la política del agente. Corresponde al actor del algoritmo. Dado un estado s produce la distribución de probabilidad sobre las acciones posibles. Esto permite al agente elegir acciones de forma estocástica.
- **Red neuronal para la función valor $V(s)$:** red neuronal que estima el valor esperado del estado $V(s)$. Corresponde al crítico del algoritmo.
- **Batch de datos generados por rollouts:** un *rollout* consiste en permitir al agente interactuar con el entorno siguiendo su política actual

mientras se recolectan datos. El conjunto de datos contiene estados, acciones, recompensas, probabilidades de las acciones con la política vieja y estimaciones $V(s)$. La información recolectada se usa para el cálculo de los retornos y ventajas, que son el núcleo de PPO y se detallan en los siguientes puntos.

- **Función de pérdida con *clipping***: los métodos de optimización anteriores a PPO presentaban problemas de inestabilidad debido a actualizaciones excesivamente grandes de la política cuando el gradiente era elevado. PPO soluciona este problema restringiendo el cambio permitido en la política en cada actualización mediante un mecanismo de *clipping*. Este mecanismo se formaliza mediante la siguiente función:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot \hat{A}_t \right) \right] \quad (2.8)$$

donde $r_t(\theta)$ es el ratio entre la probabilidad de seleccionar la acción bajo la nueva política y la antigua:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (2.9)$$

Este mecanismo evita que la política cambie de forma excesiva, ya que cuando el ratio se desvía fuera del intervalo $[1 - \epsilon, 1 + \epsilon]$, su contribución al objetivo se limita. De este modo, se reduce la variabilidad de las actualizaciones y se mejora la estabilidad del entrenamiento.

- **Generalized advantage estimation (GAE)**: es un método para estimar las ventajas A_t . Estas ventajas cuantifican en qué medida una acción en un estado resulta mejor o peor que lo esperado según la función de valor. De forma intuitiva, se puede interpretar como una diferencia entre la recompensa obtenida y la recompensa esperada bajo la política actual. La fórmula con GAE es la siguiente:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots \quad (2.10)$$

donde $\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ es el error de diferencia temporal, siendo R_{t+1} la recompensa inmediata obtenida tras aplicar una acción en el instante t .

El parámetro λ permite controlar el compromiso entre sesgo y varianza en la estimación de ventajas. Valores de λ cercanos a 1 reducen el sesgo pero aumentan la varianza, mientras que valores cercanos a 0 reducen la varianza a costa de un mayor sesgo. En la práctica, es habitual utilizar un valor de $\lambda = 0.95$.

- **Optimización por varias épocas sobre el mismo *batch*:** en muchos algoritmos *on-policy*, las transiciones generadas en un *rollout* se utilizan una única vez y posteriormente se descartan. Esto implica que solo se entrena una vez con cada conjunto de datos, lo que puede dar lugar a gradientes de alta varianza y, en consecuencia, a un aprendizaje inestable. PPO, en cambio, reutiliza los datos obtenidos del *rollout* durante varias actualizaciones o épocas. El proceso se detalla a continuación:

- Ejecutar la política actual en el entorno.
- Generar experiencias mediante *rollouts*.
- Dividir el *batch* en *mini-batches*.
- Realizar varias épocas de optimización sobre los mismos datos.

Con cada *mini-batch* se actualiza tanto la política como la función de valor, con un optimizador de gradiente descendente como AdamW [38]. Reutilizar datos permite aprovechar mejor cada episodio del agente sobre el entorno lo que resulta en una mayor eficiencia y un menor tiempo de entrenamiento.

El algoritmo de PPO se describe de forma general mediante los siguientes pasos:

1. **Inicialización:** se inicializan los modelos correspondientes al actor y al crítico. En particular, el actor parametriza la política estocástica $\pi(a \mid s)$, mientras que el crítico estima la función de valor $V(s)$.
2. **Recolección de experiencias (*rollout*):** el agente interactúa con el entorno siguiendo la política actual generando transiciones. En cada instante se selecciona una acción $a_t \sim \pi(\cdot \mid s_t)$, se ejecuta en el entorno y se almacenan las tuplas de experiencia, incluyendo $\log \pi(a_t \mid s_t)$ y $V(s_t)$.
3. **Cálculo de ventajas y ratios:** a partir de las experiencias recolectadas se calculan las ventajas $\hat{A}_t$ mediante GAE, así como los ratios r_t entre la política actual y la política anterior.
4. **Optimización de los modelos:** el conjunto de datos se divide en *mini-batches* y se realizan múltiples épocas de optimización sobre los mismos datos. En cada actualización se ajustan tanto el actor como el crítico mediante un optimizador basado en gradiente descendente.
5. **Iteración:** el proceso se repite utilizando la política actualizada, generando nuevos *rollouts* y continuando el entrenamiento del agente.

2.2.4. Desafíos del aprendizaje por refuerzo profundo

En esta subsección se comentan algunos desafíos existentes en el RL profundo. Concretamente aquellos que se consideren de interés con respecto al desarrollo de este proyecto. Por un lado, se explicarán desafíos comunes como el de la exploración frente a la explotación y las recompensas escasas, pero también se presentarán otros más específicos como el *catastrophic forgetting* y los *moving targets*.

Exploración frente a explotación

Un dilema fundamental en RL es el equilibrio entre la explotación y la exploración durante el entrenamiento del agente. La explotación con un agente consiste en elegir acciones que, según la política actual, maximicen la recompensa inmediata. La exploración con un agente consiste en probar nuevas acciones que no se han ejecutado suficientemente, esto no maximiza la recompensa a corto plazo pero permite encontrar comportamientos que sean más rentables a largo plazo.

Lograr un buen equilibrio es algo crítico para obtener buenos resultados en el entrenamiento. Un agente que explora demasiado puede que nunca converja a una política eficiente, mientras que uno que explota demasiado probablemente quedará atrapado en un óptimo local. Para lograr balancearlo, existen diversas estrategias como ϵ -greedy o softmax, que añaden aleatoriedad de forma controlada en la selección de acciones.

Recompensas escasas o retardadas

Existen entornos donde el agente no recibe retroalimentación inmediata tras cada acción. En este tipo de entornos, se dice que las recompensas son escasas (*sparse*) si la señal solo se recibe en momentos muy concretos (como ganar o perder una partida de un nivel) o retardadas si las acciones que determinan el éxito se toman mucho antes de recibir la señal.

A nivel práctico, esto causa que el gradiente de aprendizaje sea débil o nulo durante largos periodos, lo que dificulta mucho la convergencia. Además, el agente encontrará mucha dificultad a la hora de aprender qué acciones fueron responsables del resultado final.

Se han encontrado diversas técnicas para mitigar este problema, como *reward shaping* [39], que modifica la señal de recompensa para proporcionar retroalimentación intermedia. Asimismo, existe CL, mediante el diseño de currículos con dificultad progresiva, la señal de recompensa se vuelve más accesible en fases tempranas.

Catastrophic forgetting

El *catastrophic forgetting* [40] es un fenómeno que puede ocurrir en CL donde el modelo olvida conocimiento previamente aprendido al entrenar sobre nuevos datos. Como consecuencia, la red pierde la capacidad de resolver problemas básicos que previamente dominaba.

En CL, el *catastrophic forgetting* ocurre cuando el agente progresa en el aprendizaje de problemas cada vez más complejos. A medida que la dificultad aumenta, el agente puede llegar a olvidar comportamientos previamente adquiridos en etapas más tempranas, correspondientes a tareas más simples. Esto provoca que tareas que antes eran resueltas correctamente vuelvan a resultar difíciles o incluso inviables, ya que los conocimientos necesarios para resolverlas han sido parcialmente sobrescritos durante el entrenamiento. En consecuencia, se produce una degradación significativa del rendimiento en tareas anteriores, lo que también puede interferir negativamente en el aprendizaje de las tareas actuales.

Una solución es la incorporación de un mecanismo conocido como *experience replay* [41] [42]. El mecanismo consiste en mantener una memoria de problemas previamente resueltos. Esta memoria actúa como un almacén de experiencias pasadas que se revisitan aleatoriamente en fases posteriores del entrenamiento, evitando que se olviden.

Moving targets

Un escenario de *moving targets*, como se analiza en [42], se produce cuando en un proceso de optimización, el objetivo que el modelo intenta aproximar cambia continuamente a lo largo del entrenamiento. En estas condiciones, el sistema trata de ajustarse a una referencia que no es fija, sino dinámica, lo que genera inestabilidad y dificulta la convergencia. Este fenómeno es característico de entornos de aprendizaje con distribuciones de datos cambiantes, donde el propio proceso de aprendizaje modifica el contexto sobre el que se aprende.

En RL, este problema se acentúa especialmente en escenarios con múltiples agentes con roles diferenciados, donde uno de los agentes interactúa directamente con el entorno mientras que el otro aprende a partir del comportamiento del primero. En este tipo de configuración, el agente que actúa sobre el entorno modifica continuamente la distribución de experiencias disponibles, mientras que el segundo agente ajusta su aprendizaje en función de dichas interacciones. Como consecuencia, los objetivos de entrenamiento del segundo agente dependen de un modelo que está en constante actualización. Esta dependencia genera una fuente adicional de inestabilidad que puede dificultar significativamente la convergencia del aprendizaje conjunto.

Una forma de mitigar este problema es mediante las *target networks*, una técnica diseñada para estabilizar el cálculo de los objetivos de entrenamiento en entornos no estacionarios. La idea consiste en desacoplar la red que se optimiza mediante descenso de gradiente de la red utilizada para construir los objetivos. En concreto, se mantiene una copia congelada de los parámetros del modelo, que se actualiza de forma periódica a partir de la red principal. De este modo, durante cada intervalo de entrenamiento, los objetivos se calculan utilizando una versión fija del modelo. En escenarios con múltiples agentes interdependientes, este mecanismo resulta especialmente útil, ya que atenúa la inestabilidad derivada del cambio simultáneo de las políticas y permite que cada agente disponga de una referencia más estable sobre la que aprender.

2.3. *Neural logic machines*

En este trabajo se emplean NLMs [43] para el aprendizaje de la generación y resolución de problemas de planificación. Una NLM es una arquitectura de redes neuronales dentro del paradigma neuro-simbólico. Este paradigma tiene como objetivo combinar el aprendizaje de las redes neuronales con el razonamiento lógico estructurado. Para ello, las NLMs incorporan operaciones diseñadas para razonar sobre relaciones entre objetos y pueden ser entrenadas mediante descenso de gradiente. Esto les permite trabajar con representaciones estructuradas basadas en predicados y resolver tareas que requieren inferencia lógica.

Una NLM recibe como entrada un conjunto de predicados instanciados sobre un conjunto de objetos $\mathcal{O}$, y produce como salida otro conjunto de predicados sobre los mismos objetos. Es decir, transforma una representación lógica inicial en otra representación inferida. En la Figura 2.8 se puede observar un ejemplo del proceso.

La NLM trabaja con representaciones relacionales basadas en predicados sobre objetos. Esto le permite aprender patrones que generalizan a diferentes conjuntos de objetos que cumplen las mismas relaciones. Aunque se entrene con ejemplos concretos, las representaciones aprendidas pueden aplicarse a nuevos escenarios, lo que le confiere una buena capacidad de generalización.

De forma interna, una NLM representa los predicados mediante tensores agrupados por aridad. Es decir, todos los predicados de una misma aridad r se almacenan conjuntamente en una estructura tensorial, donde cada dimensión adicional indexa las instancias sobre el conjunto de objetos. Para cada predicado p_i de aridad r y para cada tupla de objetos $(o_1, \dots, o_r) \in \mathcal{O}^r$, el valor del tensor representa la probabilidad de que el átomo lógico $p_i(o_1, \dots, o_r)$ sea verdadero.

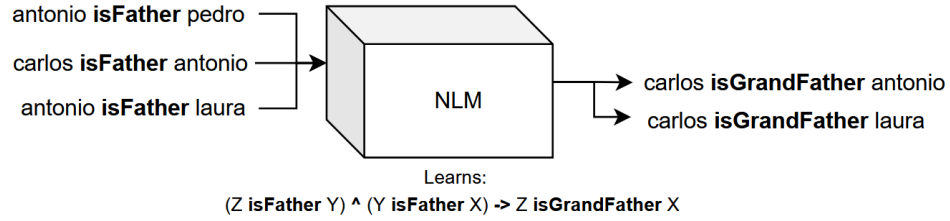


Figura 2.8: Funcionamiento simple de una NLM. En este ejemplo, la NLM aprende a inferir una regla para determinar si una persona es abuela de otra. Recibe como entrada predicados sobre quién es padre de quién; a partir de ellos, aplica reglas de primer orden intentando derivar nuevas reglas que le permitan inferir quién es abuelo de quién. Tras cada iteración, se calcula el error cometido y se realiza una propagación hacia atrás para actualizar los pesos. Gradualmente, el modelo aprende que la regla para identificar a un abuelo es que la persona sea el padre del padre de otra.

Ejemplo de representación tensorial

Conjunto de objetos: $\mathcal{U} = \{\text{Ana, Bob, Carlos}\}$

Predicados de aridad 2:

- **conoce**(x, y)
- **amigo**(x, y)

La NLM representa todos los predicados de aridad 2 mediante un único tensor de orden 3:

$$T[o_i, o_j, k]$$

donde:

- o_i, o_j indexan los objetos del conjunto $\mathcal{U}$
- k indexa el tipo de predicado ($k = 1$ para **conoce**, $k = 2$ para **amigo**)

Un posible estado del tensor es:

$$T[:, :, 1] = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad T[:, :, 2] = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

donde el primer slice corresponde al predicado **conoce** y el segundo al predicado **amigo**.

Por ejemplo, en el tensor se representa que Ana conoce a Bob y Bob es amigo de Carlos.

El funcionamiento de una NLM se basa en una pila de capas que realizan inferencia sobre representaciones lógicas de primer orden. Cada capa i recibe un conjunto de predicados P^{i-1} y produce un nuevo conjunto P^i , aplicando transformaciones que permiten combinar y propagar información entre predicados de distinta aridad. De forma conceptual, estas operaciones pueden interpretarse como una generalización de las operaciones lógicas básicas *AND* ($\wedge$), *OR* ($\vee$) y *NOT* ($\neg$), lo que permite a la red construir expresiones lógicas complejas a partir de predicados elementales.

Ejemplo de inferencia lógica

Mediante la combinación de predicados de tipo **esPadre**, es posible inferir relaciones más complejas como **esAbuelo**:

$$\text{esPadre}(\text{carlos}, \text{jaime}) \wedge \text{esPadre}(\text{jaime}, \text{juan}) \\ \Rightarrow \text{esAbuelo}(\text{carlos}, \text{juan})$$

En cada capa se aplican además operaciones que permiten conectar predicados de distinta aridad. En particular, las operaciones de expansión y reducción actúan como mecanismos de cuantificación, permitiendo aumentar o disminuir la aridad de los predicados. La expansión introduce nuevas variables en una relación, lo que facilita razonar sobre estructuras más complejas.

Ejemplo de expansión de relaciones

Se considera el predicado binario:

$$\text{on}(x, y)$$

que indica que un bloque x está sobre un bloque y . A partir de este predicado, es posible razonar sobre estructuras más complejas:

$$\text{on}(x, y) \wedge \text{on}(y, z) \Rightarrow \text{tower}(x, y, z)$$

El nuevo predicado ternario **tower**(x, y, z) representa una estructura de torre formada por tres bloques.

Por otro lado, la reducción realiza la operación inversa, disminuyendo la aridad de los predicados mediante agregación de variables.

Ejemplo de reducción de predicados

A partir del predicado relacional:

esPadre(carlos, jaime)

puede aplicarse una reducción que elimina uno de los argumentos, obteniendo una versión más abstracta:

esPadre(carlos)

Este nuevo predicado expresa que Carlos es padre de alguien, sin especificar de quién.

Estas operaciones permiten conectar predicados de distinta aridad dentro de la red, facilitando el razonamiento relacional. Sin embargo, para permitir la inferencia de nuevos predicados a partir de los predicados de entrada considerando diferentes órdenes de sus argumentos, se introduce una operación de permutación. Esta operación reordena las dimensiones asociadas a los objetos dentro de los tensores que representan los predicados. De este modo, se conectan las distintas ordenaciones de los objetos dentro de los predicados.

En la Figura 2.9 se ilustra la organización por capas del modelo. Para cada aridad $r \in [0, B]$, se obtiene un nuevo tensor combinando el tensor t_r con la información procedente de los tensores de aridad $r - 1$ mediante la operación de expansión y de aridad $r + 1$ mediante la operación de reducción, cuando estos existen. De este modo, cada capa puede combinar información procedente de relaciones de distinta complejidad, permitiendo establecer dependencias entre predicados con diferente número de variables. Una vez integrada esta información, se aplican permutaciones sobre las dimensiones asociadas a los objetos. Finalmente, la misma red neuronal se aplica a cada combinación de objetos para obtener las relaciones de salida inferidas por cada capa. La combinación de la operación de permutación con la red neuronal puede interpretarse como la implementación de una regla de tipo FOL $q(x_1, \dots, x_r) \leftarrow \text{expression}(x_1, \dots, x_r)$, donde *expression* denota una fórmula booleana (es decir, sin cuantificadores ($\forall$) o ($\exists$)) sobre los predicados de entrada, y donde cada nuevo predicado q pertenece al conjunto de predicados generados. Por lo tanto, este mecanismo implementa de forma diferenciable los operadores lógicos ($\neg$), ($\wedge$) y ($\vee$).

A medida que incrementa el número de capas, mayor es el nivel de abstracción que se puede obtener. Por ejemplo, la salida de la primera capa puede representar **Clear(x)**, mientras que capas más profundas pueden producir predicados más complejos como **Movable(x)**. De esta forma, la propagación hacia adelante en las NLMs puede ser interpretada como una

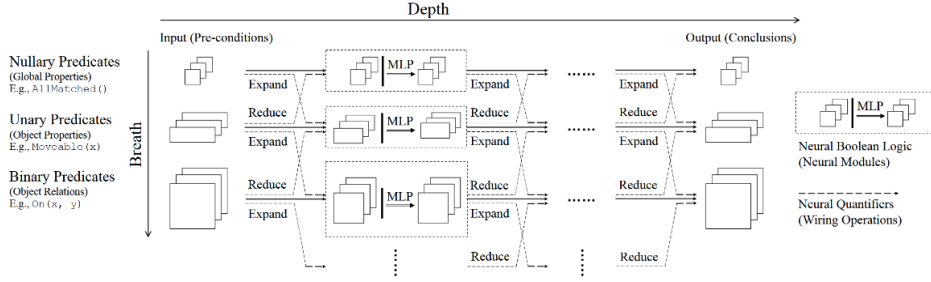


Figura 2.9: Representación interna de una NLM. Durante la propagación hacia delante, la red recibe como entrada un conjunto de predicados y relaciones definidos sobre un conjunto de objetos, sobre los cuales aplica transformaciones neuronales que aproximan operaciones lógicas en cada capa. Como resultado, se generan nuevas relaciones inferidas sobre los mismos objetos. Figura extraída de [43].

secuencia de aplicación de reglas.

La capacidad expresiva de una NLM depende de múltiples factores:

1. La profundidad D (número de capas) de la NLM restringe el número de pasos deductivos.
2. El ancho B (número máximo de variables de todos los predicados considerados) de la NLM limita la aridad de las relaciones entre objetos. A nivel práctico, la mayoría de predicados intermedios son binarios o ternarios. Este parámetro se establece según la tarea (normalmente con valor 2 o 3).
3. El número de predicados de salida usados en cada capa. El artículo original recomienda usar valores pequeños (8 o 16).
4. El poder expresivo de la MLP (número de capas ocultas y neuronas) restringe la complejidad de la lógica *booleana* representada. El artículo original recomienda redes poco profundas (0 o 1 capa oculta) con un número pequeño de neuronas (8 o 16). Esto puede interpretarse como una regularización de baja dimensionalidad que favorece el aprendizaje de reglas lógicas simples.

La complejidad computacional de la propagación hacia delante o hacia atrás en las NLMs es $O(m^B DC^2)$ donde m es el número de objetos y C el número máximo de predicados de salida de entre todas las capas. La red tiene $O(DC^2)$ parámetros. La complejidad polinomial favorece el uso de NLMs en dominios relacionales donde el número de bloques puede variar entre problemas y se requiere de una buena capacidad de generalización.

2.4. NeSIG: *Neuro-Symbolic Instance Generator*

NeSIG [17] es un método de generación automática de problemas independiente del dominio en el ámbito de la planificación automática. Genera problemas que son diversos, válidos y difíciles de resolver. NeSIG no necesita observar ni un solo ejemplo de problema por lo que es considerado un modelo generativo *data-free*. Recibe como entrada el dominio del problema en PDDL, reglas de consistencia e información extra como el tamaño máximo del problema o el tipo de objetivos y predicados. Luego, entrena dos políticas generativas diferentes con RL, una para generar los estados iniciales y otra para generar los objetivos. Finalmente, devuelve como salida un conjunto de problemas.

Las dos políticas de NeSIG se implementan mediante NLMs y se entrenan utilizando el algoritmo PPO. Dado que PPO sigue un esquema actor-crítico, se incorpora una NLM adicional como red crítica para cada política encargada de evaluar los estados del MDP. En consecuencia, el sistema NeSIG está compuesto en total por cuatro NLMs.

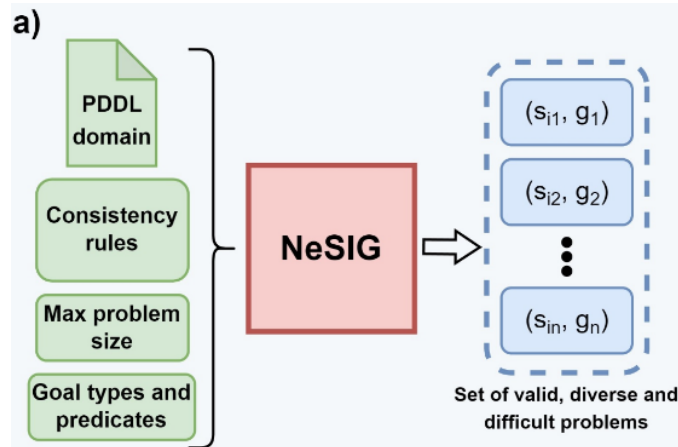


Figura 2.10: Entradas y salidas de NeSIG. Como entrada recibe el dominio PDDL, el tamaño máximo de los problemas, los tipos de objetivos, los tipos de predicados y las reglas de consistencia. Estas últimas son especificaciones de cómo se debe elaborar el estado inicial de los problemas para ser válidos. Como salida, NeSIG devuelve un conjunto de problemas válidos, diversos y difíciles. Figura extraída de [17].

Los problemas generados por el sistema siguen una serie de criterios diseñados para garantizar su calidad. En particular, se busca que cada problema cumpla un conjunto de propiedades que aseguren su coherencia interna, su variedad y su dificultad. En este sentido, los problemas generados persiguen satisfacer las siguientes tres propiedades:

- **Validez:** un problema es válido si es consistente y es resoluble. Un problema es consistente si su estado inicial tiene una estructura válida. Se garantiza mediante las reglas de consistencia, que guían el proceso de generación del estado inicial. Se distinguen dos tipos de reglas de consistencia:
 - **Reglas de consistencia continuas:** estas reglas se deben satisfacer en cada paso a lo largo de la generación del estado inicial. Esto se puede garantizar haciendo que NeSIG nunca añada ni objetos ni átomos que produzcan un estado de inconsistencia.
 - **Reglas de consistencia puntuales:** no es necesario que se satisfagan en todo momento, pero deben haber sido satisfechas al completar el estado inicial.

Por otro lado, un problema es resoluble si existe al menos un camino que permita alcanzar el estado final desde el inicial. Por definición, NeSIG genera problemas resolubles, ya que partiendo de un estado inicial válido aplica acciones permitidas hasta llegar a un estado objetivo.

- **Diversidad:** mide qué tan diferentes son los problemas generados entre ellos. Para calcular esta medida, se extrae un conjunto de características de los problemas, se construye un vector para cada uno y se mide la distancia entre ellos. La diversidad de un problema será igual a la distancia media hacia el resto de problemas del conjunto.
- **Dificultad:** esta es la medida de calidad de los problemas. Se define como el número de nodos que deben expandir un conjunto de planificadores para alcanzar la solución. Esta métrica empuja a NeSIG a generar problemas de alta complejidad. No obstante, este criterio puede sustituirse por otros objetivos de diseño del problema.

El funcionamiento de NeSIG se puede dividir en dos etapas. En la primera, se genera el estado inicial partiendo de una escena vacía y añadiendo objetos progresivamente. En cada paso se deben verificar las reglas de consistencia continuas. Cuando se detiene el proceso de añadir objetos, se verifica que se cumplen las reglas de consistencia puntuales, si es afirmativo, entonces se ha generado un estado inicial válido; si es negativo se descarta el problema. A continuación, para cada escena inicial válida del paso anterior se genera un estado objetivo. Se aplicarán acciones sucesivamente sobre los estados iniciales. Estas acciones modifican el estado actual del problema en cada paso dando lugar a nuevos estados. Cuando se detiene el proceso, se construye el objetivo del problema a partir del estado resultante. Para ello, se selecciona un subconjunto de los átomos finales de acuerdo con las precondiciones especificadas en las entradas. Finalmente, se devuelve el estado inicial junto con el estado objetivo.

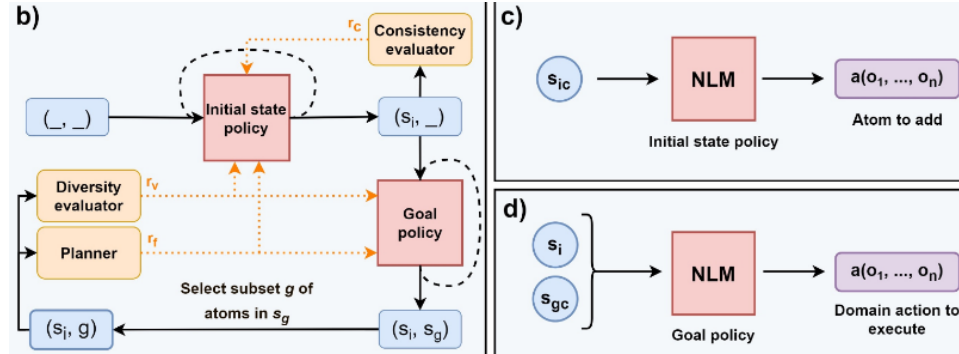


Figura 2.11: **b) Entrenamiento de la política con RL.** Las líneas discontinuas representan la aplicación de varias acciones MDP, correspondientes a añadir un átomo al estado inicial en el caso de la política del estado inicial o ejecutar acciones del dominio en la política del objetivo. Las líneas de puntos indican una señal de recompensas, ya sea de consistencia (r_c), diversidad (r_v) o dificultad (r_f). **c) Política del estado inicial.** Recibe un estado MDP correspondiente a un problema con el estado inicial parcialmente generado y selecciona un átomo para añadirle. **d) Política del objetivo.** Recibe un estado MDP que representa un estado inicial completo pero con un estado objetivo parcialmente generado y selecciona la siguiente acción del dominio a ejecutar. Figura extraída de [17].

Internamente, NeSIG genera problemas de la forma (s_i, g) , donde s_i es el estado inicial del problema y g es el objetivo. La primera política de NeSIG (*initial state policy*) se encarga de generar el estado inicial. La segunda política (*goal policy*) se encarga de la generación del objetivo. El proceso está representado de forma visual en la Figura 2.11. Se puede formular como un MDP de horizonte finito sin descuento (S, A, T, r) :

- S : el espacio de estados del MDP. En nuestro caso, cada estado corresponde a un problema de planificación generado o en proceso de generación, representado como $s = (s_{ic}, s_{gc})$. El subíndice c indica que el estado inicial y el estado objetivo pueden no estar completamente generados aún.
- A : el espacio de acciones. En la política de generación del estado inicial, las acciones consisten en añadir un átomo al estado inicial parcial s_{ic} . En la política de generación del objetivo, las acciones consisten en aplicar una acción del dominio cuyas precondiciones se satisfacen en el estado actual.
- $T : S \times A \rightarrow S$: la función de transición, que define la evolución determinista del estado tras la aplicación de una acción.

- $r : S \times A \rightarrow \mathbb{R}$: la función de recompensa original de NeSIG, que evalúa la consistencia (r_c), diversidad (r_v) y dificultad (r_f) del problema en construcción, definida como:

$$r = r_c + \min\left(\frac{r_v}{\epsilon}, 1\right) \cdot r_f \quad (2.11)$$

2.5. *Curriculum learning*: conceptos básicos

CL es una estrategia de entrenamiento en ML que se inspira en el aprendizaje humano. Los humanos empezamos aprendiendo a realizar tareas más sencillas y progresivamente aumentamos la dificultad hasta poder resolver tareas complejas. La idea es que el agente aprenderá de manera más efectiva si se entrena con una serie de tareas de dificultad progresiva y estructurada, mejorando la estabilidad del entrenamiento además de su capacidad de generalización.

2.5.1. *Automated curriculum learning*

ACL es una evolución de CL en la que el sistema selecciona o genera automáticamente las tareas de diferente dificultad sin intervención humana. Son una serie de mecanismos que automatizan el proceso manual de selección del currículo de tareas. La idea central es mantener al agente en una zona de aprendizaje óptima. Si la tarea es demasiado fácil, el agente no aprende comportamientos útiles y si es muy difícil no progresa. Por tanto, se deben seleccionar tareas desafiantes que estén ligeramente por encima del nivel actual del agente.

Estos métodos dotan de escalabilidad a CL, ya que eliminan el diseño manual de los currículos. Además, abren la puerta a sistemas inteligentes capaces de seleccionar currículos de forma dinámica adaptados a los modelos en cada momento. Esto es especialmente poderoso en RL donde los agentes aprenden de su interacción con el entorno. Esta idea ha sido formalizada en la literatura reciente, en particular *Automatic Curriculum Learning For Deep RL: A Short Survey* [16] realiza un repaso del paradigma ACL aplicado al RL profundo. En dicho trabajo se define ACL como:

”Automatic Curriculum Learning for deep RL is a family of mechanisms that automatically adapt the distribution of training data by learning to adjust the selection of learning situations to the capabilities of deep RL agents.”

En este contexto, ACL selecciona tareas de un conjunto con el objetivo de plantear nuevas dificultades al agente. La idea central es que tanto si

el agente es multitarea como si solo debe resolver una tarea, se encuentren los entornos que mejor se ajusten a su entrenamiento. Especialmente, en los casos donde el agente dispone de una única tarea muy difícil, se puede dividir la tarea final en subtareas o variaciones de esta que vayan acercando progresivamente al agente al objetivo final. Este mecanismo es especialmente útil en entornos con recompensas escasas (*sparse rewards*).

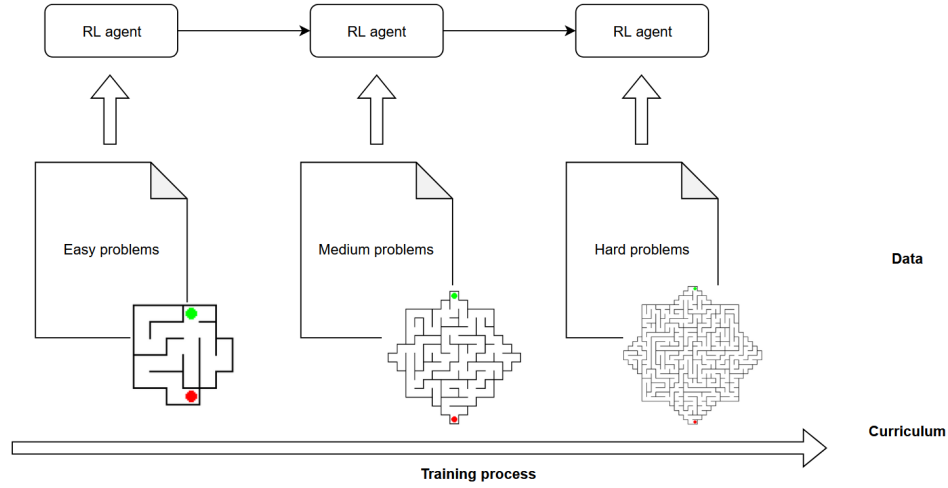


Figura 2.12: Ilustración del proceso de CL. El entrenamiento se organiza de forma progresiva, comenzando con tareas sencillas y avanzando hacia otras de mayor complejidad. Los ejemplos consisten en laberintos cuya dificultad aumenta gradualmente. Este enfoque permite que el agente adquiera primero comportamientos básicos que sirven de base para afrontar con éxito tareas más complejas en etapas posteriores.

En este trabajo, se formaliza ACL como una función de selección de tareas $\mathcal{D} : \mathcal{H} \rightarrow \mathcal{T}$, donde $\mathcal{H}$ puede contener cualquier información sobre iteraciones pasadas y $\mathcal{T}$ es el espacio de tareas. El objetivo sería maximizar una métrica de rendimiento P calculada sobre una distribución de tareas objetivo $\mathcal{T}_{target}$ de forma continua. Formalmente, se describe como:

$$\max_{\mathcal{D}} \frac{1}{N} \sum_{t=1}^N \mathbb{E}_{T \sim \mathcal{T}_{target}} [P_T^t] \quad (2.12)$$

donde P_T^t cuantifica el rendimiento del agente en la tarea T tras el paso de entrenamiento t , y N denota el número total de pasos de entrenamiento.

Esta formulación refleja el rendimiento medio del agente a lo largo del proceso de aprendizaje. El operador de esperanza $\mathbb{E}_{T \sim \mathcal{T}_{target}}[\cdot]$ promedia el rendimiento del agente sobre la distribución de tareas objetivo. La suma

temporal $\frac{1}{N} \sum_{t=1}^N$ introduce una evaluación a lo largo de todo el proceso de entrenamiento, capturando no solo el rendimiento final del agente, sino también su evolución durante el aprendizaje. De este modo, la formulación penaliza currículos que conduzcan a mejoras tardías del rendimiento, favoreciendo aquellos que permitan una progresión más rápida y estable.

2.5.2. *Automated curriculum generation*

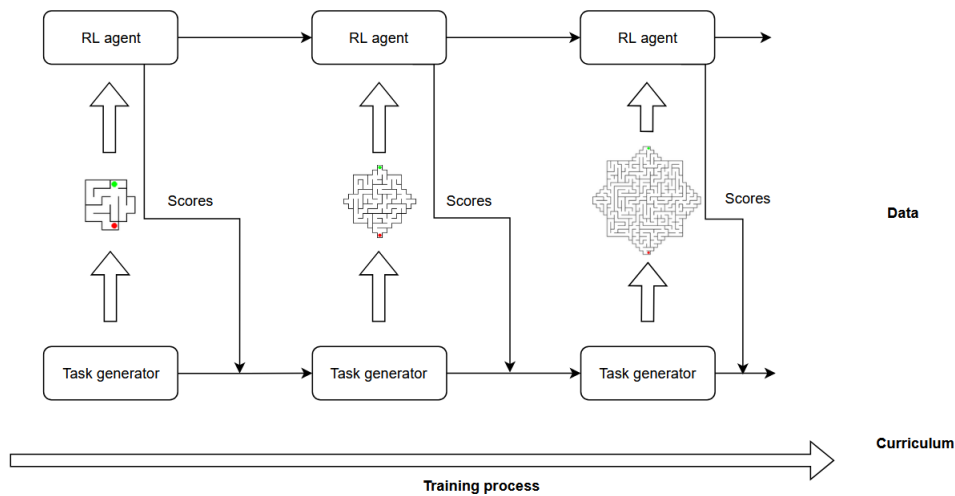


Figura 2.13: En esta imagen se aprecia el proceso de ACG en RL. Los problemas se generan en cada iteración por un generador. El agente se entrena con esos problemas y el generador recibe información de su desempeño. En la siguiente generación de problemas, se ajusta la dificultad de acuerdo con dicho desempeño. Este proceso forma un currículo dinámico que se ajusta al agente.

ACG es un área dentro del ACL que engloba aquellos métodos que, además de seleccionar el currículo de forma automática, también son capaces de generar las propias tareas de entrenamiento. Dentro de un sistema de ACG, el agente aprende a partir de las tareas generadas y, una vez completadas, se evalúa su rendimiento para generar nuevas tareas ajustadas a sus capacidades actualizadas, construyendo así un currículo de forma progresiva. De este modo, el entorno de aprendizaje deja de ser estático y se busca mantener al agente en una zona de dificultad óptima, donde las tareas no sean ni demasiado fáciles ni demasiado difíciles. Gracias a ello, ACG permite la construcción de currículos altamente adaptativos, siempre que el sistema de generación de tareas sea suficientemente flexible y expresivo. Por este motivo, la capacidad del generador constituye un elemento clave en este enfoque, siendo especialmente relevante su expresividad y diversidad.

Se han explorado diversas técnicas para este propósito, como la generación parametrizada [44], la generación adversaria [45] o los métodos basados en redes neuronales [46], entre otros (véase la sección 3 para más detalles sobre estos métodos). En este contexto, este TFG aborda el diseño e implementación de un sistema de ACG basado en la interacción de dos agentes, con el objetivo de estudiar y analizar este tipo de mecanismos de generación adaptativa de tareas en entornos de RL.

Capítulo 3

Estado del Arte

CL ha experimentado un aumento de interés en los últimos años. En la Figura 3.1 se puede observar el crecimiento en el campo desde 2009, cuando se acuñó el término CL. Dicho crecimiento ha tenido una tendencia al alza recientemente, coincidiendo con la explosión en popularidad del DL. Esta relación no es casual, el aumento de inversión e investigación en DL ha incentivado el interés por entrenar modelos y agentes cada vez más complejos. Tal demanda ha generado una necesidad de encontrar métodos como CL que permitan desarrollos más eficientes y sencillos.

En comparación con CL, en la Figura 3.2 se aprecia que los estudios sobre ACG son escasos, el primero que se muestra es de tan solo 2019. Desde entonces, ha habido un incremento no muy elevado, pero con una tendencia creciente. En 2024, se marca el máximo con 16 publicaciones. Estos números describen una falta de desarrollo en el área, lo que contrasta con el creciente interés por mecanismos de aprendizaje eficientes en ML. Esta contradicción se puede interpretar como una gran oportunidad de investigación. ACG parece un campo con mucho recorrido donde aún hay mucho por descubrir, lo que refuerza la relevancia de proyectos como este.

Las *queries* de Scopus usadas para obtener tanto la Figura 3.1 ¹ como la Figura 3.2 ² se pueden consultar al pie de página.

¹TITLE-ABS-KEY((curriculum learning OR automatic curriculum learning OR adaptive curriculum learning) AND (machine learning OR deep learning OR reinforcement learning OR supervised learning))

²TITLE-ABS-KEY((curriculum learning OR automatic curriculum learning OR adaptive curriculum learning) AND (machine learning OR deep learning OR reinforcement learning OR supervised learning) AND (automatic curriculum generation OR generate curriculum OR generate automatic curriculum OR procedural generation))

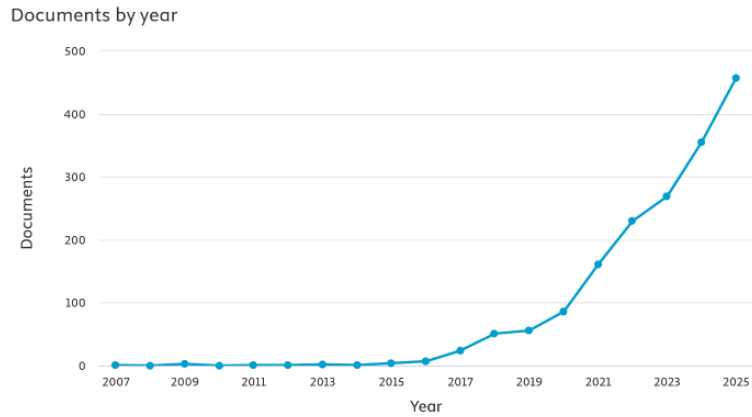


Figura 3.1: Crecimiento de publicaciones en el campo de CL según Scopus.

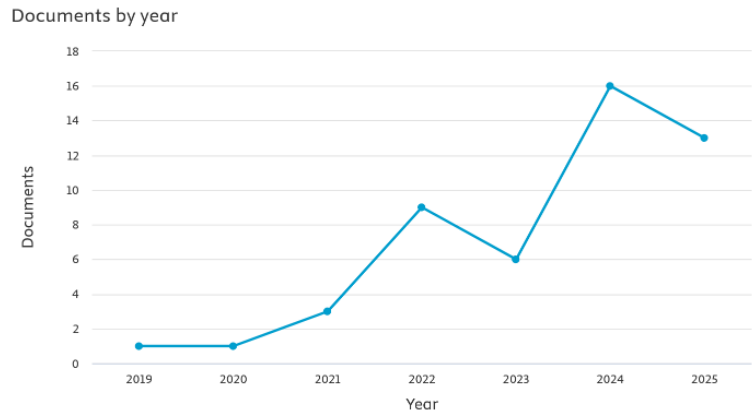


Figura 3.2: Crecimiento de publicaciones en el campo de ACG según Scopus, desde 2019.

3.1. Orígenes del *curriculum learning*

El concepto formal de CL surgió en 2009 en el trabajo del mismo nombre *Curriculum Learning* de Bengio, et al. (2009) [12]. En él, se formulaba la pregunta: "¿puede un algoritmo de machine learning beneficiarse de esta metodología?". En respuesta, los autores investigaron dentro del marco del aprendizaje supervisado y propusieron desarrollar una función o heurística que calculara la dificultad de cada muestra de entrenamiento. Esta función se usó para ordenar por dificultad las muestras de entrenamiento que recibe un modelo. Comienza con un conjunto reducido de ejemplos catalogados como sencillos y se agrega dificultad a medida que el entrenamiento avanza, hasta usar el conjunto completo de muestras. Evaluaron la propuesta en tareas supervisadas de problemas sintéticos y clasificación de imágenes. Al

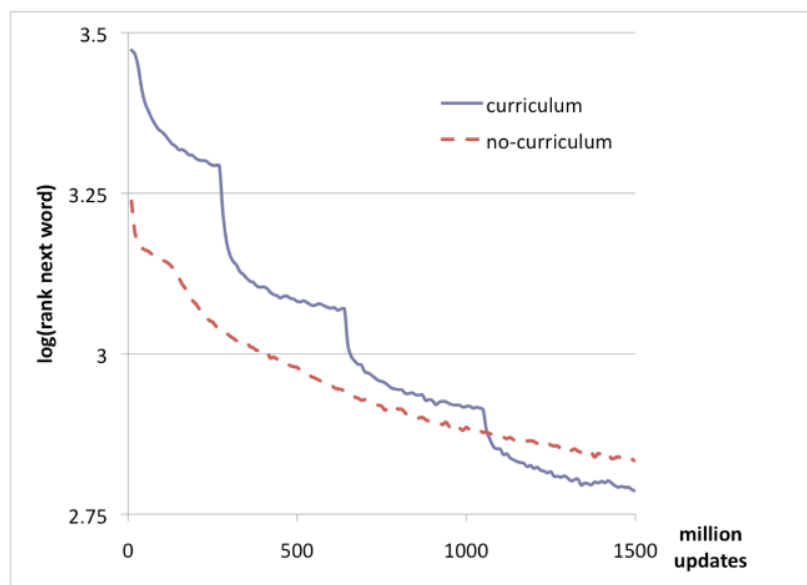


Figura 3.3: Modelo de lenguaje entrenado con y sin currículum en el trabajo *Curriculum Learning* de Bengio, et al. (2009) [12]. El modelo se entrenó para predecir la palabra más adecuada para continuar una frase en inglés. El eje horizontal representa el número de actualizaciones de los parámetros (en millones). El eje vertical mide la dificultad del modelo para predecir la siguiente palabra: cuanto menor es su valor, más acertadas son las predicciones. El modelo con currículum alcanza un error final inferior, y su curva escalonada refleja las sucesivas adaptaciones del modelo a etapas de dificultad creciente. Figura extraída de [12].

comparar el entrenamiento propuesto frente a un entrenamiento estándar (orden aleatorio), observaron que CL podía llegar a mejorar la capacidad de generalización del modelo en varios escenarios y acelerar la convergencia. A pesar de las ventajas, la elaboración manual de los currículos era un gran inconveniente para su uso generalizado en problemas más complejos o variados.

3.2. *Automated curriculum learning* en aprendizaje por refuerzo

En esta sección se presentan los métodos de ACL aplicados a RL más relevantes para el presente trabajo. Algunos autores han elaborado recopilaciones sistemáticas del área. Portelas et al. (2020) [47] ofrecen en *Automatic Curriculum Learning For Deep RL: A Short Survey* una revisión amplia de

las propuestas aplicadas a RL. Por otra parte, Weng (2020) [48] elabora una clasificación basándose en los diferentes paradigmas en los que puede enmarcarse el ACL dentro del RL (Figura 3.4). A continuación, se describen algunos de los métodos más destacados. Los métodos que también incorporan generación automática de tareas (ACG) se tratan por separado en la sección 3.3.

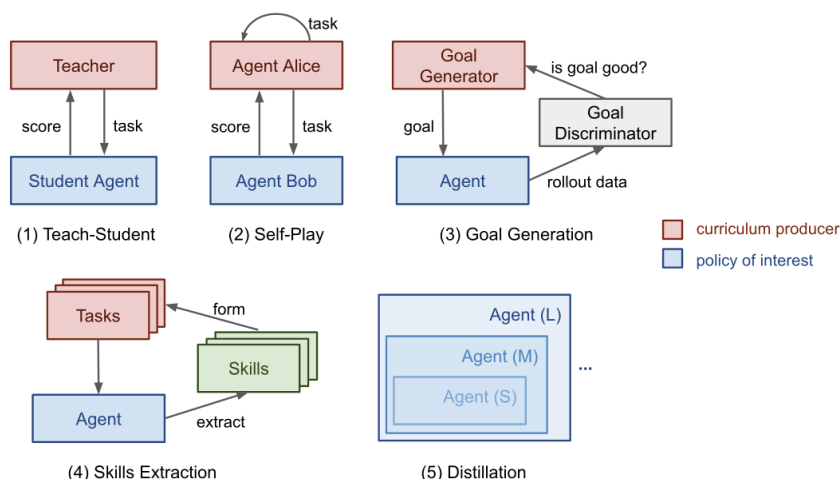


Figura 3.4: Clasificación de tipos de CL dentro de RL. Aunque, principalmente son usados en RL, algunos también han sido aplicados a aprendizaje supervisado. Figura extraída de [48].

Self-paced learning (2010)[49] representó uno de los primeros intentos serios de automatizar el proceso de selección de muestras. Este enfoque es una estrategia de optimización iterativa que ajusta dinámicamente la distribución de los datos de entrenamiento. Se comporta como un filtro adaptativo que reduce el impacto de muestras ruidosas o difíciles en las primeras fases del aprendizaje. En cada iteración, calcula la pérdida de cada ejemplo con el modelo. De la distribución, se entrena solo con los ejemplos de menor pérdida. Para ello, se establece un umbral y se seleccionan aquellos que queden por debajo. El umbral se aumenta progresivamente para incluir ejemplos cada vez más difíciles.

Este método tiene una gran ventaja con respecto al trabajo original de CL [12], es adaptativo, no requiere conocimiento experto ni trabajo manual para la elaboración del currículo. Además, es muy robusto al ruido. Los ejemplos con *outliers* o etiquetas erróneas quedan al final de la distribución de pérdida y solo se presentan cuando el umbral es muy alto, incluso pueden excluirse fácilmente si es conveniente. El problema principal de este método es que tiene una complejidad computacional mucho mayor que las alternativas manuales. El cálculo de las pérdidas individuales supone un incremento

considerable. Aun así, este marco sentó las bases de ACL, permitiendo que surgieran variantes que exploraran diferentes criterios para definir la "dificultad" de las muestras, así como estrategias adaptativas más sofisticadas.

En 2017, Graves, et al. (2017) [13] planteaban un modelo ACL como un problema del bandido multi-brazo [50]. Dicho problema se compone de un agente cuyo objetivo es equilibrar la exploración frente a la explotación en un problema de optimización. En CL, este marco se modela sobre un conjunto de tareas, un agente debe seleccionar diferentes tareas (exploración) priorizando aquellas (explotación) que maximicen el aprendizaje de otro agente.

Este proceso de proponer currículos mediante otro agente que actúa como selector de tareas fue formalizado como *Teacher-Student Curriculum Learning* por Matiisen, et al. (2017) [51]. En este trabajo, un estudiante es un agente de RL que resuelve tareas y el profesor es otro agente que se encarga de seleccionarlas. El estudiante debe dominar una tarea que puede ser muy difícil en un inicio. Para asistir al estudiante, se le proporciona un agente profesor programado para seleccionar subtareas que optimicen su proceso de aprendizaje. En la Figura 3.5 se detalla la estructura de un sistema profesor-estudiante.

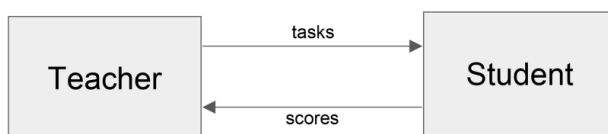


Figura 3.5: Esquema de interacción entre el profesor y el estudiante. Este problema se formula como un problema de optimización bi-nivel [52]: en el nivel inferior el estudiante actualiza sus parámetros aprendiendo de las tareas asignadas mientras que en el nivel superior el profesor optimiza la selección de las tareas en base al rendimiento del estudiante. Figura extraída de [51].

En ambos métodos [13] [51] se señala que, cuando el conjunto de tareas es discreto y finito, resulta difícil que un agente actuando como profesor supere de manera consistente a un selector de tareas uniforme aleatorio. Aparentemente, esto se debe a que el espacio limitado de tareas restringe la mejora obtenida con estrategias complejas de ACL.

Las redes progresivas introducidas en [53] constituyen otro enfoque diferente de ACL. Se trata de una arquitectura que permite transferir conocimiento entre tareas secuenciales evitando *catastrophic forgetting* [40]. Su funcionamiento comienza entrenando la red en una única tarea. Una vez finalizado el proceso, se congelan todos los pesos de la red y se añade una

nueva columna para aprender a resolver la siguiente tarea. Las nuevas columnas tienen acceso a las representaciones aprendidas por las anteriores. Este mecanismo permite expandir la arquitectura de forma incremental, incorporando conocimiento de nuevas tareas a medida que aparecen. Al entrenarse las tareas de manera secuencial, frecuentemente con una complejidad creciente, se induce un currículo implícito. La principal limitación de este enfoque es el crecimiento indefinido de la arquitectura, lo que restringe su escalabilidad. Además, requiere definir manualmente el orden de las tareas.

3.2.1. Estrategias *harder-first* y *easier-first*

En el trabajo *A Survey on Curriculum Learning* de Wang, et al. (2021) [54] se discute si el enfoque de ordenar tareas de fáciles a difíciles es correcto (*easier-first*). Existen algunos modelos que han demostrado que entrenar con un currículo invertido (*harder-first*), seleccionando las tareas más difíciles primero, puede dar mejores resultados según el entorno.

La estrategia de selección *hard example mining* (HEM) [55] consiste en seleccionar para cada conjunto de entrenamiento las tareas más difíciles, asumiendo que son las más informativas. Sorprendentemente, este enfoque tan contrario respecto a CL encuentra usos en diversas aplicaciones. Trabajos como [56] de Zhang, et al. (2018) demuestran el valor que puede tener el uso de ejemplos difíciles por encima de la progresión sencilla de CL. En el trabajo, se prueba el entrenamiento de redes neuronales para traducción usando un currículo invertido. Se puede observar cómo en algunos casos estos currículos invertidos son los que dan los mejores resultados. Otros trabajos señalan que usar un currículo ordenado de fácil a difícil puede ser incluso contraproducente. Hacoheh, et al. (2019) [57] describe este fenómeno con SPL en sus experimentos.

Esta discusión no es un tema trivial. Actualmente, no existe una respuesta clara. Lo que puede inferirse es que diferentes sistemas pueden beneficiarse de distintos tipos de entornos, ya sean más difíciles o más sencillos. Crear métodos de currículo que permitan ajustarse a estas necesidades de forma dinámica constituye una dirección de investigación prometedora. Sistemas de ACG capaces de reaccionar a una señal del agente para ajustarse a sus necesidades pueden ser el camino a currículos más flexibles y generales.

3.2.2. *Benchmarks* y entornos de evaluación

Surveys como [54] y [47] destacan en sus conclusiones que hay una escasa cantidad de estudios comparativos de ACL. Hay una gran necesidad de *benchmarks* estandarizados que permitan un mejor avance en el estado del arte del campo. Aunque existen aplicaciones en distintos dominios, los con-

juntos de datos, métricas y protocolos usados varían. No hay nada parecido a currículos óptimos de referencia o métricas que unifiquen los resultados experimentales.

En [58] se discute cómo elegir el mejor sistema de CL para una determinada tarea no es sencillo. Para ello se necesitan trabajos comparativos como [59], donde se prueban varios planificadores en tareas de predicción con modelos LSTM [60]. Sin embargo, este tipo de artículos son escasos.

El campo del CL aún requiere una mayor profundidad de investigación. Aunque existen estudios empíricos que respaldan su uso y evidencian sus ventajas, todavía carece de una base teórica sólida. Se han llevado a cabo diversos análisis, pero siguen siendo necesarias garantías más robustas que sustenten de forma general su eficacia.

3.3. *Automated curriculum generation* en la literatura reciente

En esta sección se revisan métodos de ACG, entendidos como aquellos que no dependen de un conjunto de tareas predefinido, sino que generan las tareas del currículo de forma explícita, mediante un proceso de generación deliberado, o de forma implícita, cuando las tareas emergen como consecuencia de la propia dinámica del aprendizaje.

Una de las primeras aproximaciones a un sistema ACG puede encontrarse en la generación procedural de contenidos, donde los entornos se crean mediante reglas paramétricas y semillas aleatorias. Para un conjunto de parámetros y funciones definidos se produce contenido distinto de acuerdo a una semilla o estado inicial. Estos sistemas permiten generar un gran número de tareas diversas a un coste computacional muy bajo, pero dependen de la aleatoriedad computacional y del diseño de funciones generadoras por parte de expertos.

Este tipo de generación se utilizó, por ejemplo, en GVGA [44] por Justesen, et al. (2018) [61] para crear un currículo progresivo de tareas para un agente. Por otra parte, Portelas et al. (2019) [47] profundizaron en el uso del paradigma profesor-estudiante para tareas en un espacio parametrizado. Este nuevo planteamiento expande el dominio de acción del profesor, permitiendo encontrar currículos más expresivos y adaptados al agente. A pesar de que el método es computacionalmente muy eficiente, presenta una restricción importante, los entornos deben ser parametrizables de forma significativa. El problema de este método es que aunque genere escenarios correctos estructuralmente, puede que no tengan mucho valor de aprendizaje debido a las limitaciones de los parámetros.

Otro enfoque de ACG aparece con *automatic domain randomization* (ADR) en la publicación [62]. Dentro de este trabajo, se hace uso de ACG como herramienta. El currículo se forma automáticamente a través del aumento progresivo de la complejidad del entorno mediante ADR. Cada entorno se define como un vector de parámetros que se eligen de forma aleatoria. El aumento de dificultad se obtiene incrementando el rango posible de valores aleatorios de esos parámetros. En otras palabras, expandiendo las distribuciones de los entornos. La virtud de este método es que el agente siempre entrena en el límite de su capacidad, la dificultad se ajusta de forma dinámica. El problema vuelve a ser que requiere entornos parametrizables y que la dificultad esté correlacionada con esos parámetros.

Un enfoque conceptualmente diferente es el planteado por los autores Sukhbaatar, et al. (2017) [45]. Presentan *Self-Play*, donde dos agentes, Alice y Bob, resuelven una misma tarea. Alice parte de un estado inicial y realiza una secuencia de acciones hasta alcanzar un nuevo estado s^* . Ese estado se convierte en el estado inicial de Bob y su objetivo será el estado inicial de Alice. Bob será recompensado si resuelve la tarea eficientemente y Alice si propone tareas que Bob sea capaz de resolver sin ser demasiado sencillas. Este enfoque es muy útil ya que no se requiere de la definición de un conjunto de tareas explícito, el currículo se construye de forma dinámica a partir del agente Alice. Este rasgo dota al método de mayor capacidad de generalización a otros problemas. La limitación es que solo se puede usar en entornos donde cualquier estado al que Alice pueda llegar sea válido como objetivo. Esto implica que los estados se pueden representar en forma de objetivos para el agente y que la distancia entre distintos estados tenga sentido de modo que puedan conformar una progresión de aprendizaje incremental.

Un enfoque más radical es POET propuesto por Wang, et al. (2019) [46]. Consiste en la generación de currículos en los que no solo evoluciona el agente, sino también el entorno. Esto se denomina esquema de optimización coevolutiva. Por un lado, los entornos evolucionan a partir de la generación de nuevos entornos con dificultad creciente, pero conservando aquellos que siguen siendo útiles. Por otro lado, cada política se entrena en un entorno diferente, permitiendo la transferencia entre políticas. Una gran virtud es que el currículo no está predefinido, la dificultad emerge conforme evoluciona el sistema, buscando desafíos para el agente de forma dinámica. Su principal limitación es el alto coste computacional y el escaso control sobre las habilidades que desarrolla el agente.

Existen problemas en RL en los que una política no solo debe rendir bien en una tarea concreta, sino en un conjunto de objetivos. Este escenario, en el que la política se condiciona a diferentes metas, se conoce como *goal-conditioned reinforcement learning*. La política se optimiza para que el agente sea capaz de maximizar la recompensa en presencia de múltiples ob-

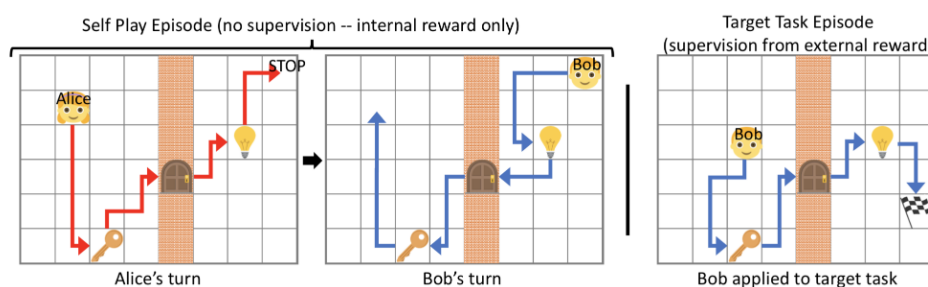


Figura 3.6: Ejemplo del sistema de *Self-Play* con los agentes Alice y Bob. Hay dos tipos de episodios. En el primero, Alice y Bob se turnan recorriendo el entorno. Alice se mueve alterándolo, cuando finaliza le cede el control a Bob. Este debe restablecer lo más rápido posible el entorno a su estado inicial para recibir una recompensa. Así, el proceso se repite, de modo que Alice construye un currículo de forma automática. En la tarea objetivo, se usa la política de Bob para controlar el agente, que recibirá recompensa si llega a la bandera. Figura extraída de [45].

jetivos. Para este tipo de problemas, Florensa et al. (2018) [63] proponen un método llamado *generative goal learning*, donde una red genera objetivos de entrenamiento. Para lograrlo, sigue un proceso iterativo: 1) Evalúa la dificultad de los objetivos generados para la política actual en rangos de dificultad, definiendo un área como dificultad intermedia. 2) Se entrena una red para la generación de objetivos dentro de la región de dificultad intermedia. 3) Los objetivos producidos con la red se emplean para entrenar la política y repetir el proceso desde el primer paso. La principal ventaja de este método es que mantiene al agente en una región de dificultad intermedia, lo que favorece un aprendizaje estable y progresivo. Sin embargo, este beneficio se obtiene a costa de una mayor complejidad computacional, ya que es necesario entrenar y estabilizar simultáneamente un modelo generativo adicional, lo que introduce inestabilidad y aumenta el coste del entrenamiento.

Esta idea fue ampliada por Racaniere, et al. (2020) [64], quienes proponen un sistema formado por un módulo *setter* que genera metas y un *solver* que aprende a alcanzarlas. Este enfoque destaca por no depender de currículos predefinidos, la dificultad se adapta al agente, lo que lo hace especialmente útil en entornos donde las recompensas son escasas. El generador se optimiza para que las metas cumplan tres objetivos: validez (la meta pertenece al espacio del problema), factibilidad (se puede alcanzar el objetivo con alguna política) y cobertura (el conjunto de metas explora suficientemente el espacio de tareas). Estos objetivos garantizan la calidad de las metas generadas. No se define explícitamente una noción de dificultad, sino que los problemas dependen del estado del agente en cada momento, creando un currículo dinámico adaptativo. La principal desventaja de es-

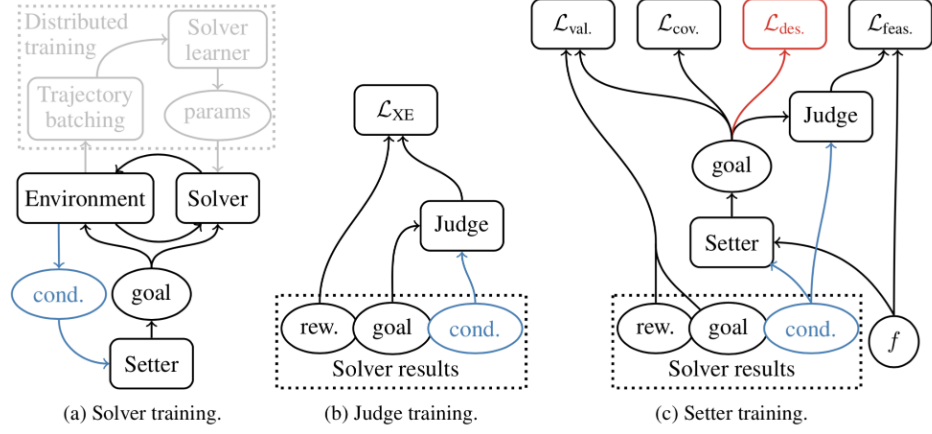


Figura 3.7: Esquema de entrenamiento del *setter-solver*. (a) Durante el entrenamiento del *solver*, este interactúa con el entorno a partir de un objetivo (*goal*) propuesto por el *setter*, que puede estar condicionado por el estado del entorno (azul). (b) El *judge* se entrena supervisadamente mediante la entropía cruzada ($\mathcal{L}_{XE}$), evaluando los resultados del *solver* en términos de recompensa, objetivo y condición. (c) El *setter* se entrena optimizando conjuntamente cuatro criterios evaluados por el *juez*: validez ($\mathcal{L}_{val.}$), cobertura ($\mathcal{L}_{cov.}$), deseabilidad ($\mathcal{L}_{des.}$, en rojo) y factibilidad ($\mathcal{L}_{feas.}$), incorporando además una función auxiliar f . Figura extraída de [64].

te sistema es que consiste en una estructura muy compleja con múltiples componentes entrenándose simultáneamente.

Una visión drásticamente diferente es CARML de Jabri, et al. (2019) [65], una estrategia basada en el aprendizaje de habilidades (*skill-based curriculum*). No se selecciona una distribución de tareas aisladas, sino que se impulsa al agente para aprender habilidades útiles. A partir de la acumulación de estas habilidades, el agente desarrollará comportamientos más complejos. En CARML, las habilidades se organizan dentro de un esquema de meta-aprendizaje, es decir, el agente no solo aprende comportamientos individuales, sino que se adapta a nuevas tareas a partir de experiencias pasadas. Esto simplifica el diseño del currículo, ya que, surge de forma implícita al estructurar el aprendizaje alrededor de habilidades cada vez más complejas. El problema es que las habilidades se aprenden en forma de un espacio latente. Esto significa que es difícil de interpretar y controlar qué habilidades se están aprendiendo exactamente.

Más recientemente, en el artículo *Automaton-Guided Curriculum Generation for Reinforcement Learning Agents* [66] se propone un método ACG usando autómatas finitos deterministas. El punto de partida es expresar el

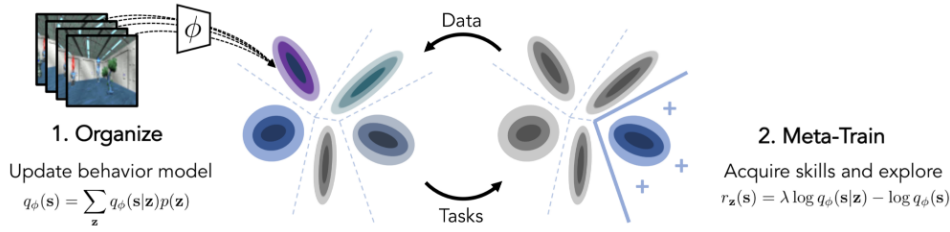


Figura 3.8: Esquema de CARML. En el paso 1 se organiza la experiencia acumulada. Se agrupan comportamientos similares para conocer qué habilidades domina el agente y cuáles necesita mejorar. En el segundo, el agente recibe señales de recompensa que fomentan aprender habilidades que sean útiles y generalizables. Figura extraída de [65].

objetivo final del agente en lógica temporal lineal o LTL [67]. Esto es un formalismo que permite describir propiedades que se deben cumplir a lo largo del tiempo. Por ejemplo, "primero coge la llave, luego abre la puerta". A partir de la especificación, se construye un autómata finito donde los estados intermedios son las subtareas que se deben cumplir. Las transiciones representan las precondiciones que debe verificar el agente para avanzar de un estado a otro. De esta forma, el currículo toma la forma de un grafo de subtareas. Las subtareas iniciales serían las más sencillas, las intermedias aumentarían la dificultad progresivamente y la tarea final representaría el objetivo. Este proceso construye un currículo muy bien estructurado y que se ajusta al objetivo final, pero requiere que el objetivo pueda expresarse formalmente en lógica temporal, lo cual no siempre es sencillo.

En el ámbito de la planificación automática, Torralba et al. (2021) [68] proponen *Autoscale*, una herramienta que genera automáticamente instancias de problemas de planificación para su uso como *benchmarks*. *Autoscale* selecciona instancias de dificultad apropiada teniendo en cuenta el rendimiento de los planificadores actuales y las restricciones del dominio, evitando además sesgos hacia planificadores concretos. Sin embargo, al igual que otros métodos de ACG, depende de generadores parametrizados por dominio, lo que limita su aplicación a dominios para los que se dispone de dicho generador.

Finalmente, es importante mencionar una tesis [69] en la que se estudia el entrenamiento de una política neuronal con los problemas que NeSIG genera. Esta línea de investigación se alinea notablemente con el tema de este TFG, lo que pone de manifiesto el interés en el área. De forma parecida, en dicho trabajo se propone un proceso iterativo en el que se alterna entre entrenar una política en los problemas generados por NeSIG y entrenar NeSIG para generar problemas difíciles de resolver para la política actual. Sin embargo, en la práctica solo realizan una iteración de este proceso. Además, para

entrenar a NeSIG hacen uso de dos métricas distintas que recompensan al modelo por generar instancias lo más difíciles posible, incluso cuando la política es incapaz de resolverlas.

Como puede verse, existen diversas estrategias para abordar ACL y ACG, cada una de ellas presenta virtudes y problemas distintos. No es sencillo determinar cuáles son las virtudes exactas de cada una. Actualmente, no existen muchos estudios comparativos que enfrenten unas contra otras, ni un uso amplio y generalizado de ningún método concreto que respalde su utilización. Esta ausencia dificulta la delimitación precisa del estado del arte actual. En esta sección se han presentado los principales métodos existentes como ejemplos representativos de un área de investigación todavía en desarrollo. Este TFG propone un nuevo enfoque que busca aportar nuevas ideas al área. Para ello, se parte de ideas presentes en trabajos como el *teacher-student* [51], la tesis [69], el modelo *setter-solver* [64] y la generación automática de instancias de planificación [68], con el objetivo de desarrollar un método más refinado y aplicable a un marco más amplio de problemas.

Capítulo 4

Método Propuesto

El método propuesto consiste en un sistema de ACG para entrenar de forma automática agentes de RL en dominios de planificación definidos en PDDL. En este capítulo, se describe la arquitectura general del método junto con el funcionamiento de los distintos componentes que lo integran. Además, se proporcionan detalles sobre cómo se implementa y entrena la arquitectura.

4.1. Descripción general del sistema

El método propuesto combina un agente de RL con NeSIG [17], un generador neuro-simbólico capaz de crear problemas válidos y diversos adaptados al nivel del agente. El agente recibe los problemas generados por NeSIG y aprende a resolverlos. A su vez, el rendimiento del agente se utiliza como señal de retroalimentación para guiar la generación de nuevos problemas. Este ciclo iterativo en el que el agente resuelve problemas y NeSIG recibe retroalimentación para adaptar los nuevos problemas generados conforma el sistema ACG propuesto. A continuación, se detalla la implementación de estos dos componentes en profundidad. La Figura 4.1 muestra un esquema general del sistema.

4.1.1. Agente base

El agente se implementa mediante una arquitectura neuro-simbólica basada en NLMs, lo que le permite razonar sobre la estructura relacional de los estados definidos en el dominio. Para el entrenamiento se emplea el algoritmo PPO, que optimiza la política del agente de forma estable en espacios de acciones discretos y de alta dimensionalidad. Como PPO es un algoritmo actor-crítico, la arquitectura del agente se construye con dos NLMs, una

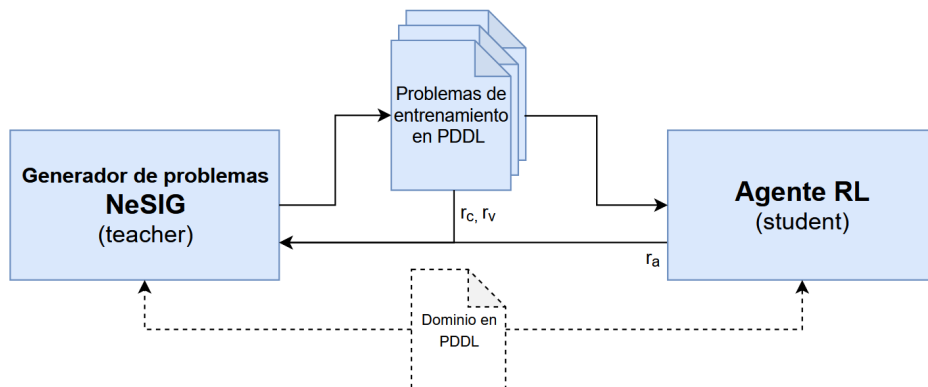


Figura 4.1: Esquema visual del sistema ACG propuesto, en azul se representan los elementos principales mientras que las líneas discontinuas representan el dominio. El modelo parte de un **dominio PDDL específico de la tarea que se desea resolver**. El dominio se proporciona como entrada al agente de RL (4.1.1) y al generador de problemas NeSIG (2.4), que actúan respectivamente como *teacher* y *student* del paradigma descrito en el estado del arte (3.2). NeSIG, tomando como referencia el dominio, genera problemas para el agente, y a cambio recibe de estos una recompensa de consistencia y otra de diversidad. El agente de RL resuelve los problemas y devuelve a NeSIG una señal de recompensa de calidad, que será mayor cuanto más haya aprendido el agente de cada problema generado. Esto forma un ciclo de entrenamiento cerrado entre NeSIG y el agente, análogo a la relación entre un profesor y su alumno.

para la política o actor y otra para la función de valor o crítico.

El agente se entrena para resolver instancias de planificación definidas en PDDL, donde cada instancia está compuesta por un estado inicial s_0 y un estado objetivo g . A partir de esta configuración, el agente aprende una política que transforme s_0 en un estado que satisfaga g , optimizando el número de pasos. Formalmente, el proceso se modela como un MDP (S, A, T, R) , en el que cada episodio corresponde a la resolución de una instancia de planificación:

- S : conjunto de estados, donde cada estado representa una configuración válida del entorno descrita mediante átomos del lenguaje PDDL.
- A : conjunto de acciones del dominio, definido por las acciones parametrizadas en la especificación PDDL. En cada estado, solo se consideran acciones aplicables, es decir, aquellas cuyas precondiciones se satisfacen.

- $T : S \times A \rightarrow S$: función de transición determinista, que describe la evolución del estado tras la aplicación de una acción válida del dominio.
- $R : S \times A \rightarrow \mathbb{R}$: función de recompensa definida en función del progreso hacia el objetivo y del número de pasos realizados. La señal de recompensa está diseñada para incentivar la resolución correcta de los problemas en el menor número de pasos posible. Para ello, el agente recibe una penalización por cada acción ejecutada, normalizada por el número máximo de acciones permitidas. Adicionalmente, si el agente alcanza el estado objetivo, recibe una recompensa positiva al final del episodio, promoviendo así el éxito en la tarea.

Sea T el número máximo de acciones permitidas para un problema, N el número de pasos ejecutados por el agente, y $c \in \{0, 1\}$ una variable indicadora que toma el valor 1 si el objetivo es alcanzado y 0 en caso contrario. La recompensa en cada paso t de la trayectoria se define como:

$$r_t = \frac{r_{step}}{T} + c * r_{goal} \quad (4.1)$$

donde $r_{step} < 0$ representa la penalización por acción y $r_{goal} > 0$ la recompensa asociada a la consecución del objetivo.

En cada instante, el agente observa una representación del estado actual junto con la descripción del dominio en PDDL, e identifica el conjunto de acciones aplicables a partir de las precondiciones. A partir de esta información, selecciona una acción que desencadena una transición hacia un nuevo estado y recibe una recompensa asociada a dicha transición. Este proceso se repite iterativamente hasta que el episodio finaliza, ya sea cuando se alcanza un estado objetivo c o cuando se ha superado un horizonte máximo de T acciones.

4.1.2. Generador neuro-simbólico de problemas

Como generador del sistema se ha usado NeSIG, un generador neuro-simbólico general de instancias de problemas de planificación automática (véase 2.4). NeSIG debe adaptar los problemas generados a las capacidades del agente. Para lograr que NeSIG se ajuste a esta necesidad, ha sido necesario modificar la señal de recompensa original. Para ello se ha heredado la señal de consistencia (r_c) y la señal de diversidad (r_v), y se ha sustituido la señal de dificultad (r_f) por una propia (r_a):

$$r = r_c + \min\left(\frac{r_v}{\epsilon}, 1\right) \cdot r_a \quad (4.2)$$

La señal de consistencia r_c toma el valor -1 cuando, al finalizar la fase de generación del estado inicial, este resulta inconsistente, es decir, incumple alguna de las reglas de consistencia eventual definidas para el dominio; en cualquier otro caso su valor es 0. Además, un problema inconsistente tiene r_v y r_a con valor 0.

La señal de diversidad r_v se calcula únicamente al término de la generación completa del problema, como la distancia media de dicho problema respecto al resto de problemas generados, medida sobre un conjunto de características interpretables (número de objetos de cada tipo, número de átomos de cada predicado y estadísticos de conectividad entre tipos de objetos); de este modo, un problema estructuralmente distinto a los demás obtiene una diversidad mayor. Los valores de la diversidad están contenidos en el intervalo $[0, 1]$, de menos a más diversos.

Por último, la nueva señal de dificultad, r_a , se define a partir del *value loss* del agente. Esta señal mide el error de la función de valor, es decir, cuánto se equivoca el agente al predecir el retorno esperado desde el estado actual. Al tratarse de un error cuadrático, r_a toma siempre valores no negativos y, a diferencia de la señal de diversidad r_v , no está acotada superiormente: su valor crece de forma proporcional al cuadrado del error de predicción, por lo que situaciones ligeramente inesperadas producen valores cercanos a cero, mientras que situaciones muy sorprendentes para el agente pueden dar lugar a valores considerablemente más altos, sin un límite máximo fijo. Un valor de r_a próximo a cero refleja que la situación resultaba predecible para el agente, bien porque ya domina ese tipo de problemas, bien porque ha aprendido a anticipar correctamente su dificultad (incluso si esta es alta). Por el contrario, un valor elevado de r_a indica que la situación resultaba inesperada y que, por tanto, las predicciones del agente eran inexactas, señalando una zona de incertidumbre genuina en su aprendizaje. De este modo, la recompensa resultante incentiva al generador a producir problemas genuinamente novedosos y desafiantes para el agente, en lugar de simplemente problemas difíciles según una medida externa y fija como el número de nodos expandidos por un planificador.

Estas tres señales se combinan mediante la Ecuación 4.2, donde el término de diversidad actúa como un factor de escala sobre la dificultad: si r_v alcanza o supera un umbral ϵ (hiperparámetro específico de cada dominio), la dificultad se aplica en su totalidad ($r = r_a$); si r_v queda por debajo de dicho umbral, la contribución de r_a se reduce proporcionalmente, llegando a anularse por completo cuando $r_v = 0$. De esta manera, el generador se ve incentivado a producir problemas que sean simultáneamente consistentes, suficientemente diversos entre sí y lo más difíciles de resolver posible.

La nueva señal de dificultad (r_a) se deriva de la función de pérdida del agente y se articula en torno al concepto de *sorpresas* [70]. Este concepto se

entiende como una medida de la novedad o la imprevisibilidad de las experiencias del agente. La sorpresa cuantifica hasta qué punto una transición resulta inesperada para el modelo, ya sea por un error elevado en la estimación del valor o por la necesidad de ajustes significativos en la política. En ambos casos, se asume que una transición altamente sorprendente es indicativa de una experiencia con alto potencial informativo para el agente. En el contexto del RL, el aprendizaje se produce principalmente cuando el modelo debe corregir errores significativos en sus estimaciones, por lo que las situaciones que generan mayor sorpresa tienden a ser aquellas que aportan mayor información útil.

4.2. Entrenamiento de la arquitectura

El entrenamiento del sistema NeSIG junto con el agente de RL se formula como un proceso de co-entrenamiento iterativo entre un generador de problemas (teacher) y un agente resolutor (student). Este apartado detalla el comportamiento temporal del sistema, incluyendo las distintas fases de entrenamiento y los mecanismos introducidos para estabilizar el aprendizaje. En la Figura 4.2 se detalla una representación visual.

Durante el entrenamiento se observó que NeSIG requiere varias iteraciones iniciales para generar problemas consistentemente válidos. En esta fase temprana, la alta proporción de instancias inconsistentes impide que el agente de RL disponga de una señal de recompensa útil, ralentizando enormemente el aprendizaje. Para mitigar este problema, antes de iniciar la interacción completa entre el generador y el agente se introduce una fase de pre-entrenamiento o *warm-up*, en la que solo se entrena a NeSIG utilizando únicamente las señales de consistencia r_c y diversidad r_v . La señal r_a se sustituye por una señal constante. Durante esta etapa, el generador aprende a producir problemas estructuralmente válidos sin depender de la retroalimentación del agente.

Una vez finalizado el *warm-up*, comienza el ciclo principal de entrenamiento. En cada iteración se ejecutan los siguientes pasos:

1. **Generación de problemas:** NeSIG genera un conjunto de instancias de planificación en PDDL. Estas instancias pueden ser inconsistentes (problemas no válidos) o consistentes (problemas válidos). Además, se obtiene la recompensa de consistencia r_c y diversidad r_v , usadas más adelante para entrenar a NeSIG.
2. **Filtrado de consistencia:** se descartan los problemas que no satisfacen las restricciones estructurales del dominio, garantizando que se entrena sobre instancias válidas.

3. **Experience replay:** durante el entrenamiento se observó que el agente tendía a olvidar rápidamente problemas previamente resueltos (*catastrophic forgetting*). Para mitigar este efecto se introduce un *buffer* de experiencias que almacena problemas que han producido un aprendizaje significativo en el agente y los reintroduce periódicamente en el entrenamiento. En esta fase, se agregan un número determinado de problemas del *buffer* escogidos de forma aleatoria a la distribución de problemas consistentes generados por NeSIG. Los problemas introducidos en el *buffer* son aquellos que tienen una recompensa r_a superior a un umbral.
4. **Resolución por el agente (student):** el agente de RL intenta resolver cada problema interactuando con el entorno desde el estado inicial, seleccionando acciones hasta alcanzar el objetivo o agotar un horizonte máximo de T pasos. Durante este proceso se recolectan trayectorias de interacción (s_t, a_t, r_t, s_{t+1}) .
5. **Actualización del agente:** el student se actualiza mediante PPO, optimizando una política actor-crítico con el objetivo de maximizar la recompensa acumulada. En la práctica, esto equivale a minimizar el número de acciones necesarias para resolver los problemas generados.
6. **Cálculo de señal de recompensa r_a :** el rendimiento del agente sobre los problemas generados se utiliza para construir la señal de dificultad adaptativa r_a .
7. **Actualización del generador:** las políticas de generación de estados iniciales y objetivos se actualizan mediante PPO, utilizando una combinación de señales de consistencia, diversidad y la recompensa adaptativa r_a .

Durante el desarrollo del sistema se identificó un problema de *moving targets*, ya que la distribución de problemas generados por NeSIG depende directamente del comportamiento del agente de RL, cuya política cambia a lo largo del entrenamiento. Como consecuencia, la recompensa de NeSIG puede volverse inestable. Para estabilizar este comportamiento, la implementación no actualiza ambas redes de forma simultánea en todo momento, sino que introduce un esquema de congelación alternada. Durante ciertos intervalos de entrenamiento se congela una de las dos redes (*teacher* o *student*) mientras la otra continúa optimizando sus parámetros. Tras un número fijo de iteraciones, se invierte el proceso. De esta forma, la red previamente congelada se descongela y se entrena, mientras que la otra pasa a congelarse. Este esquema se implementa mediante hiperparámetros que determinan durante cuántas iteraciones cada componente permanece congelado antes de alternar el proceso. Así, cada red se actualiza respecto a una versión

temporalmente estable de la otra, reduciendo la inestabilidad inducida por cambios simultáneos en ambas políticas.

En conjunto, el sistema implementa un bucle cerrado de co-evolución: NeSIG produce problemas cuya dificultad depende del rendimiento del agente, mientras que el agente mejora progresivamente gracias a la distribución de problemas generados. El *warm-up* inicial, el *experience replay* y la congelación de redes permiten que este proceso sea estable en lugar de colapsar de forma temprana.

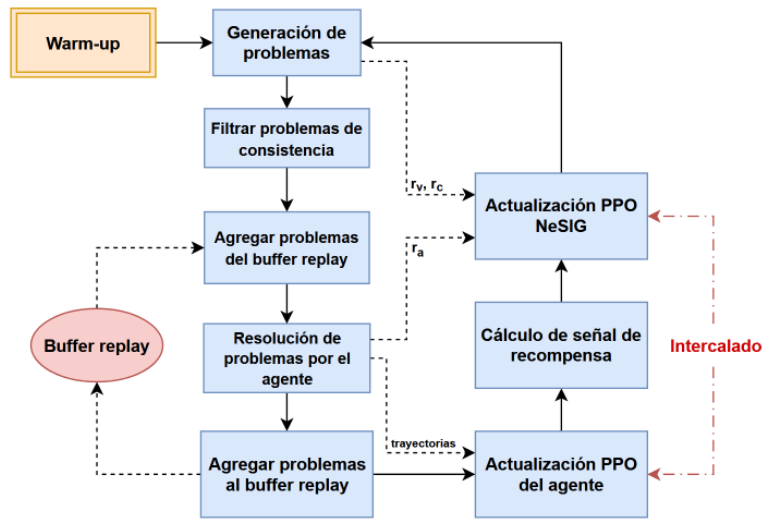


Figura 4.2: Esquema del proceso temporal de co-entrenamiento entre NeSIG y el agente de RL. Las líneas continuas representan el ciclo de entrenamiento, mientras que las líneas discontinuas indican los puntos de intercambio de información entre módulos. El sistema inicia con una fase de *warm-up*, en la que NeSIG se entrena únicamente con las señales de consistencia (r_c) y diversidad (r_v) para aumentar el número de problemas consistentes. Posteriormente, se activa el bucle de co-entrenamiento, en el que NeSIG genera problemas, se filtran aquellos inconsistentes y se agregan problemas adicionales del *buffer replay*. Los problemas inconsistentes no se regeneran. El agente toma el conjunto y obtiene las trayectorias. Tras esto, se almacenan las experiencias relevantes en el *buffer replay* y se entrena el agente mediante PPO. Su rendimiento se emplea para estimar la señal r_a , que junto con r_c y r_v se utiliza para actualizar NeSIG también mediante PPO. Es importante mencionar, que el sistema congela la actualización de NeSIG y el agente de forma intercalada para estabilizar el entrenamiento.

4.3. Detalles técnicos de la implementación

La implementación del método propuesto, incluyendo la infraestructura para entrenar los modelos, crear la arquitectura de los agentes y traducir los dominios a una representación interna; se ha implementado mediante el lenguaje de programación Python [71]. El dominio a resolver por los agentes se representa utilizando el lenguaje de planificación automática PDDL. El desarrollo y el control de versiones del código se han gestionado mediante GitHub, donde se han realizado *commits* periódicos que documentan la evolución del proyecto. El repositorio completo puede consultarse en <https://github.com/Ellipsim/agentRL-with-NLM>. Dicho repositorio se estructura en las siguientes carpetas:

- **data:** contiene los datos de entrenamiento y de prueba para los agentes. En el caso de los modelos ACG, contiene directorios donde se almacenan los problemas temporales durante el entrenamiento. Estos directorios se borran completamente al inicio de cada nuevo entrenamiento.
- **nesig_teacher:** submódulo que contiene la implementación del método NeSIG, utilizado como referencia y apoyo para el desarrollo experimental del proyecto. El repositorio se puede encontrar en el enlace: <https://github.com/ari-dasci/S-PlanningProblemGeneration>.
- **problem_generator:** submódulo que alberga un generador de problemas PDDL capaz de operar en un amplio grupo de dominios. Se usa tanto para la creación del conjunto de prueba, como para la generación de datos de entrenamiento de los métodos comparativos *baselines* de los experimentos. El repositorio se puede encontrar en el enlace: <https://github.com/AI-Planning/pddl-generators>.
- **src:** contiene toda la implementación del método principal del proyecto, así como, los métodos comparativos que se usan en los experimentos.
- **tests:** contiene pruebas para depurar el código elaboradas con la biblioteca Pytest.

La compatibilidad de las bibliotecas se ha gestionado mediante Anaconda, un gestor de entornos y paquetes. A continuación, se proporciona una lista con las principales bibliotecas auxiliares utilizadas junto con su respectivo uso:

- **PyTorch:** *framework* principal empleado para la implementación de los modelos de DL y del agente de RL.

- **CUDA para PyTorch:** soporte de aceleración por GPU que permite reducir significativamente los tiempos de entrenamiento mediante computación paralela.
- **PyTorch Lightning:** herramienta utilizada para estructurar y modularizar el bucle de entrenamiento, facilitando la reproducibilidad y organización de los experimentos.
- **NumPy:** biblioteca empleada para la manipulación eficiente de estructuras de datos numéricas y el cálculo de métricas de rendimiento.
- **Matplotlib:** biblioteca utilizada para la representación gráfica de los resultados experimentales.
- **Pandas:** herramienta destinada a la gestión, procesamiento y análisis de los datos generados durante la experimentación.
- **Neural Logic Machine:** implementación de la arquitectura neuro-simbólica empleada tanto en el agente de RL como en NeSIG.
- **PDDL Prover:** herramienta utilizada para verificar la validez lógica de los problemas definidos en PDDL, permitiendo su comprobación de forma independiente del dominio.
- **Lifted PDDL:** *framework* ligero empleado para la traducción y manipulación de especificaciones PDDL de manera estructurada.
- **TensorBoard:** herramienta de visualización de datos utilizada para monitorizar el proceso de entrenamiento y las métricas de rendimiento.
- **PyTest:** herramienta utilizada para la implementación de pruebas automatizadas, permitiendo verificar el correcto funcionamiento de los distintos módulos del sistema y garantizar la estabilidad del código durante el desarrollo.

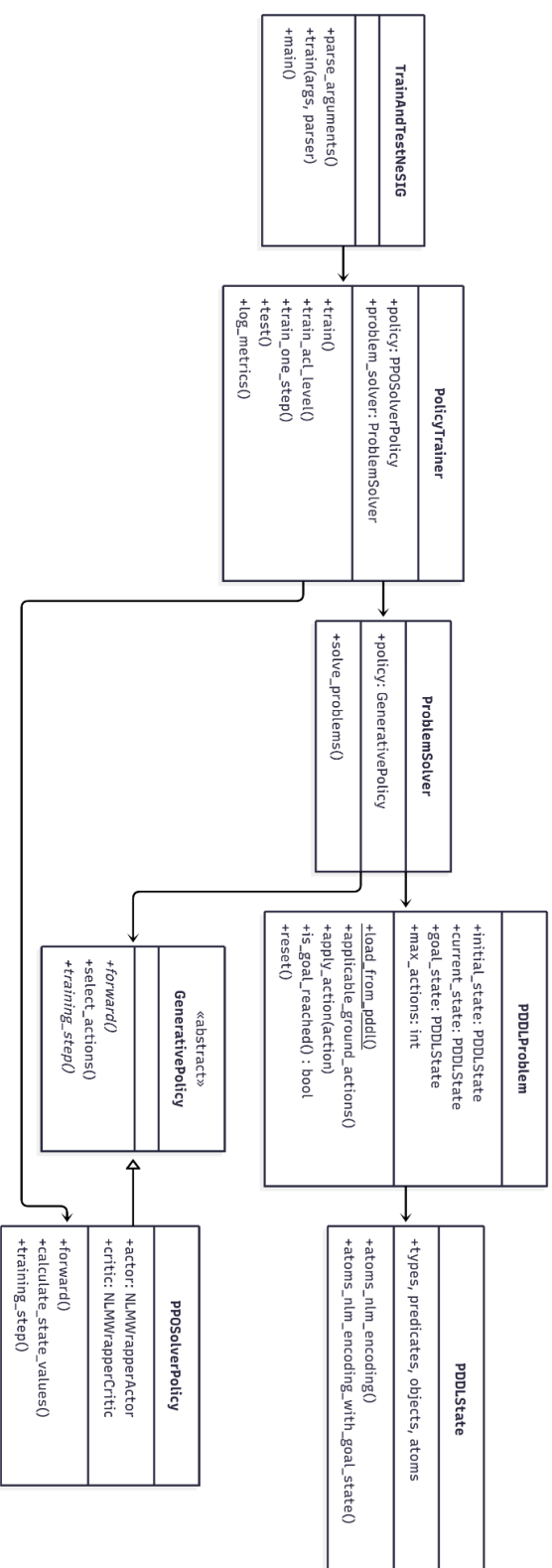


Figura 4.3: Diagrama de clases de la implementación del sistema ACG. Se muestran las principales clases que componen la arquitectura, así como las relaciones entre ellas.

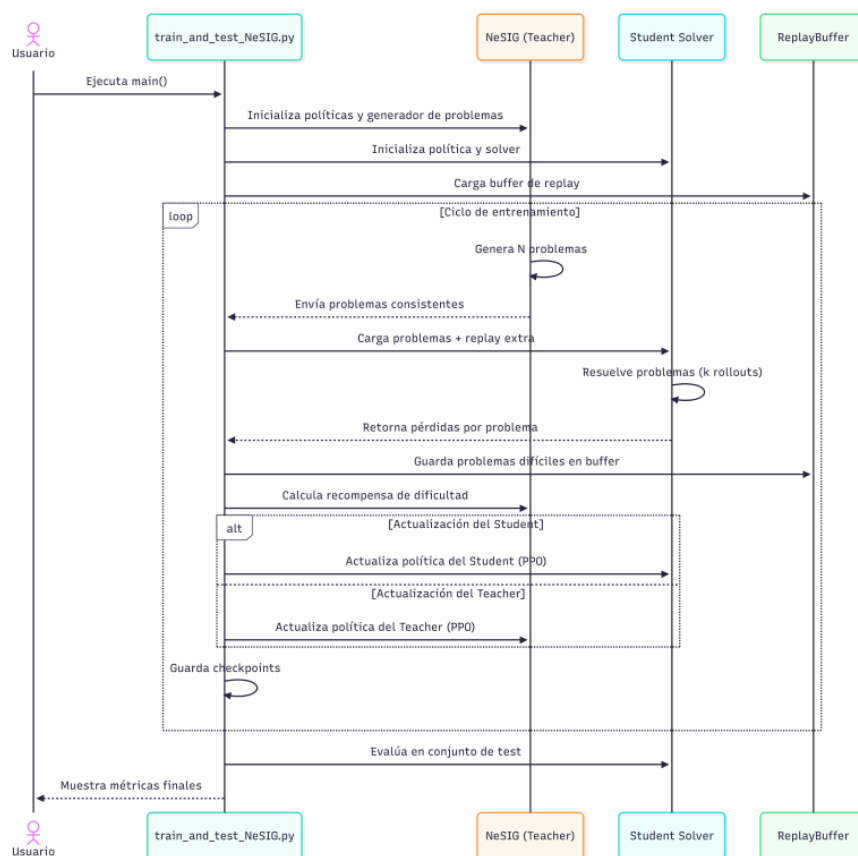


Figura 4.4: Diagrama de secuencia que representa el flujo de ejecución durante una iteración del entrenamiento del sistema ACG. Se muestra la interacción entre el controlador del entrenamiento, el generador NeSIG, el entorno de planificación y el agente de RL, desde la generación de los problemas hasta la actualización de ambos modelos a partir de las recompensas obtenidas.

Capítulo 5

Experimentación

Este capítulo presenta los experimentos realizados para evaluar el método propuesto. En primer lugar, se describe la metodología experimental empleada. A continuación, se presenta cada experimento junto con el resultado obtenido. Finalmente, se incluye una discusión global sobre las conclusiones alcanzadas.

5.1. Diseño experimental

En esta sección se describe la metodología experimental empleada en la evaluación del método propuesto. Concretamente, se especifica el entorno de ejecución, los datos y las métricas utilizadas, los métodos de referencia con los que se compara la propuesta y un breve resumen sobre cada uno de los experimentos diseñados.

5.1.1. Entorno de ejecución

La ejecución del modelo propuesto y los métodos usados durante la experimentación se realiza mediante el uso de los archivos en la carpeta *src/controller/*. Existe un archivo con el nombre *train_and_test.py* para el agente RL básico sin currículo, un archivo con el nombre *train_and_test_acg.py* para el sistema ACG usando un generador parametrizado y un archivo con el nombre *train_and_test_nesig.py* para el método propuesto. Estos archivos entrenan el modelo con los datos especificados y realizan una prueba de rendimiento final sobre el conjunto de problemas de prueba generados previamente. Reciben múltiples parámetros como entrada que se pueden consultar mediante la invocación del comando *--help* o leyendo el *README.md* del proyecto.

Para la ejecución se hace uso de los servidores GPU de la universidad.

Estos servidores contienen varios modelos de GPU agrupados en distintos nodos. Los modelos utilizados en el proyecto son: GTX Titan X Pascal, GTX Titan Xp, Titan RTX y Quadro RTX 8000. Desde la universidad, se ha permitido en este proyecto el uso de cuatro GPUs en paralelo. La gestión de dicho *hardware* se ha realizado mediante el gestor de colas Slurm [72], que permite asignar trabajos a los distintos nodos sin preocupaciones por la disponibilidad o el uso compartido con otros usuarios.

5.1.2. Dominio experimental

El dominio utilizado para los experimentos es *Blocksworld* [23], un entorno clásico de planificación automática. El dominio consiste en un conjunto de bloques apilables distribuidos sobre una mesa. Cada problema define una configuración inicial de los bloques y un estado objetivo. El agente debe alcanzar dicho estado final mediante la búsqueda de una secuencia de acciones que reorganicen los bloques de forma correcta. La especificación concreta del dominio empleado se puede consultar en el Apéndice 6.

Este dominio es ampliamente utilizado como banco de pruebas en planificación automática. La estructura del dominio es discreta y sus problemas resultan fácilmente escalables en dificultad. Estas dos propiedades lo convierten en un entorno adecuado para el estudio del sistema ACL propuesto.

El problema de *Blocksworld* puede formularse como un MDP donde:

- S : conjunto de configuraciones posibles de bloques, representadas mediante los predicados $\text{On}(x,y)$, $\text{OnTable}(x)$, $\text{Clear}(x)$, $\text{Holding}(x)$ y HandEmpty .
- A : conjunto de acciones de manipulación de bloques, definido por las siguientes operaciones:
 - $\text{PickUp}(x)$: permite levantar un bloque x desde la mesa cuando este no tiene ningún bloque sobre él y el agente no está sosteniendo otro bloque.
 - $\text{PutDown}(x)$: coloca el bloque x que está siendo sostenido por el agente sobre la mesa.
 - $\text{Stack}(x,y)$: apila el bloque x que está siendo sostenido sobre otro bloque y , siempre que y no tenga otro bloque encima.
 - $\text{Unstack}(x,y)$: retira el bloque x que se encuentra encima de y , colocándolo en la mano del agente si x no tiene un bloque encima.
- $T(s' \mid s, a)$: define la transición entre configuraciones de bloques tras la ejecución de acciones válidas. Actualiza el estado de acuerdo con las precondiciones y efectos definidos en PDDL.

- $R(s, a, s')$: función de recompensa utilizada en el entorno. Se penaliza al agente por cada paso dado y se proporciona una recompensa positiva cuando alcanza el objetivo. De esta forma, se incentiva a alcanzar el objetivo en el menor número de pasos posibles. Los detalles se pueden consultar en la subsección previa 4.1.1.

5.1.3. Datos

Los datos de entrenamiento se generan en cada iteración de forma dinámica mediante NeSIG. En cada iteración, se generan 70 problemas nuevos. Además, se agregan 30 problemas adicionales del *buffer replay*, resultando en un total de 100 problemas. Este valor se ha determinado empíricamente, comprobando que un número mayor de problemas no produce mejoras significativas en el rendimiento. Es importante aclarar que los problemas inconsistentes no se regeneran. Si de los 70 generados solo 30 son válidos, se tomarán estos últimos y se agregarán 30 del *buffer replay*, entrenando finalmente con 60 problemas.

Los problemas introducidos en el *buffer* son aquellos que poseen una recompensa (r_a) asociada superior a 0.05. Esto permite asegurar que no se introduzcan problemas demasiado fáciles que no aportan ningún aprendizaje al agente.

Para evaluar el modelo no se han empleado problemas generados por NeSIG, ya que esto puede conducir a conclusiones sesgadas: si NeSIG generase problemas muy sencillos, el agente alcanzaría fácilmente un *success rate* del 100 % en ellos, lo que daría una evaluación engañosa al compararla con el rendimiento de otros agentes. Es por ello que el conjunto de evaluación se genera haciendo uso de un generador parametrizado capaz de producir problemas de *Blocksworld* (véase sección 4.3). Dicho conjunto está compuesto por 300 problemas de *Blocksworld*, con un número de bloques comprendido entre 2 y 30. Se ha generado un conjunto de evaluación para el experimento 1. Como se toman decisiones de ajuste sobre los resultados obtenidos, se genera otro conjunto nuevo para los experimentos 2 y 3 para evitar *data snooping*.

La Figura 5.1 muestra la distribución del número de bloques de los problemas que componen el conjunto de evaluación usado para los experimentos 2 y 3. Se observa que el conjunto cubre de forma prácticamente completa el rango de tamaños considerado, entre 2 y 30 bloques, sin huecos relevantes ni una concentración marcada en ninguna zona concreta. Aunque el número de problemas por cada tamaño exacto presenta cierta variabilidad, propia de un muestreo aleatorio, la distribución agrupada por rangos confirma un reparto razonablemente equilibrado entre los distintos niveles de dificultad.

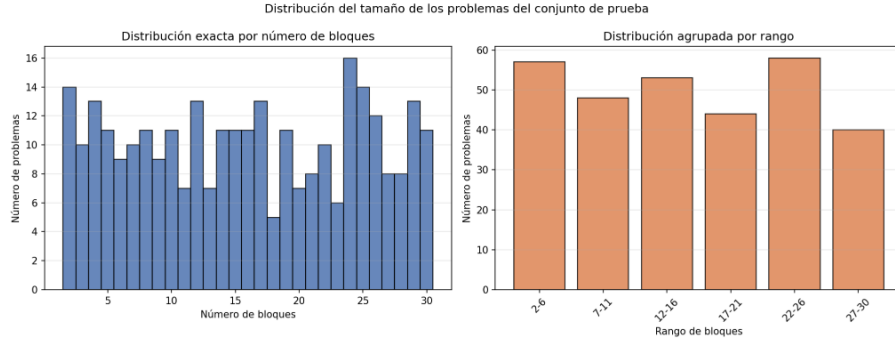


Figura 5.1: Distribución del número de bloques de los 300 problemas que componen el conjunto de evaluación para los experimentos 2 y 3. A la izquierda, número de problemas por cada tamaño exacto (de 2 a 30 bloques); a la derecha, la misma distribución agrupada en rangos de 5 bloques.

5.1.4. Métricas

Las métricas de las que se hace uso durante los experimentos se explican a continuación:

- **Success rate:** mide el porcentaje de problemas que el agente es capaz de resolver. Permite evaluar el rendimiento general de cada método presentado, de forma que se pueda conocer cuál alcanza un mayor éxito en el conjunto de evaluación.

$$Success\ Rate = \frac{\text{problemas resueltos}}{\text{problemas totales}} \quad (5.1)$$

- **Área bajo la curva (AUC):** esta métrica cuantifica el rendimiento medio de cada método durante su propia ejecución, bajo el criterio de parada establecido. Permite conocer qué métodos logran un *success rate* alto con la mayor eficiencia. Si se denota $f(t)$ como el *success rate* en la iteración t , el área bajo la curva se calcula como:

$$AUC = \int_0^T f(t)dt \quad (5.2)$$

donde T es la iteración final. Dado que el *success rate* solo se evalúa en iteraciones discretas, la integral se puede aproximar como una suma. Además, para interpretar más fácilmente los resultados, se normaliza por el número de iteraciones T :

$$\text{AUC} \approx \frac{1}{T} \sum_{i=1}^T f(t_i) \quad (5.3)$$

- **Maximum drawdown:** mide la mayor pérdida de rendimiento experimentada por el agente tras alcanzar un máximo previo durante el entrenamiento. Esta métrica permite cuantificar la severidad de las regresiones sufridas por la curva de aprendizaje. Si $f(t)$ representa el *success rate* en la iteración t , se define como:

$$\text{Maximum Drawdown} = \max_{t \in [0, T]} \left(\max_{s \leq t} f(s) - f(t) \right) \quad (5.4)$$

Valores bajos indican que el entrenamiento mantiene el rendimiento alcanzado y evita pérdidas importantes tras mejorar.

- **Downside deviation:** cuantifica la magnitud media de las caídas de rendimiento entre iteraciones consecutivas, ignorando las mejoras. De esta forma, mide cuánto empeora el entrenamiento cuando se producen regresiones. Se calcula como:

$$\text{Downside Deviation} = \sqrt{\frac{1}{T-1} \sum_{i=2}^T \min(0, f(t_i) - f(t_{i-1}))^2} \quad (5.5)$$

Cuanto menor es esta métrica, más suaves son las regresiones observadas durante el entrenamiento.

- **Regression rate:** mide la frecuencia con la que el rendimiento disminuye entre dos evaluaciones consecutivas. Se define como la proporción de iteraciones en las que el *success rate* es inferior al obtenido en la evaluación anterior:

$$\text{Regression Rate} = \frac{\sum_{i=2}^T \mathbb{I}(f(t_i) < f(t_{i-1}))}{T-1} \quad (5.6)$$

donde $\mathbb{I}(\cdot)$ es la función indicadora, que toma valor 1 cuando la condición se cumple y 0 en caso contrario. Valores bajos indican un aprendizaje más estable y con menos retrocesos.

5.1.5. Hiperparámetros del método propuesto

El método propuesto está compuesto por dos elementos, un agente básico y el generador de instancias neuro-simbólico NeSIG. La arquitectura de ambos está basada en NLMs, para las cuales se ha usado la misma configuración de la Tabla 5.1. Estos valores se han extraído de [17] donde se usa NeSIG para generar problemas en *Blocksworld*. En este trabajo se ha comprobado empíricamente que dichos valores proporcionan un buen desempeño.

Arquitectura NLM agente y NeSIG		
Componente	Parámetro	Valor
Profundidad	<i>depth</i>	5
Anchura (aridad máxima)	<i>breadth</i>	3
Unidades ocultas NLM	<i>hidden features</i>	8
MLP interno	<i>hidden features</i>	0
<i>Residual</i>	<i>residual</i>	<i>input</i>
<i>Batch normalization</i>	<i>batch norm</i>	False

Tabla 5.1: Configuración de la arquitectura NLM utilizada como *backbone* del *actor* y *critic* en el agente de resolución y el generador de instancias NeSIG.

Los hiperparámetros del agente se pueden consultar en la Tabla 5.2. Estos valores se obtuvieron mediante el ajuste experimental del agente. Se partió de una configuración estándar para problemas de RL y se modificó progresivamente mediante el entrenamiento con problemas obtenidos del generador parametrizado.

Agente de RL básico		
Hiperparámetro	Símbolo	Valor
<i>Learning rate</i>	α	1×10^{-3}
Factor de descuento	γ	0.99
Factor GAE	λ	0.95
Épocas PPO	—	2
Tamaño del <i>minibatch</i>	b	64
<i>Clipping</i> de gradiente	—	0.5
Peso del crítico	—	0.1
Recompensa objetivo	—	1.0
Número máximo de acciones	—	116
Penalización por paso	—	$-1/116$

Tabla 5.2: Hiperparámetros del agente de RL básico.

Los hiperparámetros de NeSIG se pueden consultar en la Tabla 5.3. Una vez configurado el agente básico, se dedicó un esfuerzo considerable a obtener

una buena configuración para NeSIG. Se obtuvo mediante repetidas pruebas y un análisis detallado, ya que el modelo resultó ser bastante delicado e inestable ante los cambios en los hiperparámetros como se demuestra en los experimentos (véase 5.2.1).

Generador NeSIG	
Hiperparámetro	Valor
<i>Learning rate (init)</i>	1×10^{-3}
<i>Learning rate (goal)</i>	1×10^{-3}
Épocas PPO (<i>init</i>)	12
Épocas PPO (<i>goal</i>)	12
Coefficiente de entropía (<i>init</i>)	0.04
Coefficiente de entropía (<i>goal</i>)	0.04
Máx. acciones (<i>init</i>)	30
Máx. acciones (<i>goal</i>)	60
<i>Clipping</i> PPO	0.2
Umbral de diversidad	0.02
Factor de intercalación	3
<i>Warm-up</i>	300

Tabla 5.3: Hiperparámetros del generador de instancias NeSIG.

5.1.6. Métodos del estudio de ablación

Un estudio de ablación consiste en aplicar variaciones al método principal propuesto, eliminando o modificando alguno de sus módulos, con el objetivo de evaluar el impacto que tiene cada uno en el resultado final. El método propuesto presenta una complejidad elevada debido a la cantidad de elementos que contiene, por lo que se ha decidido diseñar varias variaciones o ablaciones que permitan realizar un estudio de ablación:

Ablación 1: Sistema exclusivamente con *warm-up*

La primera ablación propone un sistema en el que el profesor únicamente se entrena en el *warm-up* con una señal de dificultad constante. De esta forma, el agente entrenaría sin enviar ningún tipo de retroalimentación a NeSIG. Con esta variación se busca evaluar si el ciclo iterativo mejora los problemas generados por NeSIG y cuánta inestabilidad aporta.

Los hiperparámetros y arquitectura usada son las mismas que en el método principal (véase Tablas 5.1, 5.2 y 5.3), aunque al eliminar el ciclo iterativo ciertos hiperparámetros se pueden ignorar, ya que no tendrán impacto en el entrenamiento.

Ablación 2: Sistema con *warm-up* y 1 iteración de entrenamiento

La segunda ablación también entrena al profesor principalmente en el *warm-up* con una señal constante, pero además realiza una sola iteración de ajuste con el estudiante después del *warm-up*. Con esta variación se busca evaluar si un pequeño ajuste al inicio es más efectivo que ajustar repetidamente en cada iteración.

De la misma forma que la primera ablación, los hiperparámetros y arquitectura usada son las mismas que en el método principal (véase Tablas 5.1, 5.2 y 5.3).

5.1.7. Métodos de referencia (*baselines*)

Con el objetivo de comparar el sistema ACG desarrollado y comprobar su efectividad, se han diseñado dos métodos *baselines*. El primero es un agente sin ningún tipo de *curriculum learning*, que se entrena con problemas totalmente aleatorios. Se usará como referencia mínima que cualquier sistema más complejo debería superar. El segundo es un sistema de ACG implementado mediante un generador parametrizado (véase sección 4.3), que permite comparar el rendimiento de NeSIG frente a un método más básico.

Agente sin *curriculum learning*

El agente básico no genera sus propios datos por lo que aprende con un conjunto de entrenamiento generado mediante el generador parametrizado (véase sección 4.3). Este conjunto estará formado por 100 problemas generados de forma aleatoria en cada iteración. La dificultad de los problemas no es progresiva, sino que el tamaño de los problemas se selecciona de forma aleatoria entre 2 y 30 bloques.

Este método reutiliza la implementación del agente de RL desarrollada para el método propuesto. Se entrena utilizando el algoritmo PPO con la configuración de la NLM y de los hiperparámetros del método principal (véase Tablas 5.2 y 5.1).

Sistema de *automated curriculum generation* con generador parametrizado

Este sistema sustituye NeSIG por el generador parametrizado de la sección 4.3. Este método de generación no es neuronal, por lo que no recibe una señal que le permita ajustar los problemas. Como solución para crear un currículo dinámico se ha diseñado un proceso iterativo que ajusta la dificultad de forma heurística. En cada iteración, se generan 70 problemas del

dominio *Blocksworld* de un tamaño determinado y se añaden 30 problemas de iteraciones anteriores, cada 5 iteraciones se comprueba el rendimiento del agente en dicho conjunto. Si supera cierto umbral de éxito, se aumenta la dificultad incrementando el tamaño de los futuros problemas generados.

De la misma forma que el método *baseline* anterior, se reaprovecha la implementación del agente de RL del método propuesto. Para el entrenamiento se emplea el algoritmo PPO. La configuración de hiperparámetros del agente es la misma que la del método principal (véase Tabla 5.2), pero añadiendo los elementos de la Tabla 5.4. La arquitectura de la red se implementa mediante NLMs, la configuración específica es idéntica a la del método principal también, y se puede consultar en la Tabla 5.1.

Sistema ACG <i>baseline</i> (extensión Tabla 5.2)		
Hiperparámetro	Símbolo	Valor
Umbral de progresión	τ	0.8
Tamaño del <i>buffer replay</i>	$ \mathcal{B} $	3000

Tabla 5.4: Extensión de hiperparámetros de la Tabla 5.2 para el método *baseline* de ACG con generador parametrizado.

5.1.8. Experimentos realizados

Para estudiar el funcionamiento y la viabilidad del método propuesto se han diseñado tres experimentos. En la Tabla 5.5 se pueden consultar los detalles sobre cada uno de ellos.

5.2. Resultados experimentales

En esta sección se presentan los experimentos realizados junto con una descripción de los resultados obtenidos.

5.2.1. Experimento 1: ajuste de hiperparámetros

Durante el desarrollo del modelo se han encontrado grandes dificultades para el ajuste de los hiperparámetros. Por un lado, para una misma configuración, diferentes semillas pueden dar lugar a resultados muy dispares. Por otro lado, se ha detectado una gran variabilidad de los resultados ante pequeños cambios en los hiperparámetros. Esto ha llevado a considerar que el modelo puede ser algo inestable y presentar una alta sensibilidad a configuraciones distintas. Estas propiedades presentan un alto interés de investigación, ya que implican la necesidad de un esfuerzo importante para

Experimento	Detalles
5.2.1 Ajuste de hiperparámetros	Se estudia la sensibilidad del método propuesto a cambios en tres hiperparámetros de NeSIG: umbral de diversidad, <i>learning rate</i> y épocas de PPO. En las pruebas, se evalúa el impacto de cada uno sobre el <i>success rate</i> . El objetivo es determinar si el modelo es sensible a pequeños cambios en la configuración de los hiperparámetros. Además, el proceso permite seleccionar la configuración final empleada en el resto de experimentos.
5.2.2 Estudio de ablación	Se compara el rendimiento y la eficacia del método propuesto respecto a dos variantes con capacidades reducidas (véase subsección 5.1.6). En las pruebas, se evalúa el impacto de cada una sobre el <i>success rate</i> y la estabilidad. El objetivo es determinar el aporte que tiene cada módulo del modelo al rendimiento global. Además, también se pretende estudiar el problema que presenta NeSIG al generar instancias triviales ya resueltas.
5.2.3 Comparativa de ACG basado en NeSIG con métodos <i>baseline</i>	Se compara el rendimiento y la eficacia del método propuesto respecto a dos <i>baselines</i> de menor complejidad (véase subsección 5.1.7). En las pruebas, se evalúa el impacto de cada uno sobre el <i>success rate</i> , el AUC, el coste computacional y la estabilidad. El objetivo es concluir si la mejora que aporta la propuesta justifica el aumento de complejidad.

Tabla 5.5: Resumen de los tres experimentos diseñados para evaluar el método propuesto, indicando el objetivo y las métricas consideradas en cada caso.

encontrar un ajuste que funcione correctamente con cada dominio. Por estos motivos, se ha decidido diseñar un experimento que permita evaluar los resultados de variar diferentes hiperparámetros de forma aislada. Además, se aprovecha para ajustar de forma más fina el modelo.

La configuración de la red se puede consultar en la subsección 5.1.5. No obstante, en este experimento los valores de la Tabla 5.3 para la diversidad y las épocas, son 0.1 y 10 respectivamente. Estos eran los valores originales, que se actualizaron posteriormente a partir de los resultados de este experimento, al identificarse una configuración mejor.

Para las pruebas, se toman tres hiperparámetros de la Tabla 5.3 y se realizan ejecuciones variando cada uno de ellos por separado. El primero es el umbral de diversidad, que controla la diversidad mínima que deben tener los problemas generados entre sí para no ser penalizados. El segundo

es el *learning rate*, que controla la velocidad con la que NeSIG actualiza su política. El tercero es el número de épocas que usa PPO. Por cada prueba se han realizado 5 ejecuciones y se ha obtenido la media, la desviación típica y el valor máximo del porcentaje de éxito sobre el conjunto de evaluación. En cada ejecución se realizan 300 iteraciones de preentrenamiento con señal de dificultad constante y 500 iteraciones de entrenamiento con señal de recompensa del agente. Estas cantidades de iteraciones se han determinado experimentalmente como suficientes para verificar la tendencia del modelo y su desempeño.

Los resultados sobre el umbral de diversidad se pueden observar en la Tabla 5.6. El mejor resultado se obtiene con 0.02, pero no existe una diferencia elevada entre resultados: la mayoría de medias oscilan entre el 40 % y el 60 % de éxito. Tampoco se observa ninguna tendencia creciente o decreciente con el aumento del umbral. Destaca la elevada desviación típica de las ejecuciones, siendo en el mejor de los casos 0.1569.

Umbral de diversidad	<i>Success rate</i>		
	Mean	Std	Max
0.010	0.5507	0.2525	0.7833
0.020	0.6113	0.1569	0.8167
0.030	0.5487	0.2133	0.8300
0.040	0.4960	0.2528	0.7567
0.050	0.4253	0.1599	0.6333
0.075	0.4253	0.1599	0.6900
0.100	0.4573	0.1913	0.7167
0.200	0.5247	0.1967	0.8500
0.300	0.3607	0.1923	0.5900
0.500	0.4233	0.2297	0.6667

Tabla 5.6: Resultados del ajuste del método principal variando la diversidad. Se observa que el mejor promedio y dispersión se obtiene con 0.02, siendo el único que supera el 60 % de *success rate*. No se aprecia ninguna tendencia clara con el aumento del umbral.

Los resultados sobre el *learning rate* se pueden observar en la Tabla 5.7. Las medias obtenidas son similares entre sí, aunque con una desviación típica de nuevo elevada en todos los casos. Se selecciona 1×10^{-3} para las futuras pruebas, ya que, pese a no presentar la mejor media, ofrece un mejor equilibrio entre rendimiento y estabilidad frente a 5×10^{-4} , que muestra una dispersión excesiva.

Los resultados sobre el número de épocas de PPO se pueden observar en la Tabla 5.8. En este caso, con el valor de 12 se obtiene una media que supera por mucho al resto y una dispersión también menor. La dispersión vuelve a ser elevada también en esta prueba, alcanzando el máximo de 0.2962.

<i>Learning rate</i>	<i>Success rate</i>		
	Mean	Std	Max
5×10^{-4}	0.3947	0.2847	0.7933
1×10^{-3}	0.3713	0.2335	0.6900
5×10^{-3}	0.3140	0.1230	0.5367

Tabla 5.7: Resultados del ajuste del método principal variando el *learning rate*. Se observa que 5×10^{-4} obtiene la mejor media, pero presenta una dispersión bastante elevada. Por otro lado, 1×10^{-3} tiene una media similar con mejor dispersión, siendo este el valor finalmente seleccionado.

Épocas de PPO	<i>Success rate</i>		
	Mean	Std	Max
3	0.4178	0.2962	0.8167
6	0.3674	0.2433	0.7567
9	0.3856	0.2537	0.8133
12	0.5826	0.1715	0.7733

Tabla 5.8: Resultados del ajuste del método principal variando el número de épocas de PPO. Destaca el valor 12, con una media y una desviación típica mejor que las demás. No se aprecia ninguna tendencia clara con el aumento de las épocas y la desviación típica es alta.

En las Tablas 5.6, 5.7 y 5.8, se ha observado mediante la desviación típica de las distintas semillas (alrededor de un 0.2 de media) que existe una alta variabilidad de resultados, independientemente del hiperparámetro modificado. Además, ni el umbral de diversidad ni el número de épocas de PPO muestran una tendencia clara o monótona sobre el porcentaje de éxito.

Por otro lado, en la Tabla 5.9 se ha recogido la media y desviación típica de los valores obtenidos en cada prueba. Mediante la desviación, se puede resumir cómo de alejados están los resultados al variar el valor del hiperparámetro. La dispersión más elevada alcanza un valor de 0.1 aproximadamente, la mitad de la desviación observada con respecto a la semilla elegida (0.2 aproximadamente). En conjunto, estos resultados ponen de manifiesto que, más allá de la elección puntual de valores óptimos para cada hiperparámetro, el propio proceso de entrenamiento presenta una variabilidad considerable dependiendo de su inicialización aleatoria.

5.2.2. Experimento 2: estudio de ablación

Durante el desarrollo del método propuesto se han encontrado muchas dificultades para ajustar y estabilizar el entrenamiento. Por estos motivos se ha planteado un estudio de ablación, que consiste en modificar el método

Hiperparámetro	<i>Success rate</i> (Mean $\pm$ Std)
Umbral de diversidad	0.482 ± 0.077
<i>Learning rate</i>	0.360 ± 0.042
Épocas de PPO	0.438 ± 0.098

Tabla 5.9: Media y desviación típica del *success rate* promedio obtenido para las distintas configuraciones probadas de cada hiperparámetro. Una mayor desviación típica indica una mayor sensibilidad del modelo a cambios en dicho hiperparámetro.

mediante la eliminación de algún módulo del mismo para determinar su impacto sobre el rendimiento global del sistema.

Con este estudio se desea cuantificar la mejora que introduce el ajuste iterativo de NeSIG, en contraste con usar NeSIG sin dicho ajuste, es decir, únicamente con el preentrenamiento. Además, también se busca descubrir cuánta complejidad e inestabilidad se introduce.

Por otro lado, a lo largo del desarrollo del sistema se ha encontrado el problema de que NeSIG puede generar problemas triviales (resueltos en el paso 0). En la Figura 5.2 se puede observar un ejemplo. En la tesis [69] estudian el uso de NeSIG como generador de instancias de problemas para el entrenamiento de una política. En este caso, no usan un proceso iterativo, sino que entrenan NeSIG solo al inicio. Durante sus pruebas, también se encuentran con el problema de la generación de instancias triviales. Para solucionarlo, aplican un preentrenamiento de 50 iteraciones. En el caso de este trabajo, ya se usa el preentrenamiento por lo que se desea estudiar si un método sin el proceso iterativo, como el de la tesis, elimina el problema de las instancias triviales.

Para el experimento se han diseñado dos ablaciones, una con preentrenamiento sin ajuste (ablación 1) y otra con preentrenamiento y una iteración de ajuste (ablación 2). Se pueden consultar más detalles sobre las ablaciones en la subsección 5.1.6. La arquitectura y los hiperparámetros se pueden consultar en la subsección anterior 5.1.5. Debido a la alta variabilidad en los resultados, para este experimento se realizan 10 ejecuciones por cada método variando la semilla con 500 iteraciones de entrenamiento, de esta forma se aseguran resultados representativos. Para cada prueba se ha obtenido el *success rate*, métricas para la estabilidad (*max drawdown*, *downside deviation* y *regression rate*) y el número de problemas triviales de promedio durante cada entrenamiento. Para cada medida se obtiene la media y la desviación típica, con el máximo incluido en algunos casos. Para más detalles sobre las métricas se puede consultar la subsección 5.1.4.

En la Tabla 5.10 se observa la comparación del *success rate* de los diferentes métodos. La propuesta principal con el ajuste iterativo es la que

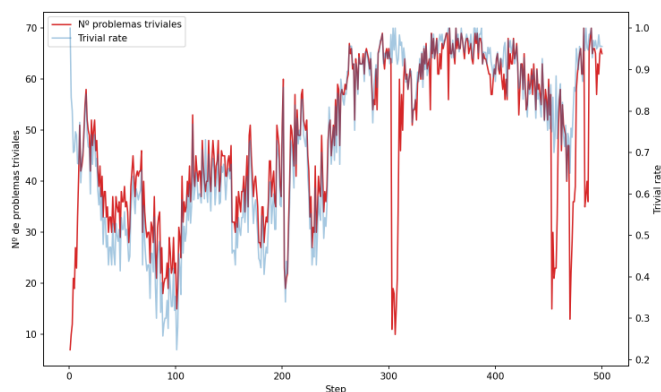


Figura 5.2: Ejemplo de una instancia de entrenamiento en la que se visualiza el problema de generación de instancias triviales de NeSIG. En el eje izquierdo, se representa el número total de problemas triviales sobre el total de setenta que se generan. En el derecho, se representa el *ratio*, que refleja el porcentaje de problemas triviales sobre los problemas consistentes, es decir, aquellos que son válidos. En el gráfico se aprecia un elevado número de oscilaciones, pero con una presencia alta de problemas triviales durante la mayoría de iteraciones.

mejores valores consigue con 62.58 % de éxito de media, esto supone 30 puntos porcentuales más que las ablaciones aproximadamente. La gran diferencia muestra que la adición del método iterativo aporta una mejora al rendimiento global del sistema. Asimismo, la ablación 2 no presenta ninguna mejora en cuanto al promedio sobre la ablación 1, lo que parece señalar que el ajuste inicial no aporta una mejora sustancial. Estos resultados sugieren que a NeSIG no le basta con un ajuste inicial, sino que es necesario el proceso iterativo continuo para observar una mejora en el rendimiento.

Sin embargo, a pesar de un peor rendimiento general, la ablación 2 consigue obtener un máximo de éxito de 85 % aproximadamente, muy cercano al logrado por el método completo. Para analizar mejor este fenómeno, se ha dibujado un gráfico de cajas para ver la distribución de los resultados obtenidos (véase Figura 5.3). En esta Figura, se puede observar que el máximo logrado por la ablación 2 parece corresponder a un posible valor atípico, posiblemente asociado a una inicialización especialmente favorable. Ignorando dicha instancia, las ablaciones 1 y 2 son muy similares, con una dispersión parecida, lo que vuelve a reforzar la hipótesis de que el ajuste inicial no aporta mucha mejora al rendimiento. Por último, el método principal vuelve a destacar con una mayor media y una dispersión bastante alta. Esto indica que, aunque el método completo obtiene el mejor rendimiento medio, todavía existe una variabilidad considerable entre ejecuciones, probablemente debida a la complejidad del entrenamiento conjunto entre NeSIG y el agente.

Método	<i>Success rate</i>		
	Mean	Std	Max
Ablación 1	0.3047	0.1348	0.5400
Ablación 2	0.3057	0.2186	0.8567
Método propuesto	0.6258	0.2186	0.8767

Tabla 5.10: Resultados del estudio de ablación donde se compara el *success rate* de las ablaciones y el método principal. Para cada método se representa el *success rate* promedio de sus ejecuciones junto con la desviación típica y el máximo. Se observa que la propuesta principal alcanza un promedio de éxito muy superior al resto, ligeramente más del doble.

Método	Max Drawdown	Downside Deviation	Regression Rate
	Mean $\pm$ Std	Mean $\pm$ Std	Mean $\pm$ Std
Ablación 1	0.1493 $\pm$ 0.0953	0.0314 $\pm$ 0.0229	0.3720 $\pm$ 0.0739
Ablación 2	0.1350 $\pm$ 0.0970	0.0271 $\pm$ 0.0195	0.4120 $\pm$ 0.0595
Método propuesto	0.1909 $\pm$ 0.0854	0.0398 $\pm$ 0.0192	0.3200 $\pm$ 0.0700

Tabla 5.11: Resultados del estudio de ablación donde se compara la estabilidad de las ablaciones y el método principal. Se han calculado tres medidas de estabilidad con la curva de aprendizaje sobre el conjunto de evaluación (véase subsección 5.1.4). En la Tabla se muestra que el método principal obtiene valores más altos que las ablaciones en *max drawdown* y *downside deviation*, lo que indica mayores caídas de rendimiento y una menor estabilidad del aprendizaje, aunque presenta el menor *regression rate*, indicando una menor frecuencia de regresiones.

En la Tabla 5.11 se observan los resultados obtenidos para la estabilidad de los tres métodos. Los resultados muestran que el método completo presenta el mayor *maximum drawdown* de los tres, lo que indica pérdidas de rendimiento más pronunciadas tras alcanzar nuevos máximos, y también el mayor *downside deviation*, reflejando que las caídas son menos consistentes. La ablación 2 es la que mejor comportamiento presenta en ambas métricas, seguida de cerca por la ablación 1. En cuanto al *regression rate*, el método completo obtiene el valor más bajo de los tres, ligeramente inferior al de la ablación 1 e inferior al de la ablación 2, lo que sugiere que, aunque sus caídas de rendimiento son más severas cuando ocurren, la frecuencia con la que se producen regresiones es menor. En consecuencia, el método completo presenta una evolución con mayores oscilaciones puntuales, pero con regresiones menos frecuentes que las ablaciones.

Se esperaba, a raíz de los experimentos de la tesis [69], que los métodos no iterativos (ablaciones) redujeran el problema de la generación de instancias triviales. En contra de lo esperado, se observa en la Tabla 5.12 que las ablaciones presentan valores muy elevados (entre el 69 % y el 74 % de los

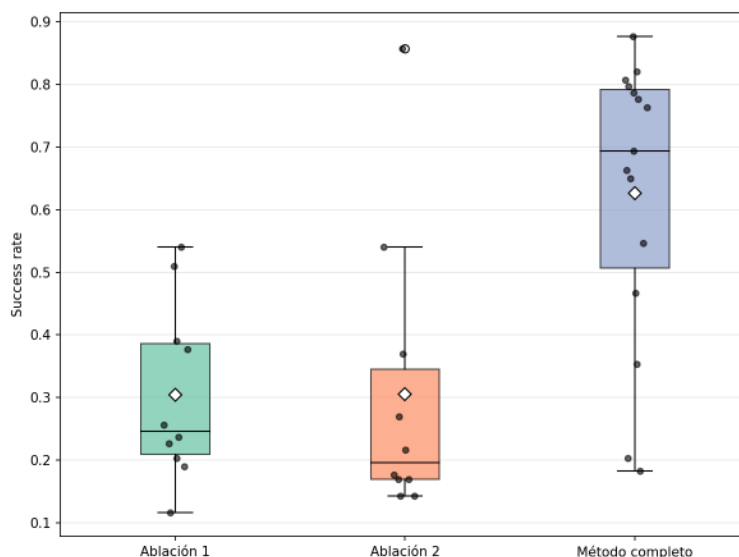


Figura 5.3: Gráfico de cajas de los resultados del *success rate*. Se observa a rasgos generales que el método principal tiene una mayor media y dispersión que los otros métodos. Sin embargo, la ablación 2 presenta un valor muy elevado que parece algo anómalo. Ignorando este caso concreto, los resultados de la ablación 1 y 2 son bastante similares y no presentan mucha diferencia.

problemas generados), al igual que el método propuesto. Esto señala que el problema puede no estar tan relacionado con el proceso iterativo como con el propio funcionamiento de NeSIG, por lo que posiblemente sea necesario ajustar la configuración del generador para mitigarlo.

5.2.3. Experimento 3: comparativa de *automated curriculum generation* basado en NeSIG con métodos *baseline*

En este experimento se realiza la comparación final del método propuesto con el objetivo de evaluar su rendimiento frente a alternativas de menor complejidad. En particular, se pretende determinar si el incremento en la complejidad del enfoque se traduce en una mejora del desempeño que justifique el coste computacional adicional asociado. Para ello, se analizan tanto las diferencias en el rendimiento alcanzado como el coste de ejecución de cada método, con el fin de valorar la viabilidad de la propuesta o si, por el contrario, resulta más conveniente emplear métodos más sencillos.

En este experimento se prueba el método principal frente a los dos métodos *baselines*. El primero consiste en un agente sin ningún sistema curricular

Método	Promedio del ratio de problemas triviales		
	Mean	Std	Max
Ablación 1	68.79 %	10.45 %	92.44 %
Ablación 2	69.84 %	16.10 %	97.62 %
Método propuesto	73.94 %	18.64 %	98.57 %

Tabla 5.12: Resultados del estudio de ablación donde se compara el número de problemas triviales de las ablaciones y el método principal. Se considera que un problema es trivial si su estado inicial ya satisface el objetivo. Para cada ejecución se ha obtenido el promedio del ratio de problemas triviales sobre el número de problemas consistentes de cada iteración. Aunque existen diferencias en los valores medios, estas no parecen suficientemente marcadas como para evidenciar un comportamiento claramente distinto entre métodos (5 % de diferencia aproximadamente entre el mejor y peor caso). Los tres métodos presentan el problema de la generación de instancias triviales, lo que hace pensar que este problema no es resultado del ciclo iterativo sino del propio método generativo NeSIG.

(*baseline 1*), mientras que el segundo corresponde a un sistema ACG en el que se sustituye NeSIG por un generador parametrizado (*baseline 2*). Los detalles, arquitectura e hiperparámetros se pueden consultar en la subsección 5.1.7. La arquitectura y los hiperparámetros del método principal se pueden consultar en la subsección anterior 5.1.5. Las ejecuciones duran un mínimo de 100 iteraciones y se permite que continúen hasta un máximo de 500 si no logran alcanzar el 90 % de éxito. Se sigue esta metodología porque no solo interesa el mayor rendimiento en *success rate*, sino también la velocidad con la que se alcanza, por lo que no es esencial limitar el número de iteraciones de forma estricta. Cada método se ejecuta 10 veces y se registran las siguientes métricas: *success rate*, AUC, estabilidad y tiempo de ejecución. Como las condiciones de ejecución son las mismas que las del experimento 2 y debido al alto coste en tiempo de volver a repetir las pruebas, se reutilizan los resultados para el método propuesto.

En la Tabla 5.13 se puede observar la comparación del *success rate* de los diferentes métodos. El método propuesto obtiene un promedio aproximadamente 30 puntos porcentuales inferior al de los *baselines*. Además, la desviación típica pone de manifiesto una menor consistencia en el rendimiento de la propuesta. Incluso el máximo resultado obtenido por el método propuesto no alcanza ninguno de los promedios de los dos *baselines*.

Por otro lado, el *baseline 1* obtiene resultados superiores a los del *baseline 2*, a pesar de no hacer uso de CL. La diferencia es pequeña y ambos alcanzan un *success rate* cercano al 100 %. Estos resultados sugieren que el dominio a resolver puede ser demasiado sencillo como para que se aprecie la mejora que

Método	<i>Success rate</i>		
	Mean	Std	Max
<i>Baseline 1</i>	0.9853	0.0091	0.9933
<i>Baseline 2</i>	0.9533	0.0279	1.0000
Método propuesto	0.6258	0.2186	0.8767

Tabla 5.13: Resultados del estudio comparativo donde se compara el *success rate* de los métodos *baseline* y el método principal. Para cada método se representa el *success rate* promedio de sus ejecuciones junto con la desviación típica y el máximo. Se observa que ambos *baselines* superan ampliamente al método propuesto. Incluso el mejor resultado alcanzado por la propuesta no llega a igualar la media de los *baselines*.

aporta un currículo progresivo sobre el rendimiento. El agente parece que es capaz de aprender a resolver *Blocksworld* a partir de cualquier conjunto de problemas lo suficientemente representativo y diverso, como por ejemplo, un conjunto aleatorio de problemas de distintas dificultades.

Método	AUC		
	Mean	Std	Max
<i>Baseline 1</i>	0.7863	0.0386	0.8359
<i>Baseline 2</i>	0.8356	0.0177	0.8609
Método propuesto	0.4344	0.1658	0.6611

Tabla 5.14: Resultados del estudio comparativo donde se compara el AUC de los métodos *baseline* y el método principal. Para cada método se representa el AUC promedio de sus ejecuciones junto con la desviación típica y el máximo. Se puede observar que el *baseline 2* es el que mayor velocidad de aprendizaje presenta, mientras que el método propuesto se queda muy atrás, obteniendo una velocidad de aprendizaje considerablemente más lenta que los otros dos métodos.

En la Tabla 5.14 se observan los resultados obtenidos para el AUC de los tres métodos. En conjunto, los resultados muestran que el método principal se queda por detrás de los *baselines* en cuanto a velocidad de aprendizaje. En conjunto con los resultados de la tabla anterior, el método principal no solo obtiene un peor rendimiento global en *success rate*, sino que también requiere más iteraciones para alcanzarlo. Dado que una de las principales ventajas del aprendizaje curricular (CL) es acelerar la convergencia del entrenamiento, estos resultados plantean dudas sobre las causas del bajo rendimiento obtenido por la propuesta.

Por otro lado, el *baseline 2* presenta un AUC ligeramente superior al del *baseline 1*, lo que sugiere una velocidad de aprendizaje algo mayor. Aunque la diferencia es moderada, este resultado apunta a que el uso de un currículo sí

parece favorecer la rapidez con la que el agente alcanza un buen rendimiento.

Método	Tiempo de ejecución (horas)		
	Mean (H)	Std	Max
<i>Baseline 1</i>	4.47	0.14	4.68
<i>Baseline 2</i>	6.17	0.60	6.71
Método propuesto	11.79	2.43	14.39

Tabla 5.15: Resultados del estudio comparativo donde se compara el tiempo de ejecución de los métodos *baseline* y el método principal. Para cada método se representa el tiempo de ejecución promedio de sus ejecuciones junto con la desviación típica y el máximo. Se observa que el coste computacional es mayor cuanto más complejo es el método ejecutado. El *baseline 2* tarda de media unas dos horas más que el *baseline 1* aproximadamente, mientras que el método propuesto tarda casi el doble que el *baseline 2*. También, se aprecia que con la complejidad aumenta la variabilidad de los tiempos, el máximo del método propuesto toma 2.6 horas más que la media, mientras que en el *baseline 1* la diferencia es de apenas 0.21 horas.

Los resultados de la Tabla 5.15 parecen reflejar una relación proporcional entre la complejidad del método y su coste computacional. El *baseline 1* es la variante más simple y la más rápida además de ser la más estable entre ejecuciones. El *baseline 2*, que presenta mayor complejidad, obtiene un tiempo medio y desviación superiores. Por último, el método propuesto que es el más complejo presenta un salto considerable: su tiempo de ejecución medio es 2.6 veces mayor que el de *baseline 1* y 1.9 veces mayor que el de *baseline 2*. Este aumento de coste no se limita a la media, sino que también se refleja en una mayor variabilidad entre semillas, con una desviación típica de 2.43 y un máximo de 14.39. De este patrón, se podría elaborar la hipótesis de que la complejidad del método propuesto incrementa el coste computacional promedio, pero además también lo hace menos predecible. Sin embargo, no se pudo confirmar, ya que no se ha podido controlar qué nodos de GPU usa cada ejecución, por lo que serían necesarios experimentos en un entorno más controlado para validar dicha hipótesis. La investigación de esta cuestión puede ser relevante, ya que plantea dificultades a la hora de planificar los recursos computacionales necesarios.

En la Tabla 5.16 se observan los resultados obtenidos para las tres métricas de estabilidad consideradas. El *baseline 1* es el que obtiene los resultados más positivos, lo que indica una curva de aprendizaje considerablemente más estable frente a los demás métodos evaluados. El *baseline 2* muestra un comportamiento ligeramente más inestable, probablemente debido al aumento de complejidad añadido por el sistema ACG. El método propuesto es el que presenta los valores más elevados en todos los casos, alcanzando un *max*

Método	Max Drawdown Mean $\pm$ Std	Downside Deviation Mean $\pm$ Std	Regression Rate Mean $\pm$ Std
<i>Baseline 1</i>	0.0070 $\pm$ 0.0062	0.0022 $\pm$ 0.0020	0.0778 $\pm$ 0.0509
<i>Baseline 2</i>	0.0227 $\pm$ 0.0225	0.0059 $\pm$ 0.0044	0.2698 $\pm$ 0.1093
Método propuesto	0.1909 $\pm$ 0.0854	0.0398 $\pm$ 0.0192	0.3200 $\pm$ 0.0700

Tabla 5.16: Resultados del estudio comparativo donde se compara la estabilidad de los métodos *baseline* y el método principal. Se han calculado tres medidas de estabilidad con la curva de aprendizaje sobre el conjunto de evaluación. Puede verse que el *baseline 1* es el más estable, este resultado es consistente con la hipótesis de que la complejidad adicional del sistema ACG introduce nuevas fuentes de inestabilidad. En cambio, el más inestable parece ser el método propuesto, esto podría deberse a que requiere entrenamiento de una red adicional para la generación de instancias.

drawdown de 0.1909, una *downside deviation* de 0.0398 y una *regression rate* de 0.3200. Estas cifras indican que presenta las caídas más intensas y frecuentes en el rendimiento. La inestabilidad puede atribuirse a la complejidad adicional introducida por el uso de NeSIG, lo cual implica un proceso de entrenamiento conjunto entre dos redes.

La Figura 5.4 permite observar la variabilidad entre las diferentes ejecuciones de cada método. Los dos *baselines* tienen una variabilidad muy reducida y un *success rate* muy alto de media, lo que indica un comportamiento consistente entre semillas. En cambio, el método propuesto presenta una dispersión considerablemente mayor, con una diferencia notable entre las distintas ejecuciones. Además, en el método propuesto se observa que la media se encuentra por debajo de la mediana, lo que sugiere que las ejecuciones con peor rendimiento afectan de forma significativa al promedio. Este comportamiento, junto con la elevada desviación típica observada en la Tabla 5.13, refleja que el método propuesto no solo parece obtener un menor rendimiento medio, sino que también presenta una mayor variabilidad entre ejecuciones.

En la Figura 5.5 se compara el rendimiento del método propuesto frente a los dos *baselines* en términos de *success rate* sobre el conjunto de evaluación a lo largo del entrenamiento. Se observa que ambos *baselines* alcanzan un *success rate* cercano al 100 % en menos de 50 iteraciones, siendo el *baseline 2* ligeramente más rápido que el *baseline 1*, lo cual se alinea con los resultados obtenidos en la Tabla 5.14. En cambio, el método propuesto muestra una curva de aprendizaje considerablemente más lenta e inestable. Presenta una tendencia creciente a lo largo de las 500 iteraciones, pero con oscilaciones sobre todo presentes a partir de la iteración 300. Cierta inestabilidad es esperable debido a la complejidad del modelo, pero en comparativa con los otros dos métodos es bastante elevada. Además, el problema parece ser

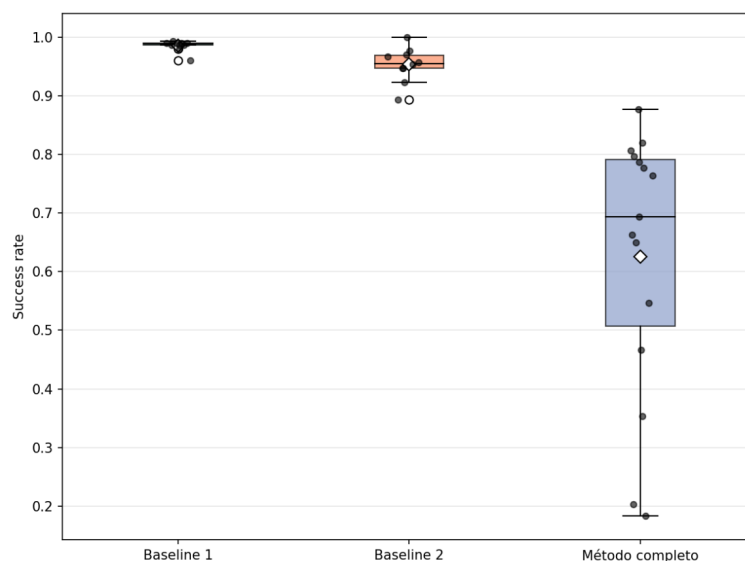


Figura 5.4: Gráfico de cajas del *success rate* del método propuesto frente a los dos *baselines*. Los *baselines* muestran poca dispersión, con valores cercanos al 100 % de éxito, en cambio el método propuesto presenta una dispersión considerablemente mayor, con ejecuciones que van desde apenas un 18 % hasta un 88 % de éxito. El rombo blanco indica la media de cada método; en el caso del método propuesto, esta se sitúa notablemente por debajo de la mediana. Los puntos blancos señalan casos con posibilidad de ser *outliers*.

relativamente sencillo de aprender para el agente, ya que el agente obtiene muy buen rendimiento incluso aprendiendo de la distribución aleatoria de problemas, por lo que cabría esperar que el método propuesto obtuviese una convergencia mucho más rápida.

Como se observa en la Figura 5.6, la tasa de éxito media del método propuesto desciende de forma consistente a medida que aumenta la dificultad del problema. Para instancias pequeñas (3-5 bloques), el modelo alcanza el 80 %, mientras que para instancias grandes (25-30 bloques) el rendimiento cae hasta el 30 % y el 40 %. Esta tendencia decreciente es esperable, dado que un mayor número de bloques implica un espacio de estados más amplio y planes de solución más largos, lo que dificulta tanto la exploración como la generalización del agente.

5.3. Discusión

A continuación se recopilan y discuten de forma conjunta los resultados obtenidos en los tres experimentos, se discute el problema de la generación de

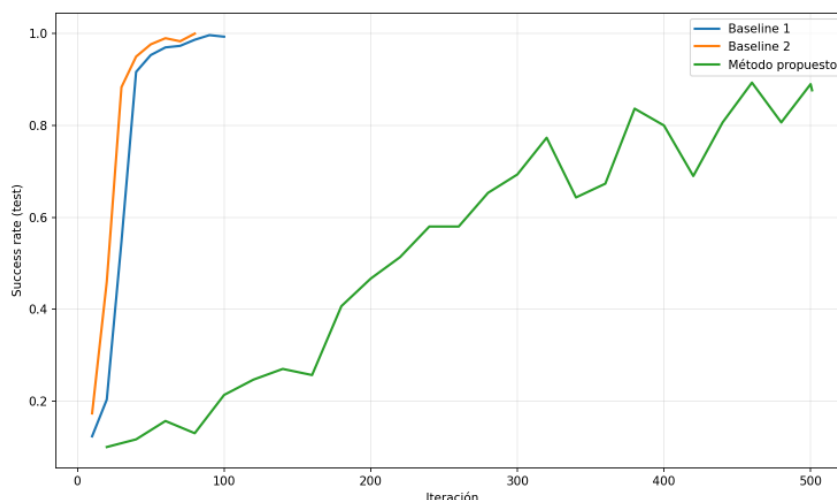


Figura 5.5: Evolución de la ejecución con mayor *success rate* en el conjunto de evaluación de cada método. Los *baselines* convergen muy rápidamente, alcanzando un *success rate* cercano al 100 % alrededor de las 50 iteraciones. El método propuesto, en cambio, necesita un mayor número de pasos para converger, mostrando además una curva más irregular, con oscilaciones notables a lo largo del entrenamiento. Aun así, mantiene una tendencia ascendente y alcanza un *success rate* cercano al 90 % en torno a la iteración 500, sin señales claras de haber llegado a un máximo de rendimiento.

instancias triviales y se discuten los motivos del mal desempeño del método propuesto frente a los *baselines*.

5.3.1. Resumen de resultados

A continuación se recogen los resultados más importantes obtenidos durante los experimentos. La Tabla 5.17 recopila los resultados obtenidos para el *success rate*, *max drawdown*, *downside deviation* y *regression rate* de los métodos probados en los experimentos 2 y 3. La Tabla 5.18 recopila los resultados del AUC y el tiempo de ejecución del experimento 3.

De estos resultados se extraen varias conclusiones relevantes. El experimento 1 mostró que el modelo tiene una gran variabilidad de resultados, con una desviación típica en torno al 20 % del *success rate* independientemente de la configuración probada, lo que anticipaba la inestabilidad que se observa en los experimentos realizados posteriormente.

En el experimento 2 se realizó un estudio de ablación que mostró que el ajuste iterativo de NeSIG aporta una mejora sustancial de rendimiento frente a las ablaciones. Sin embargo, también se apreció un aumento de

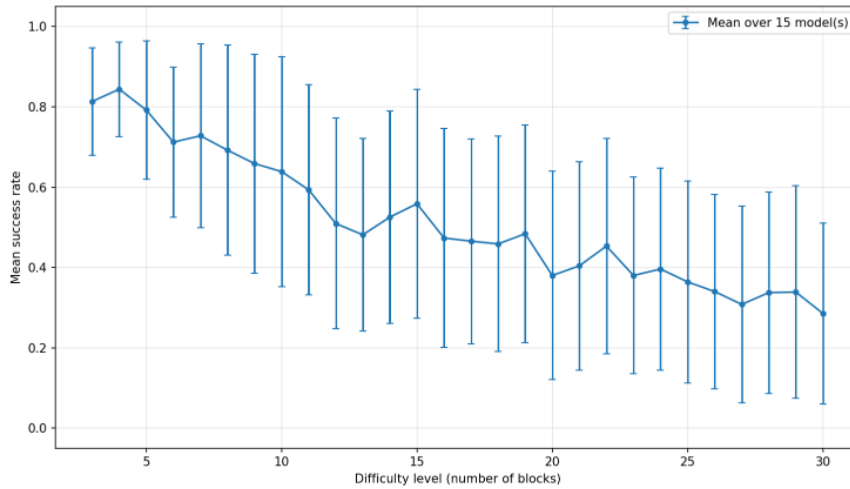


Figura 5.6: Tasa de éxito media del método propuesto en función del nivel de dificultad, medido como el número de bloques del problema de *blocksworld*. Las barras de error representan la desviación típica entre modelos para cada nivel. Cada punto se calcula sobre 50 problemas generados aleatoriamente para ese nivel de dificultad. Se observa que el modelo tiene dificultad para resolver los problemas de mayor tamaño.

inestabilidad en el aprendizaje. Además, con este experimento, se pretendía estudiar el problema de generación de instancias triviales de NeSIG y se comprobó que persiste de forma similar en las tres variantes. Esto sugiere que el problema no está relacionado tanto con el ciclo iterativo, sino con el funcionamiento interno de NeSIG.

El experimento 3 reveló que pese al buen rendimiento frente a las ablaciones, el método propuesto no consigue superar a los *baselines* en ninguna de las métricas evaluadas. En concreto, no logra superar ni acercarse al *baseline 1* que entrena con un conjunto de problemas aleatorios, lo que lleva a pensar que NeSIG no funciona correctamente. Esto puede deberse al problema de la generación de instancias triviales, que impide que NeSIG genere suficientes problemas útiles para el agente.

Por otro lado, los dos métodos *baselines* parecen resolver sin muchas complicaciones el dominio *Blocksworld*, lo que impide apreciar la mejora que pueda llegar a obtener un sistema ACG sobre métodos convencionales como el del *baseline 1*. Esto sugiere que probar los métodos en un dominio más complejo permitiría notar una diferencia más clara de rendimiento con el sistema ACG.

En conjunto, estas conclusiones señalan dos cuestiones que merecen especial atención. En primer lugar, analizar el problema de la generación de instancias triviales y entender cuáles pueden ser las posibles causas. En

Método	<i>Success rate</i> Mean $\pm$ Std	Max draw. Mean $\pm$ Std	Downside dev. Mean $\pm$ Std	Regression rate Mean $\pm$ Std
Ablación 1	0.3047 $\pm$ 0.1348	0.1493 $\pm$ 0.0953	0.0314 $\pm$ 0.0229	0.3720 $\pm$ 0.0739
Ablación 2	0.3057 $\pm$ 0.2186	0.1350 $\pm$ 0.0970	0.0271 $\pm$ 0.0195	0.4120 $\pm$ 0.0595
<i>Baseline</i> 1	0.9853 $\pm$ 0.0091	0.0070 $\pm$ 0.0062	0.0022 $\pm$ 0.0020	0.0778 $\pm$ 0.0509
<i>Baseline</i> 2	0.9533 $\pm$ 0.0279	0.0227 $\pm$ 0.0225	0.0059 $\pm$ 0.0044	0.2698 $\pm$ 0.1093
Método propuesto	0.6258 $\pm$ 0.2186	0.1909 $\pm$ 0.0854	0.0398 $\pm$ 0.0192	0.3200 $\pm$ 0.0700

Tabla 5.17: Recopilación del *success rate* y las métricas de estabilidad (*max drawdown*, *downside deviation* y *regression rate*) de todos los métodos evaluados. El método propuesto mejora claramente a sus propias ablaciones en *success rate* a costa de una mayor inestabilidad, pero queda muy por detrás de ambos *baselines*.

Método	AUC			Tiempo de ejecución (horas)		
	Mean	Std	Max	Mean	Std	Max
<i>Baseline</i> 1	0.7863	0.0386	0.8359	4.47	0.14	4.68
<i>Baseline</i> 2	0.8356	0.0177	0.8609	6.17	0.60	6.71
Método propuesto	0.4344	0.1658	0.6611	11.79	2.43	14.39

Tabla 5.18: Recopilación del AUC y el tiempo de ejecución del método propuesto comparados con los *baselines*. El método propuesto no solo obtiene el peor AUC, sino que además requiere más del doble de tiempo de ejecución que el *baseline* 2.

segundo lugar, estudiar la razón por la cual el método principal no logra superar a los *baselines*.

5.3.2. El problema de la generación de instancias triviales

La generación de problemas triviales por parte de NeSIG se ha ilustrado en la Figura 5.2 y en la Tabla 5.19. Se observa que un elevado número de los problemas generados son triviales. Este fenómeno puede perjudicar enormemente el entrenamiento, ya que si la señal de recompensa que recibe el agente es más pobre de lo esperado, el aprendizaje se ralentiza. Por estos motivos, es relevante buscar soluciones que puedan resolver este problema, ya que, de confirmarse esta hipótesis, podría explicar una parte relevante de la brecha de rendimiento observada frente a los *baselines* en el experimento 3.

Como se comentó en el experimento 2, la expectativa basada en los resultados de la tesis [69] era que eliminando el ciclo iterativo se reduciría el problema. No obstante, de forma contraria a estas expectativas, la generación de instancias triviales se ha mantenido incluso eliminando dicho ciclo. Los resultados muestran que el método propuesto presenta el promedio de problemas triviales más alto de los tres (73.94 %), pero los otros dos métodos obtienen resultados cercanos con un 68.79 % y un 69.84 % respectivamente.

Método	Promedio del ratio de problemas triviales		
	Mean	Std	Max
Ablación 1	68.79 %	10.45 %	92.44 %
Ablación 2	69.84 %	16.10 %	97.62 %
Método propuesto	73.94 %	18.64 %	98.57 %

Tabla 5.19: Recopilación de los resultados sobre el ratio de problemas triviales obtenido en el experimento 2. Para cada ejecución se ha obtenido el promedio del ratio de problemas triviales sobre el número de problemas consistentes de cada iteración. Las tres variantes presentan valores muy elevados, lo que sugiere que el fenómeno no es causado por el uso del proceso iterativo de entrenamiento del método principal.

Esto sugiere que el problema de la trivialidad no depende del proceso iterativo de ajuste.

Esto lleva a plantear que la causa del problema no reside en la estrategia de entrenamiento, sino en el propio mecanismo generativo de NeSIG, y en particular en cómo interactúan entre sí las señales de recompensa asociadas a la diversidad y a la dificultad. Como se observa en la señal de recompensa de NeSIG (Ecuación 4.2), ambos términos tienen un peso considerable en el resultado final, lo que hace que su equilibrio sea determinante en la calidad de los problemas generados. Sin embargo, durante el desarrollo del modelo se observó que ambos objetivos tienden a entrar en conflicto, de forma que mejorar uno suele producirse a costa de empeorar el otro. Por un lado, cuando se prioriza la diversidad, NeSIG tiende a generar problemas tanto sencillos como difíciles, lo que reduce la dificultad media del conjunto. Por otro lado, cuando el modelo aprende a generar problemas difíciles de forma consistente, tiende a converger hacia una única estrategia de generación, sacrificando así la diversidad.

Partiendo de esta discusión, una posible explicación a los resultados vistos en la Tabla 5.19 es que en las fases iniciales de cualquiera de los tres métodos, NeSIG tiende a generar problemas cercanos al estado inicial. Durante el preentrenamiento, la señal de dificultad es constante, por lo que a NeSIG le basta con introducir variaciones mínimas sobre dicho estado inicial para maximizar la diversidad, sin necesidad de generar problemas realmente complejos. Como consecuencia, los problemas generados resultan demasiado simples y el agente apenas recibe señal útil para aprender.

Esta hipótesis también explicaría por qué el método propuesto obtiene un rendimiento notablemente superior al de las ablaciones. Esto sería coherente con el hecho de que las ablaciones sigan generando problemas relativamente sencillos, dado que no poseen un mecanismo de ajuste tras el

preentrenamiento o, en el caso de la ablación 2, dicho mecanismo podría no ser lo suficientemente eficaz. En cambio, el método propuesto emplea la señal del agente durante el ciclo iterativo para adaptar progresivamente los problemas al nivel del agente. Esto permite que, con el tiempo, los problemas generados se vuelvan más complejos y útiles, lo que se traduce en una mejora del *success rate* final.

Para resolver el problema, una línea de trabajo prometedora sería explorar en mayor profundidad el diseño de la señal de recompensa de NeSIG. En particular, sería interesante estudiar si una penalización más explícita de problemas triviales, una señal de dificultad mejor coordinada con la diversidad o el uso de otros mecanismos de estabilización son buenas opciones para mejorar el sistema.

5.3.3. Por qué el método propuesto no supera a los *baselines*

Los resultados del experimento 3 muestran que el método propuesto queda por detrás de ambos *baselines*. Esto llama especialmente la atención teniendo en cuenta que el método propuesto parte de una base teórica sólida. Una primera idea que puede explicar este fenómeno es la propia inestabilidad del proceso de entrenamiento conjunto. En la Tabla 5.17 se observa que el método propuesto obtiene los peores valores en *max drawdown*, *downside deviation* y *regression rate* frente a los *baselines*. Esta inestabilidad es coherente con los resultados del experimento 1, donde se detectó que el entrenamiento presenta una alta sensibilidad a la inicialización aleatoria de los pesos, independientemente de la configuración utilizada.

Una de las principales causas de la inestabilidad puede ser la necesidad de entrenar conjuntamente dos redes, lo cual introduce una fuente adicional de complejidad. Por otro lado, el problema de generación de instancias triviales podría contribuir al peor rendimiento del método propuesto. Si una fracción considerable de los problemas que NeSIG genera para el agente ya están resueltos, la señal de recompensa del entrenamiento es más pobre que la que usan los otros métodos, que sí disponen del conjunto completo de problemas. Esto podría explicar, parcialmente, por qué el método propuesto necesita más iteraciones para converger y alcanza un *success rate* inferior al de los *baselines*.

Por último, la Tabla 5.18 muestra que el método también es computacionalmente más costoso que ambos *baselines*. Este aspecto es difícil de solucionar, ya que, como se comentaba antes, el entrenamiento de dos redes conjuntas supone de forma natural un mayor tiempo de ejecución e inestabilidad. No obstante, este mayor coste cuestiona la viabilidad del enfoque para un dominio de poca complejidad y motiva la necesidad de evaluarlo en dominios donde un sistema de ACG pueda suponer una ventaja.

Para mejorar los aspectos discutidos, se proponen varias posibles líneas de desarrollo. En primer lugar, como se comenta en el apartado anterior, se puede revisar la función de recompensa de NeSIG para lograr un mejor equilibrio entre diversidad y dificultad. En segundo lugar, en este trabajo se aplica el sistema de intercalación como estrategia para estabilizar el entrenamiento conjunto entre el agente y NeSIG, pero podrían ser necesarios más métodos con el objetivo de reducir la sensibilidad a la inicialización y a los hiperparámetros. Por último, una vez solucionados los problemas anteriores, sería conveniente buscar dominios más complejos donde un sistema curricular pueda aportar una mayor ventaja.

Capítulo 6

Conclusiones

ACG representa un paradigma importante en ML que puede reducir significativamente los costes de entrenamiento, acelerar la convergencia de los modelos y mejorar la capacidad de generalización en distintos tipos de problemas. En este TFG se ha explorado este campo proponiendo un enfoque dentro de ACG, con el objetivo de estudiar su aplicabilidad en entornos de planificación automática y servir como base para futuras investigaciones.

En primer lugar, se ha realizado un estudio en profundidad de la literatura relativa a ACL y ACG, abarcando desde los métodos clásicos basados en ordenación manual de currículos hasta enfoques más modernos basados en modelos generativos o adversariales. Este análisis muestra una tendencia progresiva hacia métodos de ACL, aunque todavía existen desafíos abiertos en términos de estabilidad y control de la dificultad. Dado este panorama, se considera que nuestro enfoque aporta una perspectiva nueva que no ha sido suficientemente explorada hasta ahora.

El desarrollo de la propuesta parte de la implementación de un sistema básico de RL. Esto incluye todos los elementos para que el agente aprenda, desde la capacidad de interpretar archivos PDDL hasta el diseño de la arquitectura de la red y la implementación del algoritmo de entrenamiento PPO. A partir de este sistema, se ha desarrollado el método propuesto, un sistema ACG basado en el modelo generativo neuro-simbólico, NeSIG. Tras verificar el correcto funcionamiento de todo el sistema, se han definido tres configuraciones experimentales con el objetivo de establecer una comparación de referencia. En primer lugar, un agente sin ACG; en segundo lugar, un sistema ACG basado en el generador parametrizado; y, en tercer lugar, el sistema propuesto basado en NeSIG.

Los resultados experimentales muestran que el método propuesto, a pesar de partir de una base teórica sólida y de mejorar claramente a sus propias ablaciones, no logra superar a los otros métodos en rendimiento, velocidad de

convergencia, estabilidad o coste computacional. Estas limitaciones podrían explicarse por la generación de instancias triviales por parte de NeSIG, que reduce la calidad efectiva de la señal de entrenamiento que recibe el agente.

En conjunto, estos resultados indican que el método propuesto se ha visto limitado por su propia complejidad. El entrenamiento conjunto de dos redes es un reto complicado que introduce muchas variables y fuentes de inestabilidad. Debido a estas dificultades no se ha logrado dotar a la red generativa de la flexibilidad y estabilidad necesarias para construir un sistema ACG competitivo. No obstante, se ha analizado en profundidad el sistema y sus limitaciones, lo que establece las bases para que futuras líneas de investigación puedan encontrar mejores soluciones a estos problemas.

Como trabajo futuro, se podría investigar en mayor profundidad la interacción entre las señales de recompensa de NeSIG. Concretamente, el equilibrio entre la señal de diversidad y dificultad, con el objetivo de analizar su impacto en la inestabilidad del sistema y su relación con el problema de la generación de instancias triviales. Una posible dirección sería explorar alteraciones de la señal de recompensa o mecanismos que penalicen activamente la trivialidad.

De forma más general, sería muy valioso estudiar más a fondo cómo el currículo generado afecta realmente al aprendizaje del agente, estudiando no solo métricas clásicas como el *success rate* o la velocidad de convergencia, sino también la evolución de las representaciones internas de la red durante el entrenamiento. Este análisis permitiría comprender mejor por qué ciertos currículos resultan más efectivos que otros y aportar evidencias empíricas que ayuden a diseñar métodos de ACG más fundamentados.

Para concluir, este proyecto me ha otorgado la oportunidad de aplicar muchos de los conocimientos adquiridos durante la carrera en Ingeniería Informática, especialmente en áreas como el ML para comprender los sistemas empleados, el desarrollo de software para diseñar e implementar el código e ingeniería de servidores para la gestión eficiente de las colas de ejecución en los servidores de la universidad. Además, esta experiencia me ha permitido familiarizarme con el proceso de desarrollo de un proyecto extenso y comprender mejor cómo se lleva a cabo una investigación rigurosa. Por tanto, se considera que se han cumplido los objetivos de la asignatura del TFG satisfactoriamente.

Bibliografía

- [1] Dario Amodei, Chris Hernandez, et al. AI and compute. <https://openai.com/research/ai-and-compute>, 2018.
- [2] McKinsey & Company, QuantumBlack. The state of AI. <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>, 2026.
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 1st edition, 2006.
- [4] Yaser S. Abu-Mostafa, Malik Magdon-Ismail, and Hsuan-Tien Lin. *Learning from Data*. AMLBook, 1st edition, 2012.
- [5] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 1st edition, 2022.
- [6] Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 1st edition, 2023.
- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.
- [8] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer Nature, 1st edition, 2024.
- [9] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.
- [10] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 1st edition, 2023.
- [11] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.
- [12] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 41–48, 2009.

- [13] Alex Graves, Marc G. Bellemare, Jacob Menick, Remi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. arXiv:1704.03003, 2017.
- [14] Petru Soviany, Radu Tudor Ionescu, Paolo Rota, and Nicu Sebe. Curriculum learning: A survey. *International Journal of Computer Vision*, 130:1526–1565, 2022.
- [15] Xin Wang, Yuwei Zhou, Hong Chen, and Wenwu Zhu. Curriculum learning for multimedia. Tutorial at the ACM International Conference on Multimedia (ACM MM), 2024.
- [16] Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep RL: A short survey. arXiv:2003.04664, 2020.
- [17] Carlos Núñez-Molina, Pablo Mesejo, and Juan Fernández-Olivares. NeSIG: A neuro-symbolic method for learning to generate planning problems. In *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI)*, pages 978–985, 2024.
- [18] Pascal Hitzler, Aaron Eberhart, Monireh Ebrahimi, Md Kamruzzaman Sarker, and Lu Zhou. Neuro-symbolic approaches in artificial intelligence. *National Science Review*, 9:nwac035, 2022.
- [19] Roger S. Pressman. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill, 7th edition, 2010.
- [20] Ian Sommerville. *Software Engineering*. Addison-Wesley, 9th edition, 2011.
- [21] Kent Beck et al. Manifesto for agile software development. <https://agilemanifesto.org/>, 2001.
- [22] Ken Schwaber and Jeff Sutherland. The scrum guide. <https://scrumguides.org/>, 2020.
- [23] John Slaney and Sylvie Thiébaux. Blocks world revisited. *Artificial Intelligence*, 125:119–153, 2001.
- [24] Universidad VIU. Informe de salarios en inteligencia artificial en españa 2026. <https://www.universidadviu.com/>, 2026.
- [25] Universidad Francisco de Vitoria (UFV). ¿cuánto gana un experto en inteligencia artificial en españa? <https://www.ufv.es>, 2026.
- [26] EUSCHOOL. Cuánto gana un experto en ia en españa y qué factores influyen. <https://www.euschool.es/cuanto-gana-un-experto-en-ia-espana/>, 2026.

- [27] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning and Acting*. Cambridge University Press, 1st edition, 2016.
- [28] Tom Bylander. The computational complexity of propositional STRIPS planning. *Artificial Intelligence*, 69:165–204, 1994.
- [29] Malte Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.
- [30] Jussi Rintanen. Computational complexity of automated planning and scheduling. Tutorial at the International Conference on Automated Planning and Scheduling (ICAPS), 2016.
- [31] Malik Ghallab, Craig Knoblock, David Wilkins, Anthony Barrett, Dave Christianson, Marc Friedman, Chung Kwok, Keith Golden, Scott Penberthy, David Smith, Ying Sun, and Daniel Weld. PDDL: The planning domain definition language. Technical report, Yale Center for Computational Vision and Control, 1998.
- [32] Katharina Stein, Daniel Fišer, Jörg Hoffmann, and Alexander Koller. Automating the generation of prompts for LLM-based action choice in PDDL planning. In *Proceedings of the 35th International Conference on Automated Planning and Scheduling (ICAPS)*, pages 250–259, 2025.
- [33] Planning.wiki. PDDL domain. <https://planning.wiki/ref/pddl/domain>, 2024.
- [34] RIP Tutorial Contributors. Reinforcement learning. <https://riptutorial.com/machine-learning/example/32668/reinforcement-learning>, 2023.
- [35] Tim Miller. Markov decision processes. <https://gibberblot.github.io/rl-notes/single-agent/MDPs.html>, 2023.
- [36] Junkai Yi and Xiaoyan Liu. Deep reinforcement learning for intelligent penetration testing path design. *Applied Sciences*, 13:9467, 2023.
- [37] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.
- [38] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv:1711.05101, 2019.
- [39] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

- [40] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.
- [41] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. In *Machine Learning Proceedings 1992*, pages 293–301, 1992.
- [42] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [43] Honghua Dong, Jiayuan Mao, Tian Lin, Chong Wang, Lihong Li, and Denny Zhou. Neural logic machines. arXiv:1904.11694, 2019.
- [44] Diego Perez-Liebana, Jialin Liu, Ahmed Khalifa, Raluca D. Gaina, Julian Togelius, and Simon M. Lucas. General video game ai: A multi-track framework for evaluating agents, games and content generation algorithms. *IEEE Transactions on Games*, 10:195–214, 2018.
- [45] Sainbayar Sukhbaatar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus. Intrinsic motivation and automatic curricula via asymmetric self-play. arXiv:1703.05407, 2018.
- [46] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired open-ended trailblazer (POET): Endlessly generating increasingly complex and diverse learning environments and their solutions. arXiv:1901.01753, 2019.
- [47] Rémy Portelas, Cédric Colas, Katja Hofmann, and Pierre-Yves Oudeyer. Teacher algorithms for curriculum learning of deep RL in continuously parameterized environments. arXiv:1910.07224, 2019.
- [48] Lilian Weng. Curriculum for reinforcement learning. <https://lilianweng.github.io/posts/2020-01-29-curriculum-rl/>, 2020.
- [49] M. Pawan Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. In *Proceedings of the 24th International Conference on Neural Information Processing Systems (NeurIPS)*, pages 1189–1197, 2010.
- [50] Aleksandrs Slivkins. Introduction to multi-armed bandits. arXiv:1904.07272, 2024.

- [51] Tabet Matiisen, Avital Oliver, Taco Cohen, and John Schulman. Teacher-student curriculum learning. arXiv:1707.00183, 2017.
- [52] Benoît Colson, Patrice Marcotte, and Gilles Savard. An overview of bilevel optimization. *Annals of Operations Research*, 153:235–256, 2007.
- [53] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. arXiv:1606.04671, 2022.
- [54] Xin Wang, Yudong Chen, and Wenwu Zhu. A survey on curriculum learning. arXiv:2010.13166, 2021.
- [55] Abhinav Shrivastava, Abhinav Gupta, and Ross Girshick. Training region-based object detectors with online hard example mining. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 761–769, 2016.
- [56] Xuan Zhang, Gaurav Kumar, Huda Khayrallah, Kenton Murray, Jeremy Gwinnup, Marianna J. Martindale, Paul McNamee, Kevin Duh, and Marine Carpuat. An empirical exploration of curriculum learning for neural machine translation. arXiv:1811.00739, 2018.
- [57] Guy Hacohen and Daphna Weinshall. On the power of curriculum learning in training deep networks. arXiv:1904.03626, 2019.
- [58] Jane X. Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z. Leibo, Remi Munos, Charles Blundell, Dharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn. arXiv:1611.05763, 2017.
- [59] Volkan Cirik, Eduard Hovy, and Louis-Philippe Morency. Visualizing and understanding curriculum learning for long short-term memory networks. arXiv:1611.06204, 2016.
- [60] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9:1735–1780, 1997.
- [61] Niels Justesen, Ruben Rodriguez Torrado, Philip Bontrager, Ahmed Khalifa, Julian Togelius, and Sebastian Risi. Illuminating generalization in deep reinforcement learning through procedural level generation. arXiv:1806.10729, 2018.
- [62] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving Rubik’s cube with a robot hand. arXiv:1910.07113, 2019.

- [63] Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. arXiv:1705.06366, 2018.
- [64] Sebastien Racaniere, Andrew K. Lampinen, Adam Santoro, David P. Reichert, Vlad Firoiu, and Timothy P. Lillicrap. Automated curricula through setter-solver interactions. arXiv:1909.12892, 2020.
- [65] Allan Jabri, Kyle Hsu, Ben Eysenbach, Abhishek Gupta, Sergey Levine, and Chelsea Finn. Unsupervised curricula for visual meta-reinforcement learning. arXiv:1912.04226, 2019.
- [66] Yash Shukla, Abhishek Kulkarni, Robert Wright, Alvaro Velasquez, and Jivko Sinapov. Automaton-guided curriculum generation for reinforcement learning agents. arXiv:2304.05271, 2023.
- [67] Amir Pnueli. The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 46–57, 1977.
- [68] Álvaro Torralba, Jendrik Seipp, and Silvan Sievers. Automatic instance generation for classical planning. In *Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling (ICAPS 2021)*, pages 376–384, 2021.
- [69] Daniel Ban and Dominik Sari. Learner-aware instance generation for classical planning via neural networks. Master’s thesis, Aalborg University, 2025.
- [70] Joshua Achiam and Shankar Sastry. Surprise-based intrinsic motivation for deep reinforcement learning. arXiv:1703.01732, 2017.
- [71] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, 1st edition, 2009.
- [72] Andy B. Yoo, Morris A. Jette, and Mark Grondona. Slurm: Simple linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing*, pages 44–60, 2003.
- [73] Rein Houthoofd, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. Vime: Variational information maximizing exploration. In *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, pages 1109–1117, 2016.
- [74] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. arXiv:1406.2661, 2014.

-
- [75] Daphna Weinshall, Gad Cohen, and Dan Amir. Curriculum learning by transfer learning: Theory and experiments with deep networks. arXiv:1802.03796, 2018.
 - [76] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL²: Fast reinforcement learning via slow reinforcement learning. arXiv:1611.02779, 2016.
 - [77] Jendrik Seipp, Álvaro Torralba, and Jörg Hoffmann. PDDL generators. <https://doi.org/10.5281/zenodo.6382173>, 2022.
 - [78] Jeffrey Elman. Distributed representations, simple recurrent networks, and grammatical structure. *Machine Learning*, 7:195–225, 1993.

Apéndice A

Blocksworld

Dominio Blocksworld

Dominio de *Blocksworld*

```
(define (domain BLOCKS)
  (:requirements :strips :typing)
  (:types block)
  (:predicates
    (on ?x - block ?y - block) (ontable ?x - block)
    (clear ?x - block) (handempty) (holding ?x - block))

  (:action pick-up
    :parameters (?x - block)
    :precondition (and (clear ?x) (ontable ?x) (handempty))
    :effect (and (not (ontable ?x)) (not (clear ?x))
                 (not (handempty)) (holding ?x)))

  (:action put-down
    :parameters (?x - block)
    :precondition (holding ?x)
    :effect (and (not (holding ?x)) (clear ?x)
                 (handempty) (ontable ?x)))

  (:action stack
    :parameters (?x - block ?y - block)
    :precondition (and (holding ?x) (clear ?y))
    :effect (and (not (holding ?x)) (not (clear ?y))
                 (clear ?x) (handempty) (on ?x ?y)))

  (:action unstack
    :parameters (?x - block ?y - block)
    :precondition (and (on ?x ?y) (clear ?x) (handempty))
    :effect (and (holding ?x) (clear ?y)
                 (not (clear ?x)) (not (handempty))
                 (not (on ?x ?y)))))
```

Problema Blocksworld

Ejemplo de un problema siguiendo el dominio anterior:

Problema de ejemplo de *Blocksworld*

```
(define (problem BW-rand-2)

  (:domain BLOCKS)
  (:objects b1 b2 - block)
  (:init
    (handempty)
    (ontable b1)
    (on b2 b1)
    (clear b2)
  )
  (:goal
    (and
      (on b1 b2))
  )
)
```

Glosario de acrónimos

ACG *Automated Curriculum Generation.*

ACL *Automated Curriculum Learning.*

ADR *Automatic Domain Randomization.*

AUC *Area Under the Curve.*

CARML *Contrastive Automatic Reinforcement Meta-Learning.*

CL *Curriculum Learning.*

CUDA *Compute Unified Device Architecture.*

DL *Deep Learning.*

GAE *Generalized Advantage Estimation.*

GPU *Graphics Processing Unit.*

GVGAI *General Video Game AI.*

HEM *Hard Example Mining.*

IA *Inteligencia Artificial.*

LSTM *Long Short-Term Memory.*

LTL *Linear Temporal Logic.*

MDP *Markov Decision Process.*

ML *Machine Learning.*

MLP *Multilayer Perceptron.*

NeSIG *Neuro-Symbolic Instance Generator.*

NLM *Neural Logic Machine.*

NP *Nondeterministic Polynomial time.*

PDDL *Planning Domain Definition Language.*

POET *Paired Open-Ended Trailblazer.*

PPO *Proximal Policy Optimization.*

PSPACE *Polynomial Space.*

RL *Reinforcement Learning.*

SPL *Self-Paced Learning.*

TFG Trabajo de Fin de Grado.